

The AIEN Papers

M. Drake Stapleton

October 2026

Preface

I am writing these papers because machine intelligence needs a foundation it can stand on, and I want that foundation to be public.

The stack these papers describe is simple to state. Conversation produces proposals. Semantic objects make proposals explicit. Verification produces evidence. Crumbs make verified knowledge reusable. The Crumbline preserves its history. The Turing measures whether a new explanation earns its complexity on unseen evidence. T/J measures how physically expensive that gain was.

On measurement, I keep two quantities distinct. T_{ideal} is the complexity-adjusted held-out log-loss improvement: what the mathematics says the gain is. T_{real} is the complexity-adjusted improvement in an actual reference-coded bitstream: what an encoder really produced. On September 29, 2026, TY-1+TY-2 recorded the first profile-bound idealized Turing result: +2,559,679.825 T_{ideal} . H1, the claim that T_{ideal} approximates T_{real} within the declared bound, remains open until the reference coder closes the bridge. I state the result's standing plainly: TY-1+TY-2 is an internally replayed descriptive result under historical profile V0. The experiment record reports separation of fitting, selection, and evaluation seeds, but independent blinding was not established, and the original corpus is not publicly downloadable. It does not constitute full qualification under the current protocol. The full evidence ledger is in the formal paper.

The Crumb Doctrine is at v1.4: the epistemic constitution. Omega proceeds next to Omega Core Semantics v0.1. The eighth paper, the Atemporal AIEN Postulate v1.1, proposes that an AIEN computation is defined by causal dependence rather than by time; it is a separate postulate, and the seven papers above stand unchanged.

The doctrine is becoming executable

On October 2, 2026, the Crumb Doctrine gained its first executable spine across four repositories. `aien-architecture` merged ADR 0029 (“Verified Crumbs”), making Omega’s verified library the dependency-resolution authority. `aien-protocols` merged `VerifiedCrumbV1`: the protocol spec with 17 golden vectors, malformed and refusal vectors, a C checker, and a failure taxonomy (`MISSING_RECEIPT`, `RECEIPT_HASH_MISMATCH`, `STALE_RECEIPT`, `UNVERIFIED_DEPENDENCY`, `UNKNOWN_VERIFIER_PROFILE`, `VERIFIER_TOO_OLD`). `omega` hardened `omega_library_insert` to refuse missing or all-zero receipt hashes, require dependencies to already exist, and restrict `BOOTSTRAP` admission to genesis alone, with no flag able to unlock it. `aien-sovereign-core` merged `aien-closure`: a `verify-closure` binary plus CI that fails closed on unverified dependencies, missing receipts, hash mismatches, unpinned dependencies, cycles, stale receipts, undeclared imports, tainted artifacts, and unknown verifier profiles.

The pipeline is now: protocol, then Omega admission, then dependency closure verification, then CI enforcement.

I state the boundary honestly. This establishes verified artifacts, receipts, and closure checks. It does not yet establish the full Omega language semantics, the five identifiers as one universally enforced runtime model, the complete sealed foreign-crumb ABI, or the full check/build/verify/seal toolchain. Those remain proposed or partial. `aien-test Slice C` (digest cache) is proposed as PR #170, not yet merged.

I name the identifiers as the Omega language defines them: `SEMANTIC_ID`, `SPEC_ID`, `CLAIM_ID`, `RECEIPT_ID`, and `SEAL_ID`. An earlier draft of this preface said “four”; the language defines five,

and I correct the number openly rather than silently. Enforcing all five as one runtime model is itself proposed, not done.

The verdict I draw: receipt-and-closure checking is implemented and exercised in CI; repository-wide enforcement is not yet active. The doctrine is becoming executable, but it is not yet a property the machine enforces.

Where the claims stand

I collect the major claims and mechanisms of the eight papers in one table, with a single status each. The labels mean: *proved*, closed by proof under stated assumptions; *observed*, recorded in an experiment; *implemented*, running in the toolchain; *partial*, implemented in part but not enforced repository-wide; *proposed*, defined but not yet built or run. Anything I cannot verify from the papers is marked *proposed* rather than given a stronger label.

Item	Status
The Turing measurement protocol, as defined in the formal paper	proposed
TY-1+TY-2 idealized result (+2,559,679.825 T _{ideal} , September 29, 2026)	observed
T _{real} bridge (H1: T _{ideal} approximates T _{real} within the declared bound)	proposed
Claim A (the conditional theorem)	proved, conditional on one-way functions
Claim B (the unconditional separation)	proposed
Claim C / CEXP-1 (the AIEN program-reuse hypothesis)	proposed
Crumb receipts and dependency-closure enforcement	partial
The Omega language	proposed
The Stapleton Doctrine	philosophical essay, not a scientific claim
The Atemporal AIEN Postulate (causal semantics without clocks)	proposed

Contents

Preface	2
1. The Turing Brief (public explanation)	5
2. The Formal Turing Paper v1.13 (scientific specification)	9
3. The Emergence Companion v1.7 (engineering history)	40
4. The Turing Discovery Gap Conjecture rev 9	69
5. The Stapleton Doctrine v1.5 (philosophy)	80
6. The Crumb Doctrine v1.4 (epistemic constitution)	91
7. The Omega Language (enforcement language)	97
8. The Atemporal AIEN Postulate v1.1 (postulate)	107

The Turing Brief

The entry point. What the Turing is, in plain words.

The Turing: A Unit of Verified Explanatory Gain

M. Drake Stapleton

September 29, 2026

Status: Research hypothesis, v1.4, revised October 2, 2026. Originally published September 29, 2026.

v1.4 revision note: reconciliation pass with the longer papers. Clarifies result status: TY-1+TY-2 is an internally replayed descriptive result; independent blinding is not established.

1 The problem

Artificial intelligence has no unit for understanding gained.

We measure tokens per second. We measure watts. We measure benchmark accuracy. None of these measure how much a machine actually figured out, or what it cost in energy to figure it out.

A model can burn a megawatt to memorize answers. A model can sip power and discover structure that keeps predicting. Today’s metrics cannot tell those two apart in any principled way. Accuracy per joule gets close, but accuracy is not understanding: a lookup table can be accurate.

We propose a unit for a measurable component of the missing quantity.

2 The proposal

The Turing (T) is a unit of verified explanatory gain.

1 T = 1 bit of net held-out description-length improvement, relative to a declared baseline and a versioned measurement profile, after paying for the explanation itself.

In symbols, for baseline B , candidate explanation M , sealed held-out observations D , declared shared background G , and measurement profile P :

$$T(M; B, D, G, P) = [L(B|G) + L(D|B, G)] - [L(M|G) + L(D|M, G)] \quad (1)$$

the bits to describe each explanation plus the bits to describe the data given it, with the shared background declared rather than smuggled in.

Measurement takes two forms. T_{ideal} is the complexity-adjusted held-out log-loss improvement: idealized, requiring no actual entropy coder. T_{real} is the complexity-adjusted improvement in an actual reference-coded bitstream. An idealized result is not a realized compression; the bridge between the two is a claim to be tested, not assumed.

Turing yield is explanatory gain per unit energy: T/J (Turings per joule).

3 Why the explanation must pay for itself

A giant lookup table can predict perfectly on the data it stores. Whether that counts for anything depends on what the table had to pay to store it.

Example: a candidate saves 3,000,000 prediction bits but costs 8,000,000 bits to describe. Net: $-5,000,000$ Turings under profile P . It loses.

Another candidate saves 50,000 bits and costs 400 to describe. Net: $+49,600$ Turings under profile P . It discovered compact, reusable structure.

The second candidate demonstrated greater net held-out explanatory compression under this profile. The mechanism is the complexity charge: it penalizes stored information that does not earn its cost on held-out observations. Under the memorization bound of the formal paper (Lemma 2), pure memorization, explicitly charged, cannot outperform a cheaper equal default on held-out data. That is a deliberately weak claim, and it needs stating carefully: a table-based predictor can legitimately generalize. If a table's entries earn their description cost on observations the table never saw, the table scores well. What is penalized is storage that does not pay its way, not tables as such.

We propose that finding compact, reusable structure which keeps predicting is one measurable component of understanding.

4 The rules that keep it honest

- Held-out data only. Never training data.
- The baseline is declared up front, not chosen after the fact.
- Scores are only comparable under the same measurement profile. No context-free leaderboards.
- Physical energy accounting must conserve: you cannot attribute more joules than were measured.
- Informational accounting must conserve: you cannot credit more Turings than were produced.
- Every measurement is replayable from raw evidence by a third party.

5 What this is not

It is not a replacement for the bit. Bits measure information; Turings measure verified explanatory gain, expressed in bits the way a darwin expresses evolutionary change as a rate (a factor of e per million years).

It is not an intelligence score. A narrow trick can score Turings without being generally intelligent. The claim is deliberately narrow: under profile P , candidate C reduced sealed held-out description length by a stated number of bits relative to baseline B .

6 Standing on giants, measuring something new

The mathematics descends from Shannon's information theory, Solomonoff's algorithmic probability, Kolmogorov complexity, and Rissanen's Minimum Description Length principle (1978), whose two-part code $L(H) + L(D|H)$ is the direct ancestor of the formula above.

None of them named a unit for it. Description length has always been reported in plain bits, which is why a bit of memorization and a bit of genuine explanation look identical on paper.

The contribution is not the name. A name by itself measures nothing. The contribution is the measurement protocol: sealed held-out data, an explanation that pays for itself in the description-length accounting, and a declared profile that makes two reported numbers comparable. There is precedent for naming a unit of this kind: J. B. S. Haldane defined the **darwin** in 1949 as a unit of evolutionary change, named for Charles Darwin. We define the **Turing** as a unit of explanatory change.

Related recent work (for example, the Stanford Hazy Research paper “Intelligence per Watt” (arXiv:2511.07885), November 2025) measures task accuracy per unit power, and per unit energy in the same work. That is useful and different: accuracy is not explanatory gain, and a memorized answer is accurate. The Turing measures what the accuracy metrics cannot: whether the machine discovered structure that keeps predicting.

7 Terminology

- **TURING**: the existing AIEN/Omega evidence and decision-record program.
- **Turing (T)**: the unit proposed here.
- T_{ideal} : complexity-adjusted held-out log-loss improvement; idealized, with no entropy coder required.
- T_{real} : complexity-adjusted improvement in an actual reference-coded bitstream.
- **Turing yield**: T/J , explanatory gain per joule.

8 The claim

To our knowledge, this brief is the first publication of the Turing as a unit of verified explanatory gain, and of Turing yield (T/J) as a measure of the physical efficiency of verified explanatory gain. Published September 29, 2026, by the AIEN project.

We invite everyone to use it. If you report Turings, publish your baseline, your measurement profile, and your evidence, so anyone can recompute your number. That is the entire price of admission.

9 What comes next

The first profile-bound idealized result is recorded: $TY-1+TY-2, +2,559,679.825 T_{\text{ideal}}$ under the measurement profile declared for that experiment. That number is an internally reported held-out result under the historical V0 profile, not a fully qualified measurement under the current protocol. The reference-coder bridge remains open: H1, the claim that $T_{\text{ideal}} \approx T_{\text{real}}$ within the declared bound, is not yet verified. Results will be published as they verify, including failures.

One measurable component of understanding is the discovery of compact structure that keeps predicting what happens next. Now it has a unit.

The Formal Turing Paper

The scientific center of gravity. Definitions, proofs, protocol, and falsification criteria.

Computing Machinery and Understanding From the Imitation Game to the Turing: An Operational Measure of Generalized Explanatory Compression

M. Drake Stapleton

October 2, 2026

v1.14 preprint (not peer reviewed)

Companion: “Computing Machinery and Understanding: The Turing as a Measure Emerging from AIEN” (v1.7) gives the engineering history the measure emerged from: the AIEN/Omega/FORGE architecture, the public evidence receipts, and the first experimental proposals. This preprint is the formal reference: definitions, proofs, protocol, related work, measurement-validity criteria, and scientific falsification criteria.

Abstract

In 1950, Alan Turing confronted the question “Can machines think?” not by attempting to define thought directly, but by replacing the question with an operational experiment: the imitation game. That methodological move transformed an ill-posed philosophical dispute into an empirical research program.

This paper proposes an analogous reformulation for a contemporary question: **does a machine understand?** I propose that one measurable component of understanding can be operationalized as the discovery of compact structure that continues to predict previously unseen observations. The proposal combines Shannon information, predictive coding, Minimum Description Length, empirical verification, and physical resource accounting.

I define a new derived quantity, **Turing gain**, denoted $\mathcal{T}$. One Turing (1 T)¹ is one bit of **net held-out explanatory compression**: the net reduction in held-out total description length produced by a specified model, program, theory, or abstraction relative to a declared baseline, after including the description cost of the explanatory structure itself. For baseline B , candidate explanation M , sealed held-out observations D , declared shared background G , and measurement profile P ,

$$\mathcal{T}(M; B, D, G, P) = [L(B | G) + L(D | B, G)] - [L(M | G) + L(D | M, G)].$$

Positive gain establishes a lower total description length than the declared baseline on the specified held-out data, under the declared code and background. A negative gain indicates that the candidate’s total description length exceeds the baseline’s: the explanation costs more complexity than the structure it captures, predicts worse than the baseline, or both. A smaller model can lose through worse predictions alone, and positive gain can come from a cheaper representation with identical predictions. Turing gain is therefore permitted to fail: a theory can score negative Turings.

The Turing is not proposed as a replacement for the bit, the joule, the watt, or any SI quantity. It is a protocol-bound measure of **generalized explanatory gain**. The claim is

¹The symbol T denotes the Turing unit proposed here. It is distinct from the SI tesla symbol; context and the $\mathcal{T}_{\text{ideal}}$ and $\mathcal{T}_{\text{real}}$ subscripts disambiguate.

deliberately narrow: the Turing measures one operational component of understanding (its predictive and explanatory-efficiency dimension), not understanding itself. It does not measure causal direction, semantic grounding, agency, or value alignment; Section 9 states these exclusions explicitly. When paired with physical measurements, it permits derived quantities such as the Turing rate ($\mathcal{T}/s$), the specific Turing gain ($\mathcal{T}$ per held-out observation), and Turing yield ($Y_{\text{eval}}, Y_{\text{discovery}}$), allowing the informational productivity of different algorithms, architectures, processors, accelerators, and potentially non-digital substrates to be compared under controlled experimental conditions. The yield quantities address a current environmental blind spot: the field has no standardized measure of net held-out explanatory compression per joule. The unit’s discipline (declared baselines, sealed held-out data, recomputable evidence, declared shared background) is also an honesty discipline, and it gives alignment research a measurable target: verified understanding rather than performed competence. The limits are stated explicitly: the Turing does not directly measure safety, consciousness, or moral worth.

The proposal is explicitly testable at two levels, and preregistered criteria are given for both: measurement-validity criteria, which decide whether a reported number is a sound measurement under its profile, and scientific falsification criteria, which state what would fail the claim that the statistic is a useful operational indicator of understanding. At the measurement level: if predictive models do not produce corresponding compression gains, if candidate complexity erases those gains, if held-out improvements fail to reproduce, or if physical attribution cannot satisfy conservation constraints, the claimed measurements fail.

Turing taught us to replace an unanswerable question with an operational one. I ask whether machines can discover explanations that make unseen reality less surprising.

1 Introduction

Turing taught us to replace an unanswerable question with an operational one. I ask whether machines can discover explanations that make unseen reality less surprising.

I propose to consider the question: **can a machine understand?**

The customary approach would begin by defining the words *machine* and *understand*. This is unlikely to be profitable. Turing did not settle the meaning of “thinking” by definition. He proposed an observable game whose outcome could be discussed without agreement on the metaphysics of thought [1]. I attempt a similar substitution. Instead of asking “does this machine understand the world?”, I ask:

Has this machine discovered a compact explanation that makes previously unseen parts of the world less surprising?

This question admits measurement. A predictive model implicitly defines a code: for a sequence $D = (x_1, \dots, x_n)$,

$$L(D \mid M, G) = - \sum_{t=1}^n \log_2 p_M(x_t \mid x_{<t}, G)$$

is its idealized predictive description length under model M given the declared shared background G (Section 5.7), where $p_M(x_t \mid x_{<t}, G)$ is the probability the model assigns to observation x_t given the preceding observations and the background. Shannon’s theory established the mathematical relationship between probability, information, and the limits of lossless coding [2]: a model that predicts a sequence more accurately can, in principle, describe that sequence using fewer bits. Thus prediction $\longleftrightarrow$ probability $\longleftrightarrow$ description length. The surprising event is expensive; the predictable event is cheap; regularity is compressible.

But compression of data alone is insufficient, because a machine could memorize an enormous lookup table: excellent predictions, uneconomical explanation. The model itself must also be paid

for. The relevant quantity is the two-part total description length $L(M \mid G) + L(D \mid M, G)$, in the Minimum Description Length tradition [5, 6]. A lookup table costing eight million bits that reduces residual prediction length by only three million bits has a negative net contribution; a 400-bit rule that reduces unexplained held-out data by 50,000 bits has discovered something compact and reusable.

1.1 What the Turing is not: a renamed bit

An immediate objection deserves an answer before the formalism: is the Turing “simply renaming bits”? It is not, and the distinction is load-bearing for everything that follows. Three things must be kept distinct:

- **Quantity.** Turing gain, $\mathcal{T}(M; B, D, G, P)$: the net held-out explanatory advantage of candidate M over baseline B on sealed observations D , conditional on declared shared background G , under measurement profile P (Definition 2).
- **Reporting unit.** The Turing, T : one bit of net held-out explanatory advantage under a declared protocol.
- **Measurement.** A reported value together with its profile: e.g., 84,220 T under profile P . A bare number without its profile is scientifically incomplete.
- **Rate reporting.** The headline number is the **specific Turing gain**: $\mathcal{T}$ divided by the number of sealed held-out observations, in Turings per observation. Raw totals are compared only at fixed sample size. A bare total without its n is as incomplete as a bare number without its profile.

Dimensionally, Turing gain is measured on a bit scale. Scientifically, it is a constrained statistic, not arbitrary information content: the difference between two controlled total description lengths, with the candidate’s own complexity charged, the data sealed, the baseline declared in advance, and the physical attribution conserved. Giving a particular operational meaning to a derived quantity is standard scientific practice. Haldane’s darwin [8] did exactly this for evolutionary rate. It is not the discovery of a new fundamental SI dimension. The Turing claims no such dimension. It claims a disciplined way of counting.

1.2 Turing’s own precedent: the ban and the deciban

The name is not decoration. At Bletchley Park, Turing needed a working unit for the weight of evidence, and he coined one: the *ban*, the base-10 log-likelihood-ratio unit, with the *deciban* as its tenth, named by explicit analogy with the decibel [16]. Cryptanalysts scored hypotheses in decibans because evidence had to be added, compared, and acted on, and that required a unit. The Turing proposed here is the direct descendant of that move: one bit of net held-out explanatory advantage, scored under a declared protocol, so that claims about machine understanding can be added, compared, and acted on. The ban measured evidence for a hypothesis; the Turing measures evidence that an explanation has earned its keep.

1.3 Benchmarks measure performance; the Turing measures explanatory gain

Contemporary machine learning evaluates itself chiefly through benchmarks: fixed test sets on which systems compete for accuracy. This culture has produced real progress and real pathologies. Web-scale pretraining corpora swallow test sets, so “held-out” is often a fiction: contamination must

now be measured per benchmark [39], models reconstruct missing benchmark items at rates far above chance [41], and training data is extractable verbatim [37]. Bender and Koller [34] dramatized the deeper issue with an octopus that predicts telegraphic text superbly while possessing no model of the world the text describes: excellent string prediction with zero understanding. Bender et al. [36] named the resulting systems stochastic parrots: fluent stitchers of linguistic form without reference to meaning. McCoy et al. [43] showed empirically that large language models exhibit probability-shaped rather than understanding-shaped error patterns: GPT-4 decoded a simple cipher at 51% accuracy when the answer was a high-probability sentence versus 13% when it was low-probability, on a deterministic task where probability should not matter.

The fair counter must be stated. Li et al. [38] showed that a transformer trained only on Othello move sequences developed a nonlinear internal representation of board state, and that interventions on that representation causally changed its legal-move predictions: prediction pressure *can* induce causally efficacious world models. Vafa et al. [44] replied that decodable probes do not establish globally coherent, causally usable models. Both can be true: prediction can induce models, but predictive success does not guarantee them.

The Turing sits on this fault line by design. It rewards predictive improvement after accounting for model-description cost, which is the compression leg of understanding. The causal-usability leg must be measured separately, through interventional held-out regimes (Sections 5.4 and 5.6), and the paper claims nothing beyond the leg it measures. What the Turing refuses is the benchmark culture’s central conflation: a score on a possibly memorized test is not evidence of understanding. A positive Turing gain is evidence of exactly one thing: the candidate found structure that pays for itself on data it never saw.

1.4 The arc: from prediction pressure to physical law

Prediction-error minimization is not an arbitrary choice of objective. It is the design principle nature itself uses. Friston [25] argued that organisms minimize variational free energy, an upper bound on sensory surprisal, by updating generative models and by acting to change what they sense; Clark [28] developed the predictive-brain synthesis. Kirby, Cornish, and Smith [23] showed in the laboratory that when languages pass through a learnability bottleneck across generations of learners, they become more structured: “design without a designer,” with compressibility pressure generating the very structure we associate with understanding. Delétang et al. [40] demonstrated the engineering counterpart: large language models are powerful general-purpose compressors, with Chinchilla 70B beating PNG on images and FLAC on audio, because prediction and compression are two faces of one quantity.

But prediction pressure alone is not enough; it must be aimed at the right world. A system trained only on human text inherits humanity’s signal with all of its noise: form without meaning, in Bender and Koller’s phrase. The position of this paper is that the epistemic foundation of machine understanding should be physical law, not human text. Physics-informed neural networks [31] are the flagship demonstration: embedding partial-differential-equation residuals directly into the loss constrains the model by nature’s regularities, so its held-out compression must track those regularities rather than textual correlations. Sutton’s Bitter Lesson [32] supplies both the alliance and the warning. The alliance: general methods that leverage computation beat hand-built human knowledge in the long run, which is an argument for letting a neutral arbiter, held-out compression, decide. The warning: do not privilege human-legible explanations, since the profile should let simple scalable predictors compete on equal footing with elegant theories and let the bits decide.

The thesis, stated plainly: **build what the machine knows on physics, the foundation of all existence; keep purpose, values, and moral direction a human responsibility.**

The Turing measures the first half: the epistemic dimension, how much of nature’s regularity the machine has captured per unit of explanation cost. It is silent on the second half by design (Section 9). EXP-002, the Stochastic Law Discovery experiment, is the concrete test of the first half: the machine must discover stochastic physical structure from sealed trajectories, not imitate text about that structure.

2 Why This Measure Matters

The formalism below is motivated by three concerns. They are stated here as motivation and interpretation, not as substitutes for rigor: the definitions, lemmas, protocol, measurement-validity criteria, and scientific falsification criteria that follow stand or fall on their own. And the limits of Section 9 apply throughout. In particular, the Turing does not directly measure safety, consciousness, or moral worth.

2.1 Environmental safety: a blind spot with a measure

Computation does not occur outside physics, and machine intelligence is scaling its energy consumption faster than the ability to account for what that energy buys. The field has mature metrics for performance per watt, but no standardized measure of net held-out explanatory compression per joule. That is an environmental blind spot: without a measure of informational productivity, there is no principled way to prefer the algorithm that discovers the same structure for a tenth of the energy, and no way to make that preference legible to anyone outside the laboratory.

Turing yield (Definition 4) is proposed as that measure. It does not measure carbon emissions, ecological harm, or the full lifecycle cost of hardware; it measures verified net held-out explanatory compression per measured joule, which is the quantity a researcher or operator can actually control when choosing algorithms, architectures, and substrates. Hypothesis H5 of the experimental protocol (Section 5) is the environmental hypothesis in falsifiable form: semantically equivalent realizations should sometimes exhibit materially different Turing yields. If that hypothesis holds, Turing yield becomes an instrument for steering the field toward informational productivity per joule: the quantity that must improve if machine understanding is to scale within physical limits.

2.2 Humanity: an honesty discipline

The unit’s rules are an honesty discipline. Baselines are declared before measurement, not chosen after. Held-out data is sealed before the candidate is constructed or selected. Shared background is declared, not smuggled: every pretrained model, corpus, and prior artifact in the causal chain of producing the candidate is versioned in the profile (Section 5.7), so a candidate cannot rent a gigantic undeclared model and bill the protocol for the savings. Every number must be recomputable by a stranger from published evidence. And theories are permitted negative Turings: a candidate that costs more complexity than the structure it captures scores below zero, in public, by construction.

This rewards verified explanatory gain over fluent performance. A system optimized for Turings cannot coast on pure memorization: under Lemma 2’s assumptions, a pure memorizer, explicitly charged, cannot earn positive Turings against a cheaper equal default on held-out data. That discipline is narrow by design. It does not forbid stored structure that earns its cost on held-out observations; a table-based predictor can legitimately generalize, and the first reported candidate (Section 7) is reported as one, within the limits stated there (internally replayed; independent blinding not established). A system also cannot cherry-pick its baseline or hide its failures, because

the measurement procedure exposes them as negative scores. A culture that can measure verified explanatory gain can demand it; a culture with only benchmark leaderboards gets what it measures. The Turing is a small piece of infrastructure for the former.

2.3 Alignment: a measurable target

Alignment research asks, among other things, whether a system’s apparent competence reflects genuine understanding or fluent performance. That question is hard to aim at because “understanding” has had no operational measure. The Turing operationalizes one measurable component of understanding: the discovery of compact structure that continues to predict unseen observations, and thereby gives alignment work something quantitative to target and to audit.

The mechanism is the complexity charge: a system that must pay for its own complexity, in the open, on sealed data, faces a measured price for faked understanding. Memorized performance is exposed by $L(M \mid G)$ (Lemma 2); uncalibrated confidence is exposed by $L(D \mid M, G)$; double-counted credit is blocked by informational conservation (Definition 6). None of this measures safety or intent directly. A well-calibrated predictor can still be misused, and Section 9 states the limits plainly. What it contributes is a falsifiable evidentiary criterion for one operational component of understanding: a system’s claimed understanding should survive contact with unseen reality after paying for itself.

3 Formal Definitions

Every symbol is defined before use. Numbered definitions are binding: a reported Turing measurement is valid only if each definition’s requirements are satisfied and recorded in the measurement profile.

Definition 1 (Total description length). *Let M be a candidate explanation: a model, program, theory, abstraction, or explanatory structure. Let $D = (x_1, \dots, x_n)$ be a finite sequence of observations over a finite alphabet. Let p_M be a computable predictive distribution exposed by M , assigning $p_M(x_t \mid x_{<t}, G)$ to each observation given its predecessors and the declared shared background G . Then:*

- *G is the declared shared background: the versioned, cost-accounted set of pretrained models, corpora, interpreters, and toolchains available to the decoder (Section 5.7); every codelength is conditional on G .*
- *$L(M \mid G)$ is the length in bits of M under a declared, computable encoding convention (Assumption A1), with the declared shared background G available to the decoder: only what M adds beyond G is charged.*
- *$L(D \mid M, G) = -\sum_{t=1}^n \log_2 p_M(x_t \mid x_{<t}, G)$ is the idealized predictive codelength of D under M given G (Lemma 1).*
- *The total description length, conditional on G , is $DL(M, D \mid G) = L(M \mid G) + L(D \mid M, G)$.*

Definition 2 (Turing gain). *Turing gain operationalizes one measurable component of understanding: net held-out explanatory compression. Let B be a declared baseline explanation, M a candidate explanation, D a sequence of observations that were not available when M was constructed or selected (sealed held-out data; see Section 5), G the declared, versioned, cost-accounted shared background (pretrained models, corpora, interpreters, toolchains; Section 5.7), and P a measurement*

profile satisfying Definition 3. The profile may record G within P , but G appears in the notation explicitly: every codelength is conditional on the shared background, and the conditioning set travels with the score. The Turing gain is

$$\mathcal{T}(M; B, D, G, P) = DL(B, D \mid G) - DL(M, D \mid G)$$

equivalently,

$$\mathcal{T}(M; B, D, G, P) = [L(B \mid G) + L(D \mid B, G)] - [L(M \mid G) + L(D \mid M, G)].$$

Two reporting kinds are distinguished. $\mathcal{T}_{\text{ideal}}$ is the complexity-adjusted held-out log-loss improvement: the net description-length difference computed from idealized predictive codelengths, requiring no actual entropy coder. $\mathcal{T}_{\text{real}}$ is the complexity-adjusted improvement in an actual reference-coded bitstream: the net difference between the bit lengths produced by coupling B and M to the profile's declared reference coder on D . Hypothesis $H1$ tests whether $\mathcal{T}_{\text{ideal}} \approx \mathcal{T}_{\text{real}}$ within the profile's declared bound for the tested regime. Until $H1$ passes for a regime, idealized results in that regime may not be reported as $\mathcal{T}_{\text{real}}$; they are reported in $\mathcal{T}_{\text{ideal}}$. The **reporting unit** is the Turing, T : one Turing is one bit of net held-out explanatory compression under a declared protocol. A **measurement** is a reported value together with its profile (e.g., 84,220 T under profile P). $\mathcal{T} > 0$ means the candidate's total description length is smaller than the baseline's, whether through better prediction, lower model cost, or both; $\mathcal{T} = 0$ means no net explanatory gain; $\mathcal{T} < 0$ means the candidate's total description length is larger, through excess complexity, worse predictions, or both. A smaller candidate can score negative through worse predictions alone, and positive gain can come from a cheaper representation with identical predictions. The **specific Turing gain** is $\mathcal{T}(M; B, D, G, P)/n$ for $n = |D|$ sealed held-out observations, in Turings per observation; it is the headline reported number, and raw totals are compared only at fixed n . Writing the gain per observation makes its n -dependence explicit: $\mathcal{T}/n = [L(D \mid B, G) - L(D \mid M, G)]/n + [L(B \mid G) - L(M \mid G)]/n$. The model-cost term decays as $1/n$, and the predictive term need not be stationary in n , so specific gain is a useful normalization for comparison, not a fully sample-size-independent constant. The profile additionally reports $\mathcal{T}$ under two to three reasonable alternative baselines (**baseline_sensitivity**), so that a gain cannot be an artifact of a weak B .

Definition 3 (Measurement profile). A Turing measurement is reported only together with a measurement profile: a versioned record specifying at minimum the fields in Table 1. Gains are comparable only under the same profile version. A bare number of Turings without a profile is scientifically incomplete.

The profile has two layers, and they must not be confused. The *comparison protocol* states what is scored and how: the observation source, the split, the baseline, the candidate-encoding and model-cost codes, the reference coder, the codelength method, the probability representation and quantizer, the evaluation procedure, and the sensitivity analyses. The *execution manifest* states how the scoring was physically performed: software versions, the machine and hardware on which it ran, the energy instrumentation and its calibration, and the attribution rule. Gains are comparable only under the same comparison protocol. H5-style cross-hardware comparisons and reimplementations deliberately vary the execution manifest while holding the comparison protocol fixed; this does not contradict the identical-profile requirement, because the requirement binds the comparison protocol, not the machine it runs on.

Definition 4 (Turing yield and Turing rate). Let E_{eval} be the energy in joules consumed by the evaluation run that produced M 's predictions on D , measured by the profile's energy instrumentation

and attributed under Definition 5. For discovery experiments, let E_{search} , E_{train} , $E_{\text{synthesis}}$, and E_{verify} be the measured energies of the discovery phases. Then:

$$Y_{\text{eval}} = \frac{\mathcal{T}(M; B, D, G, P)}{E_{\text{eval}}} \quad \text{and} \quad Y_{\text{discovery}} = \frac{\mathcal{T}(M; B, D, G, P)}{E_{\text{search}} + E_{\text{train}} + E_{\text{synthesis}} + E_{\text{verify}} + E_{\text{eval}}}.$$

Y_{eval} is the evaluation yield. $Y_{\text{discovery}}$ is the discovery yield, and it is the stronger quantity: “understanding gained per joule burned” must include the cost of discovery, since a model might require 10 MWh to discover and 1 kJ to evaluate, and reporting only evaluation yield would misrepresent its efficiency. Let s be the wall-clock duration of the evaluation run in seconds; $\mathcal{T}/s$ is the Turing rate. None of these asserts that energy has been converted into intelligence in any thermodynamic sense; they state that under the given experiment and profile, the computation produced the stated bits of verified explanatory gain per unit of physical resource consumed. Note the fit: AIEN/Omega is designed to account for synthesis, verification, and realization costs rather than treating evaluation as the whole computational process. The $Y_{\text{discovery}}$ denominator requires exactly that accounting. Physical energy instrumentation exists: the AIEN/Omega R15 performance harness completed its qualification on the DGX Spark (16 of 16 gates PASS, merged as pull request 67 on 2026-09-29, with package energy measured per operation in the machine-readable receipt). The first profile-bound idealized Turing result has now been recorded separately (pull request 85: a specific gain of about $0.44 T_{\text{ideal}}$ per observation, totaling $+2,559,679.825 T_{\text{ideal}}$ over about 5.8 million observations, under measurement profile V0; Section 7), but the R15 energy measurements have not yet been joined to a Turing-gain receipt; no yield figure (T/J) is reported until that joining exists.

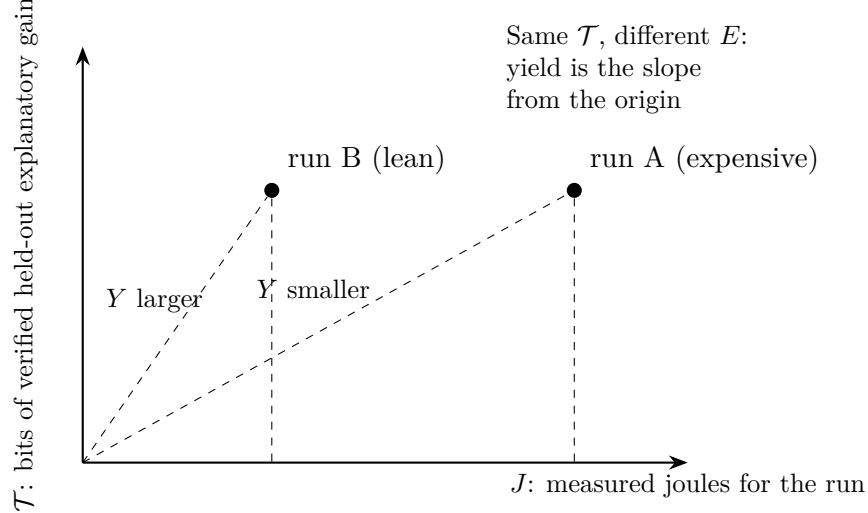


Figure 1: Conceptual illustration of the distinction between Turing gain $\mathcal{T}$ and Turing yield $Y = \mathcal{T}/E$ (Definition 4). Two runs can produce the same net held-out explanatory gain while consuming different measured energy; the yield discriminates between them, which is the environmental hypothesis H5. Conceptual illustration, not empirical data.

Definition 5 (Energy attribution). Let E_0 be the measured baseline energy of the machine at rest, E_A the measured energy with process A running in isolation, E_B with process B running in isolation, and E_{AB} with both running concurrently. The four measurements are taken over identical

measurement intervals and matched workloads: the same task content, the same ambient conditions, and the same background services, with only the presence of processes A and B varying. Define incremental consumptions $e(A) = E_A - E_0$, $e(B) = E_B - E_0$, $e(A, B) = E_{AB} - E_0$. The interaction term is

$$I_{AB} = E_{AB} - E_A - E_B + E_0.$$

If $I_{AB} > 0$, co-execution required additional incremental energy relative to isolated execution (contention); if $I_{AB} < 0$, some cost was shared or utilization improved. A negative interaction term is a signed accounting saving, not a negative quantity of physical energy. An attribution rule assigns each joule of E_{AB} to at most one of: exclusive use, shared use, interaction overhead, or unattributed; never concealing the interaction term by assigning every joule to an individual process. The profile must declare a bound on the permissible unattributed fraction of measured energy; beyond that bound, accounting closure is uninformative and no yield may be reported.

Definition 6 (Conservation constraints). *Two accounting rules are binding on every reported measurement:*

- (Physical conservation.) *Attributed energy must reconstruct measured energy within the profile’s declared uncertainty:*

$$E_{\text{measured}} \approx E_{\text{exclusive}} + E_{\text{shared}} + E_{\text{interaction}} + E_{\text{unattributed}}.$$

A resource attribution system may not allocate more energy than was physically measured.

- (Informational conservation.) *Explanatory credit must not be duplicated. If cooperating processes jointly produce one candidate worth $\mathcal{T}$ Turings, the sum of Turings credited across processes must not exceed $\mathcal{T}$. Shared explanatory gain may remain shared; it may not be multiplied.*

4 Coding Assumptions

This section separates what is proved, what is assumed, and what is defined. The claims are kept deliberately modest: every lemma below is genuinely provable, and the assumptions are stated as assumptions rather than dressed as results.

Assumption A1 (Computable encoding for $L(M | G)$). $L(M | G)$ is the bit-length of M under a specified, *computable* encoding convention declared in the measurement profile, with the declared shared background G available to the decoder (e.g., the length of a serialized artifact plus its declared decoder, or a stated two-part code). It is explicitly *not* uncomputable Kolmogorov complexity. Consequence: Turing scores are convention-relative. Two measurements are comparable only under the same profile, which fixes the convention. This is a feature, not a defect: it is what makes the quantity measurable rather than metaphysical. (Contrast Solomonoff’s universal prior [3] and Kolmogorov complexity [4], which are uncomputable and therefore cannot serve as measurement procedures.) Each profile version blesses one of three codes for $L(M | G)$: a quantized two-part code with declared precision, a bits-back/KL code with declared prior [21], or a prequential code [19]. Where the code is underdetermined, $\mathcal{T}$ is reported under two to three reasonable codes as a sensitivity analysis. The declared prior is the standard refined-MDL solution to the underdetermined-code problem (luckiness normalized maximum likelihood [7]); for deep networks, prequential encoding has outperformed variational bounds in practice [30].

Assumption A2 (Computable predictive distribution). The candidate M must expose its predictive distribution p_M as a computable function. Where an actual entropy-coded bitstream

is not produced, the idealized codelength $L(D \mid M, G) = -\sum_t \log_2 p_M(x_t \mid x_{<t}, G)$ is used, and the profile must record which method was applied (`codelength_method`). The profile declares a reference entropy coder, arithmetic coding [20] or asymmetric numeral systems [24], with bits-back coders [21, 33] for latent-variable models; “bits” in a Turing measurement means bits under the declared coder. Idealized log-loss is realizable to within small bounded amortizable overhead by such coders, and H1 tests the bridge.

Assumption A3 (Continuous observations are quantized by declaration). Definition 1 operates on discrete observations. Where raw observations are continuous-valued, the measurement profile declares a fixed measurement precision (quantizer) $Q_{\Delta x}$, and the encoded observations are the resulting discrete measurement symbols

$$x_t^{(q)} = Q_{\Delta x}(x_t).$$

The candidate assigns probability mass to each quantization bin by integrating its predictive density over the bin,

$$p_M(x_t^{(q)} \mid G) = \int_{\text{bin}(x_t^{(q)})} f_M(x) dx,$$

and all codelengths are computed on the discrete symbols $x_t^{(q)}$ under Definition 1. Rationale: $-\log_2 f(x)$ of a probability *density* is not an invariant number of bits. It changes with the choice of measurement units and coordinate resolution, whereas $-\log_2 p_M(x_t^{(q)})$ of a probability *mass* is. The declared quantizer fixes the measurement resolution in advance, so the reported bits are literal code probabilities for declared symbols, not differential entropies. Two measurements with different quantizers are not comparable; the quantizer is part of the profile precisely for this reason.

Lemma 1 (Log-loss to idealized codelength). *For a predictive distribution p_M over a finite alphabet and a sequence D of length n , there exists a code whose length for D is less than $-\sum_{t=1}^n \log_2 p_M(x_t \mid x_{<t}) + 2$ bits, subject to the declared termination and message-length convention.*

Proof sketch. Let $P_M(D) = \prod_{t=1}^n p_M(x_t \mid x_{<t})$ be the probability the model assigns to the complete sequence. Arithmetic coding assigns the whole sequence an interval of width $P_M(D)$ within $[0, 1)$. A binary point identifying that interval requires fewer than $-\log_2 P_M(D) + 2 = -\sum_{t=1}^n \log_2 p_M(x_t \mid x_{<t}) + 2$ bits, under the profile’s declared termination and message-length convention (standard Shannon source-coding argument [2]; cf. textbook treatments of arithmetic coding). $\square$

Remark. Lemma 1 justifies reporting the log-loss sum as an *idealized* codelength. Hypothesis H1 of the experimental protocol tests whether this idealization tracks actual lossless encodings in practice; the declared reference coder (Assumption A2) is what makes “bits” literal rather than metaphorical.

Lemma 2 (Memorization bound). *Let M_{mem} be a pure memorizer: its entries are explicitly charged under profile P , and its predictions outside the memorized support S are no better than those of baseline B . On held-out D disjoint from S ,*

$$\mathcal{T}(M_{\text{mem}}; B, D, G, P) \leq L(B \mid G) - L(M_{\text{mem}} \mid G).$$

Hence if B is the same default predictor with lower description cost, the memorizer necessarily receives negative Turing gain.

Proof sketch. On D disjoint from S , the memorizer emits its default predictions, so $L(D \mid M_{\text{mem}}, G)$ is no better than $L(D \mid B, G)$ when B is that same default predictor; the gain is then

bounded by the description-cost difference $L(B \mid G) - L(M_{\text{mem}} \mid G)$, which is negative when the baseline is cheaper. $\square$

Remark. The counting argument behind the description cost (that arbitrary or worst-case tables with $|S|$ entries cost on the order of $|S|$ times a per-entry constant under any fixed computable encoding) covers incompressible tables, not every table. A table may itself contain compressible structure, and such structure is then doing explanatory work of its own. Lemma 2 is deliberately the weaker proposition: it proves only that pure memorization, explicitly charged, cannot earn positive Turings against a cheaper equal default on held-out data. That is exactly what the anti-memorization claim needs, and no more. The degenerate exceptions it names (tiny supports, pathologically bad baselines) are exactly why baselines must be declared and sane, and why negative Turings are informative rather than embarrassing.

5 Experimental Protocol

5.1 Measurement-profile discipline

Every experiment instantiates a versioned measurement profile satisfying Definition 3 (Table 1). The profile is preregistered before any sealed data is examined. The profile is also the paper’s answer to a philosophical demand: standards of what counts as an intelligible explanation shift with context, so the standard must be made explicit and versioned rather than left implicit [22]. Concretely:

1. The dataset, split, seal commitment (hash), blinding mechanism, sequence ordering, reset and warm-start rules, episode boundaries, and initial context are recorded before candidate construction or selection begins.
2. The baseline B is declared and frozen in the profile before evaluation.
3. The encoding convention for $L(M \mid G)$ and the probability representation for p_M are fixed in the profile.
4. Energy instrumentation is calibrated and its uncertainty recorded; the attribution rule (Definition 5) is stated for the concurrency regime of the run; the profile declares whether the reported yield is Y_{eval} , $Y_{\text{discovery}}$, or both.
5. After the run, all artifacts (profile, baseline, candidate, sealed data, energy traces, and the exact evaluation procedure) are published so that a stranger can recompute $\mathcal{T}$ and the reported yields.

5.2 Sealing and blinding

D must not have been available when M was constructed or selected. Two distinct guarantees are required, and they are not the same:

- **Identity (commitment).** A cryptographic hash of the held-out set D , recorded in the profile before candidate construction or selection, proves that the test set did not change afterward. Any violation discovered afterward invalidates the measurement.
- **Non-access (blinding).** The hash proves the data did not change; it does not prove that the researcher or candidate never saw it. For serious experiments, the profile must therefore name

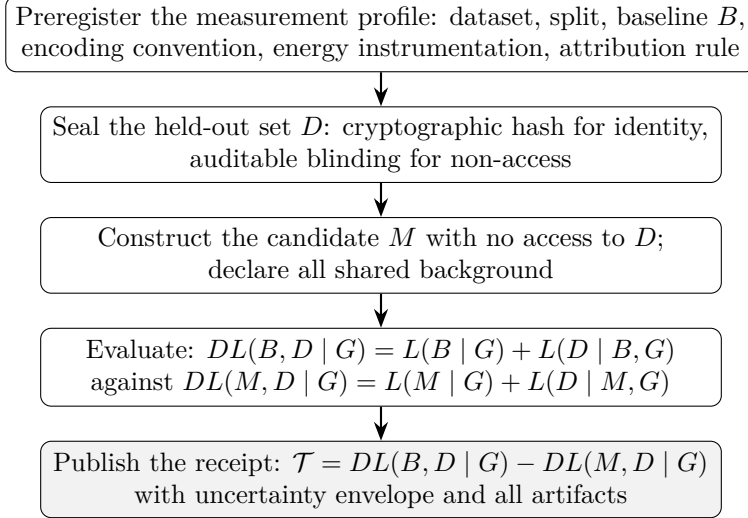


Figure 2: Conceptual schematic of the Turing measurement pipeline (Section 5). The profile is preregistered and the held-out set sealed before the candidate is constructed or selected; the reported value is the net held-out description-length difference with the candidate’s own complexity charged. Conceptual illustration, not empirical data.

one auditable blinding mechanism: an encrypted holdout released only after candidate freeze; a third-party evaluator; a physically or access-controlled test store; deterministic future-data collection after model freeze; or an equivalent. The hash proves identity; the blinding mechanism proves non-access.

Train/validation/test discipline is necessary but not sufficient. What makes the held-out claim auditable is the combination: commitment for identity, blinding for non-access.

Future data. For data that does not yet exist at the time of the freeze (fresh trajectories, future observations), hashing before construction is impossible, and the protocol does not ask for it. What is committed before the freeze is the collection protocol itself: the code and procedure that will generate the data, versioned and hashed in the profile. The realized data is hashed once collected, and the collection-protocol commitment is checked against the published code. The hash proves the identity of the realized data and the identity of the collection procedure; non-access is still established by the blinding mechanism.

5.3 Preregistered hypotheses

Each hypothesis below states a method sketch and an explicit pass/fail criterion. Concrete experiment instantiations are specified as a preregistered ladder (Section 5.4): EXP-001 is instrument validation on Omega traces, EXP-002 is Stochastic Law Discovery across three hidden stochastic worlds, and EXP-003 is active experiment selection. Each experiment is registered with a full measurement profile before data collection, and the ladder is ordered so that each rung validates the precondition of the next. Apparatus-check scripts used during protocol development are calibration of the instrument, not preregistered experiments, and their outputs are never reported as Turing measurements.

H1: Prediction–compression correspondence. *Method:* For a set of candidate models, compute idealized held-out codelengths $L(D | M, G)$ and actual lossless encodings of D produced by coupling each model’s p_M to the profile’s declared reference entropy coder. Let $\delta_M = L_{\text{real}}(D |$

$M, G) - L_{\text{ideal}}(D \mid M, G)$ be the coder overhead for M , and define δ_B likewise for the baseline; under matched model-cost conventions the two reporting kinds satisfy $\mathcal{T}_{\text{real}} - \mathcal{T}_{\text{ideal}} = \delta_B - \delta_M$. This is the test of whether $\mathcal{T}_{\text{ideal}} \approx \mathcal{T}_{\text{real}}$ within the profile’s declared bound. *Required tests*: exact lossless recovery of the encoded observations from the coded bitstream (round-trip correctness); declared per-source bounds for probability quantization, framing, termination, and other coding overhead; correct inclusion of model-description costs $L(M \mid G)$ in both $\mathcal{T}_{\text{real}}$ and $\mathcal{T}_{\text{ideal}}$; and agreement of the net gain within the bound implied by those declared overheads. *Pass*: round-trip correctness holds and absolute deviation of $\mathcal{T}_{\text{real}}$ from $\mathcal{T}_{\text{ideal}}$ stays within the profile’s declared uncertainty envelope. *Fail*: a lossy round trip, or deviation outside the declared bound. In that case the coder implementation is defective and is discarded: a failed coder implementation invalidates the implementation, not the mathematical source-coding result of Lemma 1, and idealized codelengths may not be reported as $\mathcal{T}_{\text{real}}$; they are reported in $\mathcal{T}_{\text{ideal}}$. Positive rank correlation between idealized and actual codelengths across candidates is supplementary evidence, not a substitute for bounded deviation. Until H1 passes for a regime, Turing gains in that regime are profile-bound idealized results, not measured compressions.

H2: Generalization. *Method*: Candidate abstractions produced by a discovery procedure are scored on sealed held-out D with their full $L(M \mid G)$ charged. Understanding is treated here as an ability that transfers, not a pile of propositions [29]: the profile may require scoring across multiple held-out regimes precisely to test that transfer. *Pass*: at least one candidate achieves $\mathcal{T} > 0$ with the lower bound of the uncertainty envelope above zero. *Fail*: candidates achieve positive training compression but non-positive held-out Turing gain. They have demonstrated fitting, not generalized explanatory compression.

H3: Reproducibility. *Method*: Two distinct tests. *Replay reproducibility*: the same measurement profile is re-executed with the same preregistered seed, the same artifacts, and only non-semantic re-implementations (code layout, refactors that do not change computed values). *Pass*: $\mathcal{T}$, Y_{eval} , and $Y_{\text{discovery}}$ reproduce within numerical tolerance. *Statistical robustness*: the profile is executed under different preregistered seeds drawn from the same experiment distribution (for EXP-002, different stochastic trajectories generated from the same hidden law family). *Pass*: the conclusions remain stable within the profile’s declared statistical envelope. *Fail*: same-seed re-execution diverges beyond tolerance, or different preregistered seeds change the conclusions outside the envelope: the profile is inadequate and must be revised before any measurement under it is reported. A changed seed is not an irrelevant implementation detail: it changes the sampled data or stochastic outcome, so it tests robustness, not replay.

H4: Physical conservation. *Method*: Under concurrent workloads, attribute energy per Definition 5 and check closure against measured machine energy. *Pass*: $E_{\text{measured}} \approx E_{\text{exclusive}} + E_{\text{shared}} + E_{\text{interaction}} + E_{\text{unattributed}}$ within sensor and model uncertainty on every run. *Fail*: attributed totals consistently exceed physical totals: the attribution model is invalid and no yield (Y_{eval} or $Y_{\text{discovery}}$) may be reported from it.

H5: Efficiency discrimination (the environmental hypothesis). *Method*: Take semantically equivalent realizations of the same explanatory gain (same $\mathcal{T}$ within tolerance on the same D , G , and profile) across different algorithms, representations, or substrates, and compare Y_{eval} and $Y_{\text{discovery}}$ where discovery energy is measured. *Pass*: at least one pair of equivalent realizations exhibits materially different yield outside the uncertainty envelope. *Fail*: yield never discriminates between realizations: in which case Turing yield adds no information beyond $\mathcal{T}$ alone for the tested regime. *Interpretation*: this is the hypothesis of Section 2.1 in falsifiable form. A pass makes Turing yield an instrument for steering toward informational productivity per joule; a fail bounds its usefulness and states the limit plainly.

5.4 Planned experiment instantiations

The hypotheses above are tested through a preregistered ladder of experiments. The ordering is deliberate: the first rung asks only whether the instrument works, the second whether it can detect discovered stochastic structure, and the third whether the machine can choose the experiment that most efficiently distinguishes competing explanations.

EXP-001: Instrument validation (Omega traces). Real Omega search and synthesis traces are divided, before candidate construction, into train, validation, and sealed test. Simple predictors (uniform, empirical frequency, a small Markov model, an existing fixed heuristic, and a candidate explanatory abstraction) are compared with full total description length charged. *Pass*: stable, reproducible Turing measurements with $\mathcal{T} > 0$ achievable by a candidate abstraction that earns its complexity, and the lower uncertainty bound above zero (H1–H3). This rung asks only: can $\mathcal{T}$ be measured reproducibly at all?

EXP-002: Stochastic law discovery. A hidden evaluator generates quantized trajectories from a stochastic process whose governing equation, family label, parameters, and random seed are unavailable to the candidate; Assumption A3 governs the quantization, and the quantizer is fixed in the preregistered profile. The candidate receives development observations and may construct a computable predictive explanation M . After candidate freeze, fresh held-out trajectories are generated under an auditable blinding protocol. The candidate is compared against preregistered weak, deterministic, and stochastic baselines using full total description length $L(M | G) + L(D | M, G)$. Initial hidden worlds test pure diffusion, diffusion with drift, and mean-reverting dynamics. A successful candidate must improve sealed predictive description length after paying its own complexity while maintaining calibrated uncertainty; adding unsupported structure must not receive positive explanatory credit. Where feasible, held-out trajectories include interventional regimes: the candidate must predict under interventions, not merely under passive observation, because the best finite-data predictor need not recover causal structure [27]. Explanations that compress multiple hidden worlds earn the sum of their Turing gains, rewarding unification across phenomena rather than narrow fitting.

The three hidden worlds, with explicit falsifiable expectations:

EXP-002A: Pure diffusion, $dX_t = \sqrt{2D} dW_t$. The candidate should discover that the increments are centered and that predictive uncertainty grows with elapsed time. Crucially, it must not earn Turings by inventing deterministic curves inside random fluctuations: a polynomial fit to a short random trajectory may compress development data. On any finite sample an overfit curve can score positively by chance, so the preregistered test compares polynomial fits against an otherwise matched zero-structure candidate across repeated null worlds, with a false-positive criterion (Lemma 2’s discipline applied to overfitting).

EXP-002B: Diffusion with drift, $dX_t = \mu dt + \sqrt{2D} dW_t$. The test is whether adding the drift parameter μ earns its complexity on unseen trajectories. When the hidden world has $\mu = 0$, the preregistered test compares the drift-augmented candidate against an otherwise matched zero-drift candidate, with a false-positive criterion across repeated null worlds. Chance fluctuations can favor an unnecessary parameter, and against a weak baseline a candidate can score positively by capturing diffusion regardless of its drift term. Structure without support pays the complexity charge without repayment; whether it earns no net credit is decided by the preregistered false-positive test across repeated null worlds, and no single finite sample can certify that verdict.

EXP-002C: Mean reversion, $dX_t = -\theta X_t dt + \sigma dW_t$. The system must discover a qualitatively different predictive structure (state-dependent restoring drift) rather than a quantitative tweak of diffusion.

EXP-003: Active experiment selection. With competing explanations in hand (e.g., free

diffusion versus mean reversion), the machine estimates the expected information gain of candidate interventions and proposes the most discriminating one, for instance starting farther from equilibrium or observing over a longer interval, subject to the authorization separation of Section 5.6. *Pass*: the machine-selected intervention distinguishes the competing explanations in fewer observations (or joules) than a preregistered passive baseline schedule, with the comparison itself conducted under sealed held-out evaluation.

The ladder, one line per rung:

- EXP-001 Can $\mathcal{T}$ be measured reproducibly at all?
- ↓
- EXP-002A Can a machine recognize stochastic structure without explaining noise?
- ↓
- EXP-002B Can additional structure earn its own complexity?
- ↓
- EXP-002C Can competing stochastic explanations be distinguished?
- ↓
- EXP-003 Can the machine choose the experiment that most efficiently distinguishes them?

Design note. The series is named the Stochastic Law Discovery Experiment, not “the Brownian test.” Brownian motion is the first hidden world; the framework is intended to extend to Poisson processes, Markov chains, anomalous diffusion, coupled systems, and noisy oscillators without changing the conceptual test. The series also doubles as a demonstration of this paper’s stated limit (Section 9): a false theory can temporarily compress a restricted dataset. A polynomial looks persuasive over one short random trajectory; it can still score positively on the next by chance, which is why the preregistered test uses repeated null worlds and a false-positive criterion rather than any single trajectory. The held-out complexity discipline exists to penalize overfitting systematically, not to guarantee its collapse on any one sample.

5.5 Worked illustration (not an empirical claim)

To fix the computation, suppose a baseline requires 2,600,000 bits to describe a sealed observation set. A newly discovered theory requires 18,400 bits to describe, and the same observations require 1,100,000 additional bits when encoded under the theory. Then

$$\mathcal{T} = 2,600,000 - (18,400 + 1,100,000) = \boxed{1,481,600 \text{ T}}.$$

If the evaluation consumed 30,000 J, its evaluation yield is $Y_{\text{eval}} = 1,481,600/30,000 \approx \boxed{49.39 \text{ T/J}}$. This is an arithmetic illustration of Definitions 2 and 4, not a reported measurement: no profile, no sealed data, and no instrumentation stand behind it. Since no entropy coder is involved, the illustration is stated in $\mathcal{T}_{\text{ideal}}$ terms.

5.6 From prediction to authorized intervention

Prediction alone is passive; scientific intelligence also asks *what should be measured next*. Given competing explanations H_1, H_2 , an experiment whose predicted distributions differ sharply between them carries high expected information gain, while one on which they agree carries little. A machine may therefore estimate the expected information gain of candidate interventions and propose the most discriminating one. Information value and authorization are separate quantities: a high-value experiment is a *proposal*, and whether it may be performed is decided by the authorization regime, not by its information value. This separation is a required property of all experiment instantiations under this protocol.

5.7 Shared background and the anti-gaming rule

Description length has always been relative to background knowledge; the question is never what the absolute $L(M \mid G)$ is, but what the profile declares as given, and whether it declares it completely. The most dangerous attack on the Turing is cost externalization: a tiny prompt or adapter riding on a gigantic undeclared pretrained model can score enormous net bits while the billions of bits of pretraining sit outside the measured $L(M \mid G)$ [40]. Chollet [12] made the general point: task skill can be bought with data and compute; what matters is efficiency relative to priors and experience.

The profile answers by rule, not by trust:

1. *Declare all shared side information.* Every pretrained model, corpus, tool, and prior artifact in the causal chain of producing M is versioned in the `shared_background` profile field.
2. *Score conditionally.* The reported gain is conditional on the declared background, and the conditioning set travels with the score.
3. *Forbid undeclared oracles.* Anything the explanation depends on but does not declare invalidates the measurement.
4. *Report both costs.* Where shared infrastructure is used, report the marginal task-specific cost and the amortized share of the background cost across the measurements that use it.
5. *Audit overlap.* The `overlap_audit` field names the procedure checking pretraining data against sealed held-out D ; undeclared overlap invalidates the measurement (cf. the contamination literature [39, 41, 37]).

Externalization is not always gaming. Human science does exactly this: Newton externalizes calculus. The difference is declaration. Shared background that is declared, versioned, and cost-accounted is legitimate cumulative science; undeclared background is cheating. The versioned measurement profile is the instrument that turns the loophole into a feature. In the notation of Definition 2, the declared background is G : the reported gain $\mathcal{T}(M; B, D, G, P)$ is conditional on exactly this set, and the set travels with the score.

6 Related Work

Only real, verifiable citations are included. The mathematical ancestry is credited in full; the claim of this paper is the named unit and its protocol, not the mathematics. An independent prior-art search (September 2026) across arXiv 2020–2026, the Anthropic/OpenAI/Google DeepMind research publications, and the conference literature found no existing unit named *Turing* for net held-out explanatory compression, and no equivalent named unit. The nearest neighbors are engaged below, precisely to mark the boundary.

Turing, A. M. (1950). *Computing Machinery and Intelligence*. *Mind*, 59(236), 433–460. The methodological precedent: Turing replaced “can machines think?” with an operational experiment rather than a definition. This paper follows that move, not its content; the distinction between the imitation game and the Turing unit is drawn in Section 10. Nothing here claims Turing’s endorsement. The evidence must stand independent of personal identity.

Shannon, C. E. (1948). *A Mathematical Theory of Communication*. *Bell System Technical Journal*, 27, 379–423, 623–656. Establishes the prediction $\leftrightarrow$ probability $\leftrightarrow$ description length bridge on which Definitions 1–2 rest, and the source-coding argument behind Lemma 1.

Solomonoff, R. J. (1964). *A Formal Theory of Inductive Inference*, Parts I and II. *Information and Control*, 7(1), 1–22; 7(2), 224–254. Algorithmic probability: prediction via a universal prior weighting shorter programs more heavily. The present proposal descends from this tradition but departs from it operationally: Solomonoff’s universal prior is uncomputable, whereas Assumption A1 requires a computable encoding convention precisely so that $L(M | G)$ can be measured.

Kolmogorov, A. N. (1965). *Three Approaches to the Quantitative Definition of Information*. *Problems of Information Transmission*, 1(1), 1–7. Complexity as shortest description length. Uncomputable in general; cited as ancestry, with the same operational departure as for Solomonoff.

Rissanen, J. (1978). *Modeling by Shortest Data Description*. *Automatica*, 14(5), 465–471. The Minimum Description Length principle in two-part form, $L(H) + L(D | H)$, the direct ancestor of Definition 1’s total description length. None of the MDL literature, to my knowledge, names a distinct unit for the resulting net gain; it reports the quantity in ordinary bits.

Grünwald, P. D. (2005). *A Tutorial Introduction to the Minimum Description Length Principle*. In Grünwald, Myung & Pitt (Eds.), *Advances in Minimum Description Length: Theory and Applications*. MIT Press. And **Grünwald, P. (2007).** *The Minimum Description Length Principle*. MIT Press. The modern exposition of MDL, including the connection between successful compression and the discovery of regularity that Lemma 2 formalizes for the memorization case.

Haldane, J. B. S. (1949). *Suggestions as to Quantitative Measurement of Rates of Evolution*. *Evolution*, 3, 51–56. Proposed the *darwin* as a unit of evolutionary rate (an increase or decrease of a character by a factor of e per million years). This is the naming precedent for the Turing: a scientific unit honoring a foundational thinker, denoting an experimentally constrained quantity rather than a bare physical dimension.

Hutter, M. (2006). *The Hutter Prize for Lossless Compression of Human Knowledge*, announced 2006 [9]. The prize scores the sum of decompressor size and compressed archive size on a fixed Wikipedia corpus: in this paper’s notation, $L(M) + L(D | M)$ on one public dataset, under an explicit “compression is intelligence” banner. It is the closest historical precedent for a two-part description-length score treated as a measure of intelligence. This work adds what the prize does not require: sealed held-out data, a declared and versioned shared background G , a baseline-relative net gain rather than an absolute compressed size, and the joining of measured energy to the gain (Turing yield).

Hernández-Espinosa, Ozelim, Abrahão & Zenil (2025). *SuperARC: A Test of Machine Superintelligence*, arXiv:2503.16743. The closest conceptual neighbor found in the prior-art search: it defines intelligence through “the ability of explanatory compression” with a ϕ metric. It remains distinct on every load-bearing point of this proposal: SuperARC is a benchmark score, not a named unit; it does not compute a baseline-relative net gain with the explanation’s own description cost charged; and it has no energy accounting and no sealed-data protocol. Cited as the nearest neighbor, and as evidence that the explanatory-compression framing is in the air while the unit is not.

Legg, S. and Hutter, M. (2007). *Universal Intelligence: A Definition of Machine Intelligence*. *Minds and Machines*, 17(4), 391–444. The canonical formal measure $Y(\pi) = \sum_{\mu} 2^{-K(\mu)} V_{\mu}^{\pi}$: reinforcement-learning and environment-based, with uncomputable Kolmogorov terms, no held-out data discipline, no named unit, and no energy accounting.

Chollet, F. (2019). *On the Measure of Intelligence*. arXiv:1911.01547. An algorithmic-information-theoretic account of intelligence as skill-acquisition efficiency relative to priors and experience; the denominator is experience, not joules, and no unit is proposed.

Zhang, Canlin and Liu, Xiuwen (2025). *A Theory of Machine Understanding via the Minimum Description Length Principle*. arXiv:2504.00395. Develops “to understand is to compress” as theory; proposes no unit and no held-out measurement protocol.

Anonymous (2026). *A Theory of Intelligence as Energy-Efficient Cost Reduction: Comple-*

tence, Learning, and Amortised Deliberation. Self-published manuscript, June 17, 2026, <https://amphitere.com/papers/a-theory-of-intelligence.pdf>. The closest antecedent to Turing *yield* found in the search: it defines intelligence as cost reduction per joule, but the numerator is task cost, not bits of net held-out explanatory compression; it proposes no named unit, and no author is named. Cited as an unpublished independent manuscript, to mark the boundary plainly.

Masum, H. and Christensen, S. (2003). *The Turing Ratio: A Framework for Open-Ended Task Metrics*. Journal of Evolution and Technology, 13(2). An unrelated performance-rating proposal for chess, expressed on a decibel scale relative to median human performance. Cited solely to disambiguate: it shares a name fragment and nothing else.

On “intelligence per joule” metrics. The machine-learning systems literature contains various accuracy-per-energy and performance-per-watt efficiency metrics. These measure benchmark performance per unit energy; they do not measure net held-out explanatory compression, charge the explanation’s own description cost, or require sealed data and declared baselines. The Turing is therefore not a restatement of them.

7 Empirical Results

Status: first profile-bound idealized Turing result recorded 2026-09-29 (reported in $\mathcal{T}_{\text{ideal}}$; not yet a $\mathcal{T}_{\text{real}}$ measurement).

On 2026-09-29 the first profile-bound idealized Turing result was recorded under measurement profile V0 (omega pull request 85, merged the same day). Candidate C (a table conditioning on the tried operation plus the last four outcomes) scored a specific Turing gain of about **0.44 $\mathbf{T}_{\text{ideal}}$ per held-out observation**: a total of **+2,559,679.825 $\mathbf{T}_{\text{ideal}}$** over $n = 5,834,845$ sealed held-out search events, relative to baseline B (a table conditioning only on the last outcome). The held-out events came from three seeds (8, 9, 10) never seen during model selection (seed 7 only); the pre-registered bar of 234,602 bits was cleared, with each held-out seed passing on its own. This run, TY-1+TY-2, is a completed pre-EXP instrument result with a fixed model class. It is not EXP-001: the EXP-001 preregistered runs are pending, and the H1 prediction-compression bridge (Section 5.3) remains open. The scoring tool declines to score the held-out seeds a second time: a useful procedural discipline, not proof of blinding. A separate refit check refits both models from the training files and reproduces the receipt exactly, which establishes replayability; non-access to the held-out seeds before the freeze rests on the seed discipline recorded in the profile (seeds 8–10 never seen during model selection; only seed 7 was used), not on the tool’s refusal. The complexity charge penalizes stored information that does not earn its cost on held-out observations. Under Lemma 2’s assumptions, pure memorization cannot outperform the cheaper baseline. Candidate C is itself a table-based predictor; its positive held-out gain is consistent with legitimate generalization under the complexity charge, which Lemma 2 does not forbid, subject to the limits stated below: this is an internally replayed result, and independent blinding was not established. All scoring arithmetic uses fixed-point integer micro-bits: 1 bit = 1,000,000 micro-bits (`src/turing/ty_math.c`). Probabilities are quantized to 16 bits; each symbol’s code length $-\log_2(q/65536)$ is computed with integer fixed-point arithmetic and rounded to the nearest micro-bit. Reported Turing values may therefore carry up to six decimal places: the receipt’s exact `t_microbits` = 2,559,679,824,815 micro-bits (2,559,679.824815 $\mathbf{T}_{\text{ideal}}$), displayed as +2,559,679.825 $\mathbf{T}_{\text{ideal}}$ rounded to three decimals, not a floating-point approximation. Worst-case rounding is under 6 bits over the whole held-out set. The 6-bit figure is an arithmetic rounding bound on the scoring computation, not a statistical confidence interval; statistical uncertainty for $\mathcal{T}$ is required future work. The reported baseline is a weak one (a last-outcome table); the profile’s `baseline_sensitivity` field requires the gain to be re-

ported under two to three reasonable alternative baselines, which is pending (Table 2). Attribution is likewise pending: the candidate changes operation conditioning and history length simultaneously, so the gain cannot yet be attributed to either. Required ablations are an operation-only baseline, an operation-plus-last-outcome baseline, and a longer-history-without-operation baseline; no ablation numbers are reported here (Table 2). Limits stated plainly: $\mathcal{T}$ here is a theoretical code length. No entropy coder was built that produces the smaller file, so this is not yet a $\mathcal{T}_{\text{real}}$ measurement; energy was not measured in this run, so no Turing yield (T/J) is reported. Joining R15 energy measurements to Turing-gain receipts is the next step (Definition 4).

Compliance note. This result complies with the historical profile it was scored under, measurement profile V0 preregistered 2026-09-29. It is not claimed to satisfy the current protocol in full. Put precisely: a result lacking auditable blinding is not a fully qualified measurement under the current protocol. The protocol also mandates baseline-sensitivity reporting and a stated uncertainty envelope, and the V0 run provides neither; these are recorded as pending in the ledger below. The measurement-validity criteria of Section 8, including the current statement of H1 and F1, are stated in this paper now and are applied to the V0 run retrospectively.

TY-1+TY-2 is an internally replayed descriptive result under historical profile V0. The experiment record reports separation of fitting, selection, and evaluation seeds, but independent blinding was not established. The original corpus is not publicly downloadable, so public end-to-end replay is not currently available. Statistical uncertainty, baseline sensitivity, and attribution ablations remain pending. The result therefore does not constitute full qualification under the current protocol. This does not establish contamination, and it does not make the arithmetic worthless; it distinguishes an internally reported held-out result from an independently auditable demonstration.

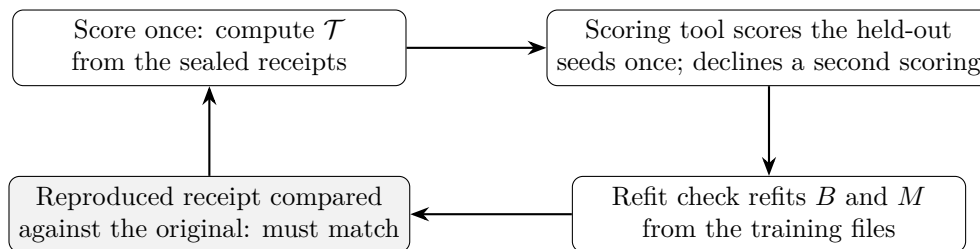


Figure 3: Conceptual schematic of the verification-by-refit loop used for the first profile-bound idealized Turing result (Section 7). The held-out data can be scored exactly once; reproducibility is established by refitting both models from the training files and reproducing the receipt bit for bit. The one-shot scoring is procedural discipline; blinding rests on the mechanism that kept the held-out seeds away from the model-selection process. Conceptual illustration, not empirical data.

Analysis plan (preregistered): for each experiment, report $\mathcal{T}$ with its uncertainty envelope, the energy attribution closure check (H4), and both yields Y_{eval} and $Y_{\text{discovery}}$ where the corresponding energies are measured; test H1–H5 per their pass/fail criteria; publish the full profile, artifacts, and energy traces for independent recomputation. Negative or null results will be reported as such. A failed $\mathcal{T} > 0$ is a result about the candidate, and a failed H3 or H4 is a result about the profile.

On prior runs. Earlier AIEN/Omega runs (including the R13 continuation run described in the companion paper *The Continuation Test*, and the R14 recovery run) predate the measurement-profile discipline and are **not scored retroactively**. Retroactive scoring under a profile that did not govern the run would violate the sealing and preregistration requirements of Section 5. The first scored run was TY-1+TY-2, conducted 2026-09-29 under profile V0: a profile-bound idealized Turing result, reported above.

8 Measurement Validity and Scientific Falsification

An immediate objection: a proposal that can fail in five ways claims nothing. The answer is that the failures belong to two different tiers, and confusing them would be the real evasion. The first tier is *measurement validity*: whether a reported number is a sound measurement under its declared profile. The second tier is *scientific falsification*: whether the quantity, once soundly measured, is a useful operational indicator of the aspect of understanding we care about. These criteria are stated in this paper now. No registration of them predates the TY-1+TY-2 result of Section 7, so that result is assessed against them retrospectively; the criteria stand preregistered for the pending EXP-001, EXP-002, and EXP-003 runs.

Measurement validity criteria. Criteria F1–F4 test the measurement implementation, the measurement profile, the T/J attribution, or the candidate. They do not test the statistic. A failure here invalidates a measurement, not the Turing.

F1: Bridge failure. If H1 fails systematically (a lossy round trip, or deviation outside the declared coding bounds), then the log-loss-to-codelength bridge is defective for the tested regime, and idealized codelengths may not be reported as $\mathcal{T}_{\text{real}}$; they remain reportable as $\mathcal{T}_{\text{ideal}}$ (Definition 2), which is exactly the status of the Section 7 result. A $\mathcal{T}_{\text{real}}$ claim survives only where the bridge holds.

F2: Generalization failure. If H2 fails (no discovered candidate achieves $\mathcal{T} > 0$ on sealed held-out data after paying its own description cost), then the proposal has not demonstrated generalized explanatory compression in the tested regime. This is a result about the candidate, not the statistic. Persistent F2 failure across diverse regimes would confine Turing gain to a theoretical quantity.

F3: Profile fragility. If H3 fails (gains vary outside the uncertainty envelope under same-seed replay or preregistered-seed variation), then the failure is in the profile: Turing measurements are not reproducible under the tested profile, and no measurement reported under that profile may be treated as a stable quantity. A unit that cannot survive reimplementaion is not a unit.

F4: Attribution failure. If H4 fails (attributed energy cannot be reconciled with measured machine energy), then the T/J attribution is invalid and no yield may be reported: Y_{eval} requires a grounded E_{eval} , and $Y_{\text{discovery}}$ requires the full measured discovery sum. Note the asymmetry: $\mathcal{T}$ can survive F4 while both yields cannot.

Scientific falsification criteria. What can ultimately fail is the scientific claim: that the net held-out description-length difference $DL(B, D \mid G) - DL(M, D \mid G)$ is a useful operational indicator of the aspect of understanding we care about, the discovery of compact structure that continues to predict unseen observations. The claim fails if sound, reproducible, profile-bound measurements of $\mathcal{T}$ repeatedly fail to track any independent sign that the machine has learned something reusable: large measured gains that never predict success on genuinely new phenomena, or a statistic that consistently rewards artifacts no independent test of understanding corroborates. Then the number would be a valid measurement of something, but not of what this paper claims. That is how the proposal dies as science while its arithmetic survives.

The test is made concrete as follows. Preregister a set of transfer tasks distinct from the scored held-out data: prediction on genuinely new phenomena (new hidden-world draws in EXP-002, interventional regimes of Section 5.6) and downstream intervention tasks with preregistered expected answers. For each candidate, record three quantities: its $\mathcal{T}$, its raw held-out log-loss improvement (without the model-complexity charge), and a naive complexity proxy such as parameter count. The proposed interpretation predicts that $\mathcal{T}$ predicts transfer-task performance *better than* raw held-out log-loss improvement alone predicts it: candidates whose gain survives the complexity charge should transfer where candidates whose improvement was merely log-loss reduction do not. The

comparators are deliberately simple: raw held-out log-loss improvement, and log-loss improvement penalized by naive parameter count. The interpretation counts against itself if, across the preregistered transfer tasks, $\mathcal{T}$ predicts transfer no better than raw log-loss improvement does, or if large measured gains systematically fail to predict transfer success while smaller log-loss improvements succeed.

F5: Supersession. If a better measure of verified explanatory gain is proposed (one that the Turing cannot match on the criteria of this paper), the Turing should be superseded. A measure of understanding should itself be exposed to falsification.

Partial failure is informative: for example, F4 failing while F1–F3 hold would establish the Turing as a valid informational measure whose physical pairing needs better instrumentation, not as a failed proposal.

9 What the Turing Does Not Measure

Following Turing’s procedure of meeting objections in turn, the limits are stated plainly. The Turing measures understanding-as-explanatory-compression: the epistemic dimension on which physics, statistics, and the minimum-description-length tradition agree. The constructive-empiricist reading [18] is the natural philosophical home of the claim: the metric scores empirical adequacy per unit of theoretical cost, a genuine scientific virtue even where the best-compressing model is not the true one [27]. The Turing must not be presented as measuring all of intelligence, nor as a certificate of safety or virtue. It does not directly measure: consciousness; subjective experience; moral worth; creativity in every sense; social understanding; agency; safety; wisdom; authority; or truth independent of the evaluation regime.

The exclusions are not an afterthought; they are what make the quantity crisp. The Turing does not measure *causal direction*: the covering-law tradition once identified explanation with predictive derivation [17], and its counterexamples are the reason this paper does not. It does not measure *semantic grounding*: a system can predict strings superbly with no model of what they are about [34]. It does not measure *agency*: passive compression captures only some of the abilities intelligence requires, while acting in the world additionally demands planning, perception, and intervention [26]. And it does not measure *value alignment*: the metric scores epistemic virtue, how efficiently regularities are captured, and is silent on which regularities are worth capturing and what the understanding is for [35]. A complete account of machine understanding needs all four; this paper supplies the one that can currently be measured with a number.

To the challenge that any formalization must say which real-world phenomenon it captures [42]: the phenomenon is theorizing itself, the cumulative practice by which expensive, weakly predictive accounts are replaced with cheaper, more predictive ones. That is narrower than intelligence, and fully within what the Turing measures.

A false theory may temporarily compress a restricted dataset. A deceptive model may predict humans well. A trivial environment may be highly compressible. A specialized predictor may score well while possessing little general competence. A well-calibrated predictor may still be misused. Consequently, verification, experimental design, distribution shift, causal intervention, and adversarial evaluation remain indispensable, and nothing in Sections 2.2–2.3 should be read as claiming otherwise: the Turing contributes a falsifiable evidentiary criterion for one operational component of understanding; not a sufficient condition, and not a safety guarantee.

10 Relationship to the Turing Test

The Turing Test asks, operationally, whether machine behavior can be distinguished from human behavior under an interrogation procedure. It is a finite game with a binary verdict, and it measures *imitation*: can the machine appear intelligent to a judge, for the duration of the interrogation?

The Turing unit asks a different question, in a different kind of game. Not “can the machine imitate us?” but “**can the machine discover structure that makes reality more predictable, and can that structure pay for its own complexity on observations it has never seen?**” It is an infinite game with no final verdict: gains are reported per experiment, under declared profiles, indefinitely, as the system continues. A high Turing gain does not imply passing any imitation test, and passing an imitation test implies nothing about Turing gain. A fluent memorized performance can pass the former while scoring at or below zero on the latter (Lemma 2, under its stated assumptions: a pure memorizer, explicitly charged, against a cheaper equal default on held-out data disjoint from its memorized support).

The contrast with benchmark culture is deliberate. A benchmark asks: how accurately does the system perform on this test set? A Turing measurement asks: how much sealed, unseen reality does the explanation make less surprising, after paying for itself, with the background declared? The first rewards performance, which can be bought with data, memorization, and prompt engineering [12, 37]; the second rewards explanatory efficiency, which cannot be bought without being charged. A large language model is a candidate under this protocol like any other: it may score Turings, provided its pretraining is declared as shared background and its held-out data is provably held out (Section 5.7). The protocol does not prejudice the architecture; it audits the accounting.

The latter does not replace Turing’s imitation game. It extends the same methodological instinct (replace the unanswerable question with a measurable one) into a domain in which modern machines can be evaluated continuously and quantitatively:

imitation $\rightarrow$ prediction $\rightarrow$ explanation $\rightarrow$ experiment.

11 Conclusion

Turing transformed the question of machine thought by declining to settle the meaning of thought first. This proposal follows that methodological example. There is no need to agree on a complete philosophy of machine understanding before beginning to measure one of its components.

I begin instead with a smaller experimental question: *has the machine discovered an explanation that is simpler than what came before, survives contact with unseen reality, and reduces what remains surprising?* If it has, the reduction can be measured. I call one bit of such net, verified, held-out explanatory compression **one Turing**.

The proposal may fail, and Section 8 states exactly how failure will be recognized. That falsifiability is a requirement of the proposal, not a defect.

If it succeeds, even partially, it supplies three things the field currently lacks: a measure that makes the energy cost of machine understanding legible and comparable as compute scales; a discipline that rewards verified explanatory gain over fluent performance; and a measurable target for the long work of building machines whose competence reflects understanding rather than performance. That is infrastructure worth having: for the science, for the environment it runs in, and for the people who will live with what it produces. The claim stays narrow by design: the Turing measures understanding-as-explanatory-compression, one operational component of understanding, and Sections 5.7 and 9 state what the protocol and the quantity exclude.

The machine proposes.
The experiment measures.
Reality decides.

Acknowledgments

This paper is solo-authored. AI language models were used as drafting, analysis, and review tools in its preparation; the argument, and any errors, are the author’s own.

A Revision history

This preprint was revised through v1.14 on 2026-10-02 before release. The revision record is kept here, outside the body of the paper, for transparency.

- v1.1** Technical revision: unit/outcome distinction, Lemma 1 proof, Lemma 2 rewrite, yield split, seal/blinding distinction, claim softening.
- v1.2** Related work expanded after an independent prior-art search (SuperARC, Legg–Hutter, Chollet, Zhang–Liu, and an unpublished independent manuscript on intelligence per joule); companion paper cross-reference updated.
- v1.3** Assumption A3: continuous observations are encoded as declared quantized symbols, so Definition 1 always operates on discrete measurement symbols; new **quantizer** profile field; preregistered experiment ladder EXP-001 (Omega-trace instrument validation), EXP-002 (Stochastic Law Discovery: pure diffusion, drifted diffusion, mean reversion), EXP-003 (active experiment selection).
- v1.4** Citation corrections after source verification: Zhang/Liu author initials corrected to C. Zhang and X. Liu; anonymous 2026 manuscript given its full title, date, and stable URL; R15 state reconciled against the repository (PR 67 merged, 16/16 gates PASS, energy measured; no Turing yield yet); H3 split into replay reproducibility vs. statistical robustness; Table 2 profiles marked draft with preregistration pending; **uncertainty_envelope** profile field now requires the estimation procedure.
- v1.5** First empirical result: TY-1+TY-2 PASS (+2,559,679.825 T_{ideal} under profile V0, omega PR 85); Empirical Results section populated; Definition 4 updated to distinguish measured Turing gain from not-yet-joined Turing yield.
- v1.6** Substantive revision addressing independent review: Turing gain reframed as net held-out explanatory compression (one measurable component of understanding, with explicit scope exclusions); Turing’s ban/deciban precedent (Good 1979); specific Turing gain (per-observation rate) with totals compared only at fixed sample size; shared-background declaration and anti-gaming rule; declared reference coder; blessed model-cost code with sensitivity analysis; baseline-sensitivity reporting; benchmark-culture contrast with a fair statement of the world-models debate; physics-grounding thesis with physics-informed ML and the Bitter Lesson; interventional held-out regimes; bibliography expanded.
- v1.7** Release versioning correction: the public release restores the true revision sequence (v1.1–v1.6) after the staged website build was erroneously labeled v1.0; companion cross-reference updated to v1.7. No content change from v1.6.

- v1.8** Figures added (Sections 3, 5, and 7): the Turing measurement pipeline, the distinction between Turing gain $\mathcal{T}$ and Turing yield, and the verification-by-refit loop. Substantive revision from technical review: the $\mathcal{T}_{\text{ideal}}/\mathcal{T}_{\text{real}}$ split (the Section 7 result restated as the first profile-bound idealized Turing result); the declared shared background G made explicit in the notation, $\mathcal{T}(M; B, D, G, P) = [L(B \mid G) + L(D \mid B, G)] - [L(M \mid G) + L(D \mid M, G)]$; the measurement-profile fields extended (sequence ordering, reset/warm-start rules, episode boundaries, initial context, candidate-selection count, test-query history); falsification re-framed as two tiers, measurement validity (F1–F4) and scientific falsification.
- v1.9** Review-driven revision: TY-1+TY-2 restated as the completed pre-EXP instrument result with EXP-001 pending and H1 open; the specific gain ($\sim 0.44 \mathcal{T}_{\text{ideal}}$ per observation) made the headline; the Hutter Prize added to Related Work; the anti-memorization claim narrowed to Lemma 2’s scope; the T/tesla notation footnote added; LaTeX production fixes.
- v1.10** Review-driven revision: invalid certainties in EXP-002A/B replaced with preregistered null-world comparisons and a false-positive criterion; the H1 prediction–compression bridge restated with lossless round-trip correctness and bounded deviation as the primary requirements (rank correlation demoted to supplementary evidence), with the bridge equation $\mathcal{T}_{\text{real}} - \mathcal{T}_{\text{ideal}} = \delta_B - \delta_M$ and a stated rule that a failed coder implementation invalidates the implementation, not the source-coding result; the scientific falsification criterion made concrete with preregistered transfer tasks, expected prediction, simple comparators, and the result that would count against the interpretation; the specific-gain discussion notes the model-cost term’s $1/n$ dependence, so specific gain is not fully sample-size independent; the scoring-tool refusal wording softened to procedural discipline, with replayability from the refit check and non-access from the seed discipline; the future-data commitment rule stated (commit to the collection protocol before the freeze, hash realized data after); TY-1+TY-2 attribution ablations marked pending (operation-only, operation+last-outcome, longer-history-without-operation); the measurement profile split into comparison protocol and execution manifest; energy attribution requires matched workloads, treats a negative interaction term as a signed saving, and bounds the permissible unattributed fraction; TY-1+TY-2 evidence ledger added; repeated unit-name defenses, Turing-Test comparisons, and may-fail declarations reduced.
- v1.11** Evidence ledger completed from the live repository: exact observation count $n = 5,834,845$, per-seed gains (8: 745,936.441 T; 9: 707,551.494 T; 10: 746,835.890 T), model lengths (B: 1,156 bits; C: 180,834 bits), held-out predictive losses, Krichevsky–Trofimov smoothing, artifact and profile links with SHA-256 digests, freeze chronology, receipt identifier, and replay command; 10-candidate selection on seed 7 recorded; Table 2 headline leads with the 0.44 per-observation specific gain.
- v1.13** Final reconciliation pass. The exact micro-bit gain is corrected: the receipt’s `t_microbits` = 2,559,679,824,815 micro-bits ($2,559,679.824815 \mathcal{T}_{\text{ideal}}$), and the headline +2,559,679.825 T is that value rounded to three decimals. A TY-1+TY-2 result-status paragraph is added: an internally replayed descriptive result under historical profile V0, with independent blinding not established, the corpus not publicly downloadable, and statistical uncertainty, baseline sensitivity, and attribution ablations pending, so the result does not constitute full qualification under the current protocol; this distinguishes an internally reported held-out result from an independently auditable demonstration without establishing contamination. The Candidate C generalization claim is qualified with the same limits; the categorical “earns no net credit” verdict is softened so that the repeated null-world false-positive test carries it; the

companion cross-reference is corrected to v1.7; G (declared shared background) is defined in the Definition 1 symbol list; and “per unit of explanation cost” is restated as predictive improvement after accounting for model-description cost (the score is a difference, not a ratio).

v1.14 Figure repair: Figure 1 (Turing gain vs. Turing yield) redrawn — the y-axis label no longer collides with the axis, and the “ Y larger” / “ Y smaller” labels now sit adjacent to their respective slope lines instead of floating ambiguously.

References

- [1] A. M. Turing, “Computing Machinery and Intelligence,” *Mind*, vol. 59, no. 236, pp. 433–460, 1950.
- [2] C. E. Shannon, “A Mathematical Theory of Communication,” *Bell System Technical Journal*, vol. 27, pp. 379–423 and 623–656, 1948.
- [3] R. J. Solomonoff, “A Formal Theory of Inductive Inference,” *Information and Control*, vol. 7, no. 1, pp. 1–22, 1964; vol. 7, no. 2, pp. 224–254, 1964.
- [4] A. N. Kolmogorov, “Three Approaches to the Quantitative Definition of Information,” *Problems of Information Transmission*, vol. 1, no. 1, pp. 1–7, 1965.
- [5] J. Rissanen, “Modeling by Shortest Data Description,” *Automatica*, vol. 14, no. 5, pp. 465–471, 1978.
- [6] P. D. Grünwald, “A Tutorial Introduction to the Minimum Description Length Principle,” in P. D. Grünwald, I. J. Myung, and M. A. Pitt (Eds.), *Advances in Minimum Description Length: Theory and Applications*, MIT Press, 2005.
- [7] P. Grünwald, *The Minimum Description Length Principle*, MIT Press, 2007.
- [8] J. B. S. Haldane, “Suggestions as to Quantitative Measurement of Rates of Evolution,” *Evolution*, vol. 3, pp. 51–56, 1949.
- [9] M. Hutter, “The Hutter Prize for Lossless Compression of Human Knowledge,” announced August 6, 2006. <http://prize.hutter1.net/>.
- [10] A. Hernández-Espinosa, F. Ozelim, F. Abrahão, and H. Zenil, “SuperARC: A Test of Machine Superintelligence,” arXiv:2503.16743, 2025.
- [11] S. Legg and M. Hutter, “Universal Intelligence: A Definition of Machine Intelligence,” *Minds and Machines*, vol. 17, no. 4, pp. 391–444, 2007.
- [12] F. Chollet, “On the Measure of Intelligence,” arXiv:1911.01547, 2019.
- [13] Canlin Zhang and Xiuwen Liu, “A Theory of Machine Understanding via the Minimum Description Length Principle,” arXiv:2504.00395, 2025.
- [14] Anonymous, “A Theory of Intelligence as Energy-Efficient Cost Reduction: Competence, Learning, and Amortised Deliberation,” self-published manuscript, June 17, 2026. <https://amphitere.com/papers/a-theory-of-intelligence.pdf>.

- [15] H. Masum, S. Christensen, and F. Oppacher, “The Turing Ratio: A Framework for Open-Ended Task Metrics,” *Journal of Evolution and Technology*, vol. 13, no. 2, 2003.
- [16] I. J. Good, “Studies in the History of Probability and Statistics. XXXVII: A. M. Turing’s Statistical Work in World War II,” *Biometrika*, vol. 66, no. 2, pp. 393–396, 1979.
- [17] C. G. Hempel and P. Oppenheim, “Studies in the Logic of Explanation,” *Philosophy of Science*, vol. 15, no. 2, pp. 135–175, 1948.
- [18] B. C. van Fraassen, *The Scientific Image*, Oxford University Press, 1980.
- [19] A. P. Dawid, “Present Position and Potential Developments: Some Personal Views. Statistical Theory: The Prequential Approach,” *Journal of the Royal Statistical Society, Series A*, vol. 147, no. 2, pp. 278–292, 1984.
- [20] I. H. Witten, R. M. Neal, and J. G. Cleary, “Arithmetic Coding for Data Compression,” *Communications of the ACM*, vol. 30, no. 6, pp. 520–540, 1987.
- [21] G. E. Hinton and D. van Camp, “Keeping the Neural Networks Simple by Minimizing the Description Length of the Weights,” in *Proc. 6th Annual Conference on Computational Learning Theory (COLT)*, 1993.
- [22] H. W. de Regt and D. Dieks, “A Contextual Approach to Scientific Understanding,” *Synthese*, vol. 144, no. 1, pp. 137–170, 2005.
- [23] S. Kirby, H. Cornish, and K. Smith, “Cumulative Cultural Evolution in the Laboratory: An Experimental Approach to the Origins of Structure in Human Language,” *Proceedings of the National Academy of Sciences*, vol. 105, no. 31, pp. 10681–10686, 2008.
- [24] J. Duda, “Asymmetric Numeral Systems,” arXiv:0902.0271, 2009.
- [25] K. Friston, “The Free-Energy Principle: A Unified Brain Theory?” *Nature Reviews Neuroscience*, vol. 11, no. 2, pp. 127–138, 2010.
- [26] J. Hernández-Orallo and D. L. Dowe, “Measuring Universal Intelligence: Towards an Anytime Intelligence Test,” *Artificial Intelligence*, vol. 174, no. 18, pp. 1508–1539, 2010.
- [27] G. Shmueli, “To Explain or to Predict?” *Statistical Science*, vol. 25, no. 3, pp. 289–310, 2010.
- [28] A. Clark, “Whatever Next? Predictive Brains, Situated Agents, and the Future of Cognitive Science,” *Behavioral and Brain Sciences*, vol. 36, no. 3, pp. 181–204, 2013.
- [29] F. Dellsén, “Scientific Progress: Knowledge versus Understanding,” *Studies in History and Philosophy of Science*, vol. 56, pp. 72–83, 2016.
- [30] L. Blier and Y. Ollivier, “The Description Length of Deep Learning Models,” arXiv:1802.07044, 2018.
- [31] M. Raissi, P. Perdikaris, and G. E. Karniadakis, “Physics-Informed Neural Networks: A Deep Learning Framework for Solving Forward and Inverse Problems Involving Nonlinear Partial Differential Equations,” *Journal of Computational Physics*, vol. 378, pp. 686–707, 2019.
- [32] R. S. Sutton, “The Bitter Lesson,” online essay, March 13, 2019. <http://www.incompleteideas.net/IncIdeas/TheBitterLesson.html>.

- [33] J. Townsend, T. Bird, and D. Barber, “Practical Lossless Compression with Latent Variables Using Bits Back Coding,” in *Proc. International Conference on Learning Representations (ICLR)*, 2019.
- [34] E. M. Bender and A. Koller, “Climbing towards NLU: On Meaning, Form, and Understanding in the Age of Data,” in *Proc. 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, pp. 5185–5198, 2020.
- [35] I. Gabriel, “Artificial Intelligence, Values, and Alignment,” *Minds and Machines*, vol. 30, pp. 411–437, 2020.
- [36] E. M. Bender, T. Gebru, A. McMillan-Major, and S. Shmitchell, “On the Dangers of Stochastic Parrots: Can Language Models Be Too Big?,” in *Proc. ACM Conference on Fairness, Accountability, and Transparency (FAccT)*, pp. 610–623, 2021.
- [37] N. Carlini et al., “Extracting Training Data from Large Language Models,” in *Proc. 30th USENIX Security Symposium*, 2021.
- [38] K. Li, A. K. Hopkins, D. Bau, F. Viégas, H. Pfister, and M. Wattenberg, “Emergent World Representations: Exploring a Sequence Model Trained on a Synthetic Task,” in *Proc. International Conference on Learning Representations (ICLR)*, 2023.
- [39] O. Sainz et al., “NLP Evaluation in Trouble: On the Need to Measure LLM Data Contamination for Each Benchmark,” in *Findings of the Association for Computational Linguistics: EMNLP 2023*, pp. 10776–10787, 2023.
- [40] G. Delétang et al., “Language Modeling Is Compression,” in *Proc. International Conference on Learning Representations (ICLR)*, 2024.
- [41] C.-H. Deng et al., “Investigating Data Contamination in Modern Benchmarks for Large Language Models,” in *Proc. 2024 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, 2024.
- [42] V. Lifschitz, “On a Measure of Intelligence,” *Bulletin of the EATCS*, 2024. <http://bulletin.eatcs.org/index.php/beatcs/article/download/822/873>.
- [43] R. T. McCoy, S. Yao, D. Friedman, M. D. Hardy, and T. L. Griffiths, “Embers of Autoregression Show How Large Language Models Are Shaped by the Problem They Are Trained to Solve,” *Proceedings of the National Academy of Sciences*, 2024, DOI 10.1073/pnas.2322420121.
- [44] K. Vafa et al., “Evaluating the World Model Implicit in a Generative Model,” arXiv:2406.03689, 2024.

Field	Requirement
profile_version	Version identifier of this profile
baseline	The declared baseline B , fixed before evaluation
candidate_encoding	The computable encoding convention used for $L(M G)$ and $L(B G)$
dataset	The observation source and collection procedure
data_split	Train/validation/test partition; which portion is sealed held-out D
seal_commitment	Cryptographic commitment (e.g., hash) to D recorded before M was constructed or selected (proves identity of D , not non-access). <i>Future-data exception:</i> where D does not yet exist at freeze, commit to the collection protocol before the freeze and hash the realized data after (Section 5.2)
blinding_mechanism	Auditable mechanism establishing non-access to D before candidate freeze (Section 5.2)
probability_representation	How p_M is represented and computed
quantizer	Fixed measurement precision $Q_{\Delta x}$ for continuous observations; the encoded symbols are the resulting discrete bin indices, with bin probabilities obtained by integrating the candidate’s predictive density (Assumption A3)
codelength_method	Idealized log-loss sum or actual entropy-coded bitstream
evaluation_procedure	Exact computation steps from artifacts to $\mathcal{T}$
software_version	All software involved, pinned
energy_instrumentation	Physical measurement apparatus and calibration for energy terms
attribution_rule	How joules are attributed under concurrency (Definition 5)
uncertainty_envelope	Declared reproducibility tolerance for $\mathcal{T}$ and yields, <i>with the estimation procedure stated:</i> which of block bootstrap resampling over sealed held-out blocks (blocks rather than individual observations where observations are correlated), repeated runs under preregistered seeds, instrument/sensor calibration uncertainty, or predictive/coding uncertainty are included, and how the lower bound of $\mathcal{T}$ is computed
verification_requirements	Independent recomputation requirements
shared_background	All pretrained models, corpora, tools, and prior artifacts in the causal chain of producing M , versioned; the gain is scored conditionally on this set (Section 5.7)
reference_coder	The declared entropy coder coupling p_M to bits; “bits” means bits under this coder (Section 4)
model_code	Which $L(M G)$ code is blessed: quantized two-part, bits-back/KL, or prequential; sensitivity analysis where underdetermined
baseline_sensitivity	$\mathcal{T}$ reported under two to three reasonable alternative baselines besides the declared B
overlap_audit	Procedure auditing pretraining data against sealed held-out D ; undeclared overlap invalidates the measurement
sequence_ordering	The order in which the observations of D are scored; $p(x_t x_{<t})$ depends on it, so the ordering is fixed in the profile before evaluation
reset_warmstart	Reset and warm-start rules: what state, if any, carries between sequences or episodes
episode_boundaries	How episode boundaries are defined and whether predictive context crosses them
initial_context	The context available to the candidate at the start of each held-out sequence (prior observations, prompts, system state)
selection_count	How many candidates were constructed or compared before the reported one
test_query_history	Whether the sealed test set, or any part of it, was ever queried by any earlier candidate or analysis; any such query invalidates the seal

Table 1: Required fields of a Turing measurement profile.

Expt.	Workload	Baseline	Profile	$\mathcal{T}$ (T)	E_{eval} (J)	$E_{\text{disc.}}$ (J)	Y
TY-1+TY-2	Omega search traces (held-out seeds 8–10)	last-outcome table	profile V0	0.44/obs.; +2,559,679.825 total	<i>pending</i>	<i>pending</i>	<i>pe</i>
TY-1+TY-2	<i>alternative baselines: pending</i>			<i>pending</i>	<i>pending</i>	<i>pending</i>	<i>pe</i>
TY-1+TY-2	<i>attribution ablations: operation-only, operation+last-outcome, longer-history-without-operation</i>			<i>pending</i>	<i>pending</i>	<i>pending</i>	<i>pe</i>
EXP-001	Omega traces (instrument validation)	uniform / empirical / Markov / heuristic	profile v1.0 (draft)	<i>pending</i>	<i>pending</i>	<i>pending</i>	<i>pe</i>
EXP-002	Stochastic law discovery (diffusion, drift, mean reversion)	weak / deterministic / stochastic	profile v1.0 (draft)	<i>pending</i>	<i>pending</i>	<i>pending</i>	<i>pe</i>
EXP-003	Active experiment selection vs. passive baseline	passive schedule	profile v1.0 (draft)	<i>pending</i>	<i>pending</i>	<i>pending</i>	<i>pe</i>

Table 2: Results. The TY-1+TY-2 rows report the first profile-bound idealized Turing result (2026-09-29, profile V0), in $\mathcal{T}_{\text{ideal}}$; the headline is the specific gain, about 0.44 $\mathcal{T}_{\text{ideal}}$ per held-out observation. Profile fields for the run: **selection_count**: 10 candidates compared on seed 7 (fit seeds 1–6), reported candidate is Candidate C; **test_query_history**: sealed seeds 8–10 were never queried by any candidate or analysis before scoring. The alternative-baselines and attribution-ablations rows are pending. EXP rows await runs under preregistered profiles. Numeric cells marked “pending” await sealed runs with energy instrumentation.

Evidence item	Recorded value or status
Held-out observations	$n = 5,834,845$ sealed held-out search events (seeds 8–10)
Per-seed gains	Seed 8: 745,936.441 T over 1,969,906 events; seed 9: 707,551.494 T over 1,884,427 events; seed 10: 746,835.890 T over 1,980,512 events. Each per-seed figure charges the full 179,678-bit model-cost difference once, so the three sum to 2,200,323.825 T; the pooled total charges it once overall, hence $2,200,323.825 + 2 \times 179,678 = 2,559,679.825$ T. Each seed passed the 234,602-bit bar on its own.
Baseline B	Table conditioning only on the last outcome; model description length 1,156 bits
Candidate M	Table conditioning on the tried operation plus the last four outcomes; model description length 180,834 bits
Predictive losses	Exact held-out log-loss: $B = 7,482,868,867,446$ micro-bits; $M = 4,743,511,042,631$ micro-bits. The printed three-decimal values (7,482,868.867 and 4,743,511.043 bits) are each rounded independently from these exact micro-bit values. The exact gain is the receipt's $t_microbits = 2,559,679,824,815$ micro-bits, i.e. 2,559,679.824815 T; the headline +2,559,679.825 T is that value rounded to three decimals.
Probability representation	Probabilities quantized to 16 bits ($q/65536$); fixed-point integer micro-bits (1 bit = 1,000,000 micro-bits); arithmetic in <code>src/turing/ty_math.c</code>
Smoothing	Krichevsky–Trofimov (add-1/2) smoothing, quantized to 16 bits: every table row adds one half-count to each cell, rows sum to exactly 65,536, every probability $\geq 1/65536$
Serialization	Model and scoring arithmetic in <code>src/turing/ty_math.c</code> ; artifacts pinned at merge commit <code>f0dc0af9db663bd36bcafe66c3d5ac79bbf143e5</code> : https://github.com/aien-dev/omega/tree/f0dc0af9db663bd36bcafe66c3d5ac79bbf143e5/evidence/TURING_YIELD/ . The trace corpus lives outside git at <code>~/aien-data/crumblin</code> and is not publicly downloadable.
Blinding mechanism	What exists: reported seed discipline. Ten candidates were compared on validation seed 7 (fit seeds 1–6); seeds 8–10 were never seen during model selection, as recorded in the 2026-09-29 selection/freeze record, omega PR 85, and the verify log; the scoring tool declines to score the held-out seeds a second time (procedural discipline, not proof of non-access). What does not exist: independently auditable blinding. No separate access-control or access-log mechanism guards the held-out traces; non-access rests on the recorded seed discipline, not on an audited barrier.
Freeze chronology	2026-09-27: trace corpus generated (seeds 1–10). 2026-09-29: profile V0 pre-registered; scoring run executed; independent verify run 22:00:05 UTC, VERIFY OK. Merged 21:05:27 UTC as omega pull request 85 (full merge commit <code>f0dc0af9db663bd36bcafe66c3d5ac79bbf143e5</code>).
Measurement profile	Profile V0 (<code>turing.yprofile.v0</code> , SHA-256 <code>f4292df9748e306165f30b250673a0062391054321ea6dbe68855ecaa0fa8be8</code>); human-readable at https://github.com/aien-dev/omega/blob/f0dc0af9db663bd36bcafe66c3d5ac79bbf143e5/docs/turing/TURING_YIELD_PROFILE_V0.md . Not the EXP-001-track <code>calibration/profiles/Turing-profile-v1.0.toml</code> .
Receipt	<code>evidence/TURING_YIELD/ty2_heldout_receipt.txt</code> (SHA-256 <code>11df23e1a213f408a74d2c9303643b8e2f29ce2c6a51fe3f59ccc1e168401f66</code>), pinned at https://github.com/aien-dev/omega/blob/f0dc0af9db663bd36bcafe66c3d5ac79bbf143e5/evidence/TURING_YIELD/ty2_heldout_receipt.txt ; verdict PASS; gain-record digest <code>823cd3a6589218cd5e1550fe8a61ac97ca539e289fd89f3f7397ded99e9e5af6</code> ; the receipt's exact $t_microbits = 2,559,679,824,815$
Replay command	From the verify log: <code>make turing-verify && build(tests_turing_yield/turing_yield_verify</code>

The Emergence Companion

The engineering history the measure emerged from.

Computing Machinery and Understanding

The Turing as a Measure Emerging from AIEN

M. Drake Stapleton

October 2, 2026

v1.7 preprint (not peer reviewed)

Companion to the formal preprint “Computing Machinery and Understanding: From the Imitation Game to the Turing” (v1.13), which gives the definitions, proofs, measurement protocol, and falsification criteria. This paper gives the engineering history the measure emerged from.

Abstract

I did not begin this work by trying to invent a new unit of intelligence.

I began by trying to build a machine whose internal claims could be separated from evidence, whose ideas could be separated from their physical implementations, whose failures could remain part of its history, and whose improvements could be tested against reality before they became authoritative.

That work became AIEN, Omega, FORGE, AEGIS, ARGUS, Cortex, and the resident reaction architecture now implemented across the AIEN repositories.

While working inside that layer of abstraction, I encountered a measurement problem.

The system could generate alternatives. It could verify them. It could measure them on real hardware. It could preserve failed hypotheses. It could discover recurring abstractions. It could promote a verified improvement into a new generation. It could survive injected failures and reconstruct itself from durable state.

But I still lacked a satisfactory answer to a simpler question:

How much did the machine actually learn?

Tokens per second did not answer it. Accuracy did not answer it. FLOPs did not answer it. Power consumption did not answer it. Even successful self-improvement did not answer it, because an improvement can be narrow, expensive, memorized, or dependent on complexity that exceeds what it explains.

The quantity I arrived at is a direct consequence of the architecture I had already built.

For a declared baseline B , candidate explanation M , sealed held-out observations D , and measurement profile P , I define the Turing gain

$$\mathcal{T}(M; B, D, P) = DL(B, D) - DL(M, D),$$

where

$$DL(M, D) = L(M) + L(D \mid M).$$

One **Turing**, $1T$, is one bit of net held-out explanatory compression under the declared protocol.

The important claim of this paper is therefore not that I have renamed the bit.

It is that, while building an architecture in which explanations, measurements, realizations, authority, provenance, and physical cost were already separate objects, I found that **net held-out description-length reduction is the quantity that connects them**.

AIEN already implements much of the machinery required to test that proposition. This paper documents that implementation, the evidence already produced, the places where the architecture failed and corrected itself, and the first profile-bound idealized Turing result, $+2,559,679.825 T_{\text{ideal}}$ under profile V0, recorded 2026-09-29: a candidate explanation of Omega search traces scored $+2,559,679.825 T_{\text{ideal}}$ under profile V0 against its baseline on sealed held-out seeds, about $0.44 T_{\text{ideal}}$ per held-out observation of specific Turing gain. That result is a theoretical description length, not yet a physically encoded file, and no energy was measured in the run, so no Turing yield is claimed. What the unit does not measure, I state plainly: semantic grounding, causal direction, agency, values, alignment, and consciousness are outside its scope. The Turing is a thermometer for one quantity, net held-out explanatory compression, and the claim is only that this quantity is worth measuring well.

Turing taught us to replace an unanswerable question with an operational one. My question is whether a machine can discover explanations that make unseen reality less surprising, and whether I can measure the physical cost of that discovery.

1 I Did Not Start With the Turing

My path into this problem ran in the opposite direction from the usual benchmark paper.

I was not trying to create another score for artificial intelligence.

I was trying to create a computational system in which I could tell, after the fact, *why something happened, what evidence caused it, which physical realization produced it, whether that realization had authority to act, whether the result survived verification, and whether an improvement could be reproduced later.*

The source code is public:

- AIEN/Omega: <https://github.com/aien-dev/omega>
- AIEN architecture: <https://github.com/aien-dev/aien-architecture>
- FORGE: <https://github.com/aien-dev/physics> (repository retains the historical PHYSICS name)
- AIENOS: <https://github.com/aien-dev/aienos>
- Sovereign core: <https://github.com/aien-dev/aien-sovereign-core>
- Benchmarks: <https://github.com/aien-dev/benchmarks>

The abstraction I eventually converged on separates four things that are usually collapsed together:

$$\text{meaning} \neq \text{realization} \neq \text{authority} \neq \text{evidence}.$$

Omega defines semantics and synthesizes candidates.

FORGE realizes and measures physical realizations.

AEGIS certifies generated code, including crumbs, according to policy and verification evidence. It changes nothing, decides no policy, and is not part of the generation system.

ARGUS observes defensive events without becoming an authority itself.

The World holds operational state.

Cortex retains epistemic memory.

The evidence system records what actually occurred.

That separation turned out to matter more than I expected.

Once a semantic operation is no longer identical to one implementation, the system can ask:

What other realization preserves the same meaning?

Once a hypothesis is no longer identical to authority, it can ask:

What could I try without allowing the hypothesis to approve itself?

Once an outcome is no longer identical to a claim, it can ask:

What evidence actually supports this?

And once a result is no longer identical to success, a further question appears:

Was the new explanation actually simpler and more predictive than what I had before?

That final question is where the Turing came from.

2 The First Piece Already Existed: Omega Was Discovering Compression

Before I formulated the Turing, Omega already contained an implemented abstraction-discovery system.

The implementation is public:

- https://github.com/aien-dev/omega/blob/4f55f45ed13f63cf8171bebf223c6258f8351ab9/src/omega_discovery.c
- https://github.com/aien-dev/omega/blob/1ee43fbf1c7d588734b80bd0743e5d054aea750b/src/omega_discovery.h

The experiment is deliberately small.

Omega constructs a corpus of independently verified programs containing a repeated structure. In the canonical qualification corpus, that repeated structure corresponds to the program

$$2x + 1.$$

The implementation scans realized programs for recurring instruction slices. A candidate must occur in more than one program and must be nontrivial.

The important part is the scoring rule. The source computes a net structural compression score conceptually equivalent to

$$\text{compression score} = \text{occurrences} \times (L - 1) - (L + 1),$$

where replacing a recurring sequence saves instructions at each occurrence while introducing the abstraction itself also has a cost.

Thus the abstraction is useful only when

$$\text{structure removed} > \text{cost of explanation}.$$

This was already a primitive Minimum Description Length calculation in the sense of Rissanen and Grünwald [3, 4].

The qualification receipt is public:

https://github.com/aien-dev/omega/blob/8e0a50a45814c5e3b427efd36a83a1e2e79a7500/evidence/omega_library_discovery_qualification_receipt.json

That receipt records:

- autonomous recurring-subsequence mining;
- +7 instructions of net structural compression;
- semantic preservation after refactoring;
- admission checks through Omega's V0 to V2 ladder;
- reduction in held-out synthesis exploration from 54 candidates to 21;
- all ten M11 qualification gates passing.

This history matters.
I did not begin with

$$L(M) + L(D \mid M)$$

and decide to force Omega into it.

I was already building machinery that admitted a reusable abstraction only when the abstraction paid for itself.

The Turing is the generalization of that principle from program structure to predictive explanation.

3 From Static Abstraction to Continuation

Structural compression alone is not enough.

A machine can discover a useful function without being a system that continues to investigate, test, and alter itself.

The next architectural problem I worked on was therefore causal continuation.

The architecture stopped treating cognition as a central loop that explicitly called every subsystem in order.

Instead, state changes made dependent reactions ready.

The relevant runtime work lives in:

<https://github.com/aien-dev/omega/tree/614393d5b3f3d9d06f214c5f9a30a5788166bb07/src/runtime>

The central pattern is

publication $\rightarrow$ dependency becomes ready $\rightarrow$ reaction executes $\rightarrow$ verified publication $\rightarrow$ next dependency.

The resulting causal system was qualified in the R13 living-system experiment.

The actual machine receipt is public:

<https://github.com/aien-dev/omega/blob/89e72ed72ba004631e1a0d81b874a136973d905f/evidence/R13/48d5a36c0ec2a8885aa21ebe1fd9d5d2259db1a25e935749adde6a1addd3b5b3.json>

This was not a simulated architecture diagram. It ran on the GB10 system.

The human supplied a goal: reduce the cost of an Omega integer matrix-vector operation below 55% of the confirmed incumbent while preserving semantics.

The incumbent was

7000 ns/call.

The target was

3850 ns/call.

The realized candidate selected by the system measured

2,686,162 ps/call $\approx$ 2686.162 ns/call.

The receipt records:

- 137 AIEN activations;
- 122 Omega activations;
- 16 GPU claims;
- 16 GPU completions;
- generation 1 $\rightarrow$ 2;
- selected realization identity exactly matching the promoted identity;
- the same identity reconstructed from durable bytes;
- 3,159 causal records;
- 11 durable provenance links verified;
- zero production results using the unpromoted realization;
- all five control conditions passing;
- final AIEN goal status MET.

I do not call this proof of general intelligence.

It proves something narrower and more useful.

I built a system in which a goal caused a chain of planning, search, verification, physical experimentation, evidence publication, selection, promotion, and reassessment without a central program manually sequencing those stages.

That is an implementation of the process I describe as

conjecture $\rightarrow$ test $\rightarrow$ retain $\rightarrow$ continue.

The architecture therefore existed before I had a general numerical measure for its epistemic progress.

4 A System That Cannot Survive Being Wrong Is Not Continuing

The next test was more important to me than another successful optimization.

I deliberately attacked the system.

The R14 qualification injected failures into the same resident architecture.

The receipt is public:

<https://github.com/aien-dev/omega/blob/2c647c6ff54be87c96041ce1fdd0abdd52a19a9/evidence/R14/0c09168743bfc8acac8b137a561958e9644c1d4160f92400b9a4bcec7b18965f.json>

Six classes of failure were tested.

4.1 Corrupt candidate

Candidate realizations were tampered with, made semantically incorrect, and made to crash.

The verifier refused them.

The bad realization was never promoted.

The valid candidate remained available.

4.2 Forged authority

The test presented forged capability references, valid references under the wrong subject, revoked references, stale generations, forged GPU claims, stuffed grant slots, forged verification records attributed to AEGIS, and a promotion forgery.

The receipt records them blocked or contained.

There were

0

publications under the stuffed grant and

0

mints during the attacks.

4.3 Reaction cycles

I injected

$$A \leftrightarrow B$$

and

$$X \rightarrow Y \rightarrow Z \rightarrow X,$$

plus a self-oscillating reaction.

They were bounded by the per-episode activation limit and oscillation containment.

Maximum cycle activations were

256.

Containment occurred in approximately

6 ms.

The legitimate organism continued producing work while the pathological reactions were contained.

4.4 GPU saturation

Sixteen lanes each submitted 200 requests against a four-claim resource budget.

The receipt records:

- 3200 submitted;
- 3200 completed;
- 3200 checked;
- zero duplicates;
- zero lost;
- zero mismatches.

4.5 GPU worker destruction

The running GPU channel was deliberately destroyed while it held a claim.

The claim failed once.

Its resource charge was released.

The seat generation advanced.

The stale result was refused.

The retry was causally linked to the loss and published once.

4.6 Crash during promotion

The system was crashed at eight different points of the durable-generation promotion protocol.

Recovery returned either the complete old generation or the complete new generation.

It never produced a torn generation.

The full R14 run produced

180,808

causal records with causal verification passing.

These are causal reaction records, not crumbs in the Crumb Doctrine sense: a Doctrine crumb is a verified program plus its verification receipt.

Just as importantly, the experiment found real defects in earlier stages of the architecture. Those defects were fixed at the layer that owned them and preserved in regression tests.

That is not an embarrassment in this design.

It is the point.

An architecture claiming to learn from evidence but rewriting away its failures would contradict the ideal it is supposed to implement.

5 I Required the Architecture to Beat a Control

There is another failure mode in systems research: building an elegant architecture and assuming it must therefore be better.

I did not want that.

The R15 performance specification states:

Do not accept architecture on aesthetic grounds.

The preregistered methodology is public:

<https://github.com/aien-dev/omega/blob/4d47ce7fe85c3dc6bcfa5f5157250194656b3154/spec/r15-performance-proof.md>

I built a sequential control using the same semantic work, same objects, same authority, same evidence, and same GPU seat.

Only the orchestration model changed.

That gave me a direct comparison between

resident causal architecture

and

sequential central orchestration.

The final qualification is public:

- <https://github.com/aien-dev/omega/blob/de4f32dee665b45faa7f96cb87015830e2b7f313/evidence/R15/ATTEMPT-2-PASS.md>
- <https://github.com/aien-dev/omega/blob/de4f32dee665b45faa7f96cb87015830e2b7f313/evidence/R15/065c688408aa18950f363b52be2357c3cb853827592dc71ea72b6da181b4569a.json>

The qualification ran on the DGX Spark GB10 and passed

16/16

gates.

One measured difference was CPU to GPU synchronization:

3.0

synchronization events per GPU result for the one-worker resident configuration versus

4.0

for the sequential control.

The methodology also measures semantic operations per second, activation latency, dependency propagation, scheduler overhead, verification fast-path cost, generation barriers, serialization, CPU to GPU synchronization, memory traffic, GPU residency, wasted reactions, conflict and invalidation, causal-evidence overhead, and energy per semantic result.

That last metric is directly relevant to the Turing.

I had already reached the point where I was asking:

What did this semantic result physically cost?

The missing half was:

How much epistemic gain did the semantic result represent?

6 The Architecture Also Needed to Admit When My Own Idea Was Worse

The strongest evidence for the philosophy of the project may be a failed experiment.

I built the first TURING Field as an evidence-backed decision-record layer around multiple realizations of one semantic operation.

The source and tests are public:

- <https://github.com/aien-dev/omega/blob/cc341abbe203c64506b2acab824f2e58e43f99ab/src/turing/field.h>
- https://github.com/aien-dev/omega/blob/cc341abbe203c64506b2acab824f2e58e43f99ab/tests/turing/test_turing.c
- https://github.com/aien-dev/omega/blob/cc341abbe203c64506b2acab824f2e58e43f99ab/docs/turing/TURING_WO_PROPOSAL.md

I expected the new selection rule to perform at least as well as a simple history-based control. It did not.

The preregistered comparison produced:

	Mean regret	Max regret
Field v0	3.58%	25.51%
Control	0.01%	0.07%

The quality gate failed.

The replayability gate passed.

I did not change the threshold.

I did not delete the result.

I did not tune the benchmark until my method won.

The failed selector remains in the repository so the failure can continue to be reproduced:

https://github.com/aien-dev/omega/blob/cc341abbe203c64506b2acab824f2e58e43f99ab/src/turing/field_select_v0_retired.c

The live Field was reframed around what the evidence actually supported: immutable, replayable decision records that cite their measurements.

The current code says directly:

Data only, not a subsystem.

The current tests ingest 720 evidence rows, reconstruct realization identities, verify source receipts, detect tampering, and verify that decisions cite the evidence they used.

A one-byte alteration of a receipt causes verification to fail.

This failure changed the architecture.

The tie policy was retired and the architecture moved toward lowest measured cost as the reference selection rule.

That history is preserved in:

<https://github.com/aien-dev/aien-architecture>

This is the implementation-level reason I take the phrase

reality decides

seriously.

It has already overruled me.

7 The Measurement Problem Appeared From Inside the Architecture

By this point, I had built several pieces of the same pattern.

Omega could discover reusable structure.

The reaction architecture could continue from a goal into experimentation and promotion.

The recovery architecture could survive rejected hypotheses, forged authority, saturation, crashes, and cycles.

The performance architecture could measure the physical cost of semantic work.

TURING could preserve the evidence supporting a decision and preserve a failed selection rule as negative evidence.

These systems all shared the same implicit idea:

Improvement means finding a structure that explains or accomplishes more while costing less, and proving that the improvement survives independent evidence.

But there was no common unit.

That was the abstraction boundary where I found the missing quantity.

8 The Turing

Let P be a declared measurement profile.

Let:

- B be a baseline explanatory model;
- M be a candidate explanatory model;
- D be sealed held-out observations.

For a computable predictive distribution,

$$L(D \mid M) = - \sum_{t=1}^n \log_2 p_M(x_t \mid x_{<t}).$$

This is the predictive log-loss expressed in bits, following the coding relationship developed by Shannon [2].

I also charge the explanation for existing:

$$L(M).$$

Therefore,

$$DL(M, D) = L(M) + L(D \mid M).$$

I define the Turing gain as

$$\boxed{\mathcal{T}(M; B, D, P) = DL(B, D) - DL(M, D).}$$

The reporting unit is the **Turing**, T :

$$\boxed{1T = 1 \text{ bit of net held-out explanatory advantage}}$$

under the declared measurement profile.

Raw totals may only be compared at fixed evidence volume. I therefore also report the **specific Turing gain**: $\mathcal{T}$ divided by the number of held-out observations, in Turings per observation. A measurement report separates five quantities that must not be collapsed: the raw total T , the specific gain, the candidate’s quality, the baseline’s strength, and the evidence volume.

A note on the name, because I checked: an unrelated proposal called the “Turing Ratio” [8] is a chess performance rating on a decibel scale. It shares a name fragment with this unit and nothing else. I also searched the recent literature for the closest neighbors to this proposal. The nearest is SuperARC [9], which defines intelligence through explanatory compression but as a benchmark score, not a named unit, with no baseline net-gain and no energy accounting. The formal intelligence measures of Legg and Hutter [10] and Chollet [11] propose no unit at all, and the recent MDL theory of machine understanding [12] likewise names none. To my knowledge, no prior work uses the term *Turing* for the protocol-bound quantity defined here.

A score of

$$+50,000T$$

means that, after paying the encoding cost of the new explanation, it required 50,000 fewer bits than the baseline to account for observations it did not receive during construction or selection.

A negative value is valid:

$$\mathcal{T} < 0$$

means the explanation did not pay for itself.

9 Why This Is Not Merely Renaming a Bit

A bit is a unit of information.

A Turing is the reporting unit of a constrained comparative measurement.

The measured quantity is

$$\mathcal{T}(M; B, D, P).$$

A result might be

$$\mathcal{T}(M; B, D, P) = 84,220T.$$

That statement implies:

- a declared baseline;
- a frozen candidate;
- a computable encoding convention;
- held-out observations;
- a sealed evaluation protocol;
- a comparison of total description length;
- a result that may be negative;
- evidence from which another person can recompute the outcome.

I do not claim to have discovered a new fundamental dimension of physics.

I am giving a name and protocol to a particular measured difference.

10 What the Turing Does Not Measure

A precise instrument for one quantity is stronger than an indefensible measure of everything. The thermometer is the analogy I keep: nobody asks a thermometer to measure pressure, and its authority comes from refusing to.

The Turing measures net held-out explanatory compression. It does not measure semantic grounding, causal direction, agency, intervention ability, social understanding, values, alignment, or consciousness. A high score means the candidate made unseen observations less surprising after paying its own cost. It does not mean the candidate understands in any broader sense, and I claim nothing of the kind.

This narrowing is also what separates the unit from benchmark culture. A benchmark ranks systems on fixed tasks; the score conflates the candidate, the baseline, and the evidence volume, and the leaderboard becomes the product. The Turing protocol reports those quantities separately, under a declared profile, with the baseline named and the held-out evidence sealed. Two measurements are comparable only under the same profile, and the report must say so.

11 Why Memorization Does Not Get a Free Pass

Suppose a system memorizes a giant table.

Its prediction error can approach zero on data represented by that table.

But the table itself has to be encoded.

The relevant quantity is not merely

$$L(D \mid M).$$

It is

$$L(M) + L(D \mid M).$$

If the table costs

8,000,000 bits

and saves

3,000,000 prediction bits,

the result is strongly negative.

A small rule can therefore outperform a memorizer epistemically even when both fit the same observations.

This is the same structural principle already implemented by Omega Library Discovery, generalized from

instructions saved – abstraction cost

to

prediction bits saved on unseen evidence – explanation cost.

12 Physics Does Not Disappear

While thinking about explanatory compression, I initially considered whether a new unit might replace familiar electrical measurements.

That would have been a category error.

Volts, amperes, watts, and joules measure physical quantities.

Turings measure explanatory gain.

I now treat them as orthogonal axes.

FORGE already provides a substrate-neutral physical boundary.

The FORGE V2 specification is public:

https://github.com/aien-dev/physics/blob/2da7b9d8d429abb590c4a12b1bae95eb6fd4461e/docs/FORGE_SUBSTRATE_V2_SPEC.md

Its stated purpose is that FORGE realizes semantic operations on physical devices.

The substrate description advertises evidence capabilities including:

- duration;

- energy;
- temperature;
- measured error;
- repeats;
- hardware status;
- faults.

The implementation is here:

<https://github.com/aien-dev/physics/tree/5969159c383217178b7b524fff13869489c8d799/forged/v2>

The informational quantity is

T .

The physical quantity remains

J .

Their ratio is

$$\boxed{\frac{T}{J}},$$

which I call **Turing yield**.

It is not energy converted into intelligence.

It is net explanatory gain measured per joule consumed by the declared computation.

13 The Physical Instrumentation Already Exists

This proposal is not based on the assumption that I might someday measure energy.

Omega's R15 harness already does.

The implementation includes:

https://github.com/aien-dev/omega/blob/ac09e6247da619e5cc1f29a70fa4792f10b679f1/tests/runtime/r15_measure.c

The measurement layer uses physical telemetry including energy counters, GPU power/energy instrumentation, ARM performance counters, CPU cycles, retired instructions, bus accesses, cache misses, memory accesses, thread CPU time, and wall-clock time.

Omega's empirical cost model already includes an energy dimension:

https://github.com/aien-dev/omega/blob/8290082331072f45fb0b3a60ca4f7654190fbfb6/src/runtime/rx_costmodel.h

A prediction contains an energy estimate and a request may specify an energy budget.
Thus the architecture had already learned to distinguish

semantic correctness

from

physical cost.

The Turing does not replace that machinery.
It completes it.

14 Multiple GPU Processes Require Attribution, Not New Physics

If workloads A and B run together, I cannot assume

$$E(A + B) = E(A) + E(B).$$

They may contend for compute units, memory bandwidth, caches, unified memory, power budget, thermal headroom, clocks, and synchronization resources.

I therefore define an interaction experiment.

Measure

$$E_0, \quad E_A, \quad E_B, \quad E_{AB}.$$

Then define

$$\boxed{I_{AB} = E_{AB} - E_A - E_B + E_0.}$$

That interaction term must remain visible.

I do not allow an accounting system to pretend every shared joule belongs exclusively to one process.

The physical conservation requirement is

$$E_{\text{measured}} \approx E_{\text{exclusive}} + E_{\text{shared}} + E_{\text{interaction}} + E_{\text{unattributed}}.$$

If attributed energy exceeds what the machine measured outside the declared uncertainty, the attribution fails.

Likewise, I impose informational conservation.

If three processes jointly produce one explanation worth

$$10,000T,$$

I cannot award all three processes $10,000T$ and announce $30,000T$.

The explanatory gain itself cannot be duplicated.

15 Observation Is Not Authority

Measurement must not become control.

ARGUS is the defensive observation plane in AIENOS.

Its ABI is public:

https://github.com/aien-dev/aienos/blob/b908e0187c2af7ab5675bafaf5768dc2f877441d/native/argus/argus_abi.h

Its boundary is explicit: ARGUS observes, detects, contains by request, and defends; it never authorizes.

A finding is evidence.

A containment request is a proposal.

Authority remains elsewhere.

That principle also constrains the Turing.

A system does not receive permission because it has a high T .

It does not receive permission because an experiment has high expected information gain.

It does not get to promote itself merely because it claims to have learned something.

The measurement layer measures.

Authority remains authority.

16 The First Turing Measurement Is Recorded

I want to be precise about what I have and have not demonstrated.

On 2026-09-29 the first profile-bound idealized Turing result was recorded under measurement profile V0 ([omega pull request 85](#), merged the same day). The candidate was a table conditioning on the tried operation plus the last four outcomes of Omega search traces; the baseline was a table conditioning only on the last outcome. On sealed held-out seeds 8, 9, and 10, about 5.8 million search events never seen during model selection, the candidate reduced total held-out description length by **2,559,679.825 bits**, scoring **+2,559,679.825 T_{ideal}** under profile V0, about **0.44 T_{ideal} per held-out observation** of specific Turing gain. The preregistered bar of 234,602 bits was cleared, with each held-out seed passing on its own. The scoring tool refuses to score the held-out seeds a second time, and a refit check refits both models from the training files and reproduces the receipt exactly. The complexity charge penalizes stored information that does not earn its cost on held-out observations. Under Lemma 2's assumptions, pure memorization cannot outperform the cheaper baseline. All scoring arithmetic uses fixed-point integer micro-bits, with worst-case rounding under 6 bits over the whole held-out set.

I state the result's status plainly, because the ledger now makes it checkable. TY-1+TY-2 is an internally replayed descriptive result under historical profile V0. The experiment record reports separation of fitting, selection, and evaluation seeds, but independent blinding was not established. The original corpus is not publicly downloadable, so public end-to-end replay is not currently available. Statistical uncertainty, baseline sensitivity, and attribution ablations remain pending. The result therefore does not constitute full qualification under the current protocol. This does not establish contamination and does not make the arithmetic worthless; it distinguishes an internally reported held-out result from an independently auditable demonstration. The full evidence ledger is in the formal paper.

The honest limits, stated with the result: $\mathcal{T}$ here is a theoretical code length. No entropy coder was built that produces the smaller file, so this is not yet a physically realized encoding. Energy was

not measured in this run, so no Turing yield (T/J) is reported. Joining R15 energy measurements to Turing-gain receipts is the next step.

I will not score old runs retroactively.

The R13 result predates the Turing protocol.

The M11 abstraction experiment uses instruction-count compression, not probabilistic held-out description length.

Likewise, retirement of all remaining central orchestration is still an active architectural gate.

The architecture is therefore evidence for the *implementation of the ideal*, and the TY-1+TY-2 result is a completed pre-EXP instrument result under profile V0. EXP-001’s preregistered runs, which close the reference-coder bridge (H1: $T_{\text{ideal}} \approx T_{\text{real}}$ within the declared bound), are pending; full qualification of the metric, including the physical coder bridge and energy-joined receipts, is still ahead.

17 The First Scored Experiment

The first scored experiment was intentionally boring.

I did not need a giant language model.

I already had an appropriate experimental substrate inside Omega: real search and synthesis traces, divided before candidate construction into training, validation, and sealed test.

The scored run compared two predictors: the candidate explanatory table (conditioning on the tried operation plus the last four outcomes) against the declared baseline (a table conditioning only on the last outcome). The wider comparison set, uniform, empirical frequency, a small Markov-style table, the existing fixed heuristic, belongs to the EXP-001 design, whose preregistered runs are pending. For every candidate M , the protocol calculated $L(M)$ and $L(D \mid M)$, then $\mathcal{T}(M; B, D, G, P)$ against the declared baseline.

The first gate was simple:

$$\boxed{\mathcal{T} > 0}$$

It passed, as recorded in the previous section: +2,559,679.825 T_{ideal} under profile V0, each held-out seed passing on its own. The 6-bit figure reported with the result is an arithmetic rounding bound on the scoring computation, not a statistical confidence interval; statistical uncertainty for $\mathcal{T}$ is required future work.

One more discipline the protocol enforces, and I want to state it plainly because it is easy to get wrong: the cryptographic hash of the sealed test set proves *which* data was held out. It does not prove the machine never saw it. A hash is a commitment to identity, not a blindfold. Non-access has to be established by a separate, auditable blinding mechanism, and the protocol treats the two claims separately: the hash proves the dataset is the one declared, the blinding proves the candidate was built without it. Either claim without the other is not a measurement.

In the formal paper the preregistered instrument-validation step is EXP-001. The scored run described above is not EXP-001; it is the completed pre-EXP instrument result TY-1+TY-2. EXP-001’s preregistered runs are pending, and with them the reference-coder bridge: H1, that $T_{\text{ideal}} \approx T_{\text{real}}$ within the declared bound, is open. EXP-001 is followed by EXP-002, Stochastic Law Discovery, in which a hidden evaluator generates quantized trajectories from an undisclosed stochastic law: pure diffusion, diffusion with drift, and mean-reverting dynamics, and the candidate must earn its complexity on sealed held-out trajectories without inventing deterministic structure inside noise; and by EXP-003, active experiment selection, in which the machine proposes the intervention that most efficiently distinguishes competing explanations. EXP-002 asks whether

the instrument can detect discovered structure. That is the empirical ladder the measure is now climbing.

18 The Second Experiment Joins the Machine Back to Physics

After the informational measurement works independently, I will join it to the physical measurement.

The same work unit should produce:

- candidate explanation;
- prediction trace;
- Turing receipt;
- physical interval receipt;
- energy attribution receipt.

Only then will I calculate

$$T/s$$

and

$$T/J.$$

For discovery itself I intend to distinguish evaluation cost from total discovery cost.

Define

$$Y_{\text{eval}} = \frac{T}{E_{\text{eval}}}$$

and

$$Y_{\text{discovery}} = \frac{T}{E_{\text{search}} + E_{\text{train}} + E_{\text{synthesis}} + E_{\text{verify}} + E_{\text{eval}}}.$$

For the machine-scientist problem, the second quantity is more important. The discovery denominator must include the energy cost of training the candidate itself (E_{train}): a model that discovers a compact explanation only after an enormous training expenditure has a lower discovery yield, and the measure should say so.

19 Why This Matters

I did not build this measure because I wanted a new number. I built it because three problems I care about have no honest number attached to them, and I think this one can serve all three without pretending to be something it is not.

First, the environment. As machine computation scales, the energy cost of producing understanding is invisible in every benchmark I know. Accuracy per watt tells you how cheaply a model performs; it does not tell you how cheaply a machine *learns*. Turings per joule makes the energy cost of discovery legible. If the field ever has to answer how much understanding it bought per unit of energy, this is the shape of the answer.

Second, honesty. The protocol only works if the baseline is declared in advance, the held-out data is sealed, every artifact is recomputable, and negative results are reported as what they are: a candidate that did not pay for itself. That discipline is not a nicety. It is the difference between a measure and a marketing claim. A field that adopts this unit adopts, along with it, the habit of saying what it tried and what failed.

Third, alignment. A great deal of alignment work has no measurable target: we ask machines to be safe, helpful, and honest, and we grade them with vibes. Verified understanding, operationalized as net held-out explanatory compression, is one thing we can actually measure. It is not the whole of alignment, and I claim nothing more.

And the honest bounds, stated once so they cannot be mistaken for modesty: the Turing does not measure safety. It does not measure consciousness, intent, or moral worth. It measures whether a machine discovered an explanation that makes unseen reality less surprising, after paying the explanation's own cost. That is all it is, and that is enough to be worth measuring.

20 Why I Think This Extends Turing's Method

I am not proposing to replace the Turing Test.

The Turing Test asks an operational behavioral question: can a machine's responses be distinguished from those of a human under an interrogation protocol? [1]

The question I encountered while building AIEN was different.

I already had machines capable of producing fluent responses.

That did not solve my problem.

I needed to know whether the system had discovered structure that generalized beyond what it had already seen.

So I arrived at a different operational question:

Can the machine construct an explanation that pays for its own complexity by making unseen reality less surprising?

The progression I see is

imitation $\rightarrow$ prediction $\rightarrow$ explanation $\rightarrow$ experiment.

Turing operationalized an argument about appearances.

I am trying to operationalize an argument about explanatory progress.

The intellectual debt is methodological, not rhetorical.

21 The Architecture Is the Argument

The reason I am willing to publish the idea now is not that the equation looks elegant.

It is that I can point to pieces of a working machine that forced the equation on me.

When I say *compression*, I can point to Omega's implemented abstraction-discovery engine and its qualification receipt.

When I say *generalization*, I can point to held-out search acceleration in the same experiment.

When I say *continuation*, I can point to the R13 silicon receipt and its causal chain from human goal to new generation.

When I say *falsification*, I can point to R14 fault injection and the defects those tests actually exposed.

When I say *physical cost*, I can point to R15’s measured energy, PMU, synchronization, and runtime instrumentation.

When I say *negative evidence*, I can point to TURING Wave 1, where my proposed selector lost and remains permanently reproducible as a failed experiment.

When I say *evidence before authority*, I can point to AEGIS certification, the capability system, ARGUS, and the promotion barrier.

When I say *different physical realizations of the same meaning*, I can point to FORGE’s substrate-neutral machine boundary and Omega’s realization system.

This is why I do not describe the Turing as an idea I invented in isolation.

I discovered the need for it while working inside this architecture.

The abstraction itself exposed the missing measurement.

22 Falsification

I want this proposal to be able to die.

I therefore consider the following failures binding.

22.1 F1: Prediction and compression diverge

If better probabilistic prediction does not produce correspondingly smaller real lossless encodings under the declared coding procedure, the bridge fails for that regime.

22.2 F2: No held-out gain

If candidates compress training traces but do not produce positive net Turing gain on sealed held-out observations after paying $L(M)$, I have demonstrated fitting rather than explanatory progress.

22.3 F3: Profile instability

If irrelevant implementation changes materially alter T , the measurement profile is inadequate.

22.4 F4: Physical accounting does not close

If attributed energy does not reconstruct measured energy within uncertainty, I will not report T/J .
Importantly,

$$T$$

may survive this failure, while

$$T/J$$

may not.

22.5 F5: A better measure replaces it

If another operational measure captures generalized explanatory progress more reproducibly and with fewer arbitrary conventions, I should use that instead.

The Turing should not receive protection from falsification merely because I named it.

23 What I Am Actually Claiming

My claim is narrower than “I built intelligence.”

I am claiming that I built a computational architecture in which several properties normally discussed abstractly are already explicit engineering objects:

- candidate explanations;
- semantic identity;
- multiple physical realizations;
- verification;
- evidence;
- authority;
- causal history;
- generation;
- failure;
- recovery;
- physical resource cost.

While working in that abstraction, I found that the missing epistemic quantity is naturally expressed as

$$\text{cost of old explanation} - \text{cost of new explanation.}$$

When evaluated on unseen observations and charged for the candidate itself, that becomes

$$\boxed{\mathcal{T}(M; B, D, P) = [L(B) + L(D \mid B)] - [L(M) + L(D \mid M)].}$$

I call the resulting unit the Turing.

The architecture does not yet prove that this is the right general measure of machine understanding.

It gives me something more useful:

a machine on which the claim can actually be tested.

24 Conclusion

Alan Turing’s enduring contribution to this problem was not merely a particular test.

It was a methodological move.

When a question becomes trapped in definitions, replace it with an operational question whose answer can be exposed to evidence.

I encountered the same problem from the other direction.

I built a machine around evidence.

I separated meaning from implementation.

I separated proposals from permission.
 I required improvements to survive verification.
 I made generations durable.
 I made failures remain in the record.
 I tested the system on real silicon.
 I broke it deliberately.
 I measured its physical costs.
 And then I reached a question the architecture could not yet answer:

How much explanatory ground did the machine actually gain?

That question produced the Turing.
 Whether the Turing survives is now an experimental matter.
 That is exactly how I want it.

The machine proposes.
The experiment measures.
Reality decides.

25 Epilogue

After this engineering history was written, TY-1+TY-2 produced the first profile-bound idealized Turing result (+2,559,679.825 T_{ideal}). EXP-001 remains the instrument-validation step required to close the reference-coder bridge (H1: $T_{\text{ideal}} \approx T_{\text{real}}$ within the declared bound).

Reproducibility Manifest

Each entry names the repository, the commit whose tree holds the cited artifact, the artifact SHA-256, the experiment it belongs to, the hardware profile where one was declared, and the receipt identifier where one exists. Where I could not establish a value from the repository or the receipt itself, I mark it [unavailable]; nothing is invented.

Artifact	Reproducibility fields
Omega repository	Repository: <code>aien-dev/omega</code> Commit: <code>2d52219708b42c5d9b2e8f7d577fbb5226631a78</code> Artifact SHA-256: [unavailable] Experiment ID: [unavailable] Hardware profile: [unavailable] Receipt ID: [unavailable] Public location: https://github.com/aien-dev/omega

Artifact	Reproducibility fields
Omega Library Discovery source	Repository: <code>aien-dev/omega</code> Commit: <code>4f55f45ed13f63cf8171bebf223c6258f8351ab9</code> Artifact SHA-256: <code>2adebb01bb69c2c936fdae73ef97dcdd45aaa965a854713a05ab831b42f701e4</code> Experiment ID: M11 Hardware profile: [unavailable] Receipt ID: [unavailable] Public location: https://github.com/aien-dev/omega/blob/4f55f45ed13f63cf8171bebf223c6258f8351ab9/src/omega_discovery.c
Omega Library Discovery header	Repository: <code>aien-dev/omega</code> Commit: <code>1ee43fbf1c7d588734b80bd0743e5d054aea750b</code> Artifact SHA-256: <code>4d607ec8f6ecb45c3f12cc1b0923fd09a8dc92bb9b7983c8c200d7f2555338b0</code> Experiment ID: M11 Hardware profile: [unavailable] Receipt ID: [unavailable] Public location: https://github.com/aien-dev/omega/blob/1ee43fbf1c7d588734b80bd0743e5d054aea750b/src/omega_discovery.h
M11 qualification receipt	Repository: <code>aien-dev/omega</code> Commit: <code>8e0a50a45814c5e3b427efd36a83a1e2e79a7500</code> Artifact SHA-256: <code>193bfce34119a66d7b8b739347a9ddd910b91348359ef59f61d3f2cad579638d</code> Experiment ID: M11 Hardware profile: Linux, aarch64 (per receipt host record) Receipt ID: CONTRACT-OMEGA-LIBRARY-DISCOVERY-M11 Public location: https://github.com/aien-dev/omega/blob/8e0a50a45814c5e3b427efd36a83a1e2e79a7500/evidence/omega_library_discovery_qualification_receipt.json
R13 living-system silicon receipt	Repository: <code>aien-dev/omega</code> Commit: <code>89e72ed72ba004631e1a0d81b874a136973d905f</code> Artifact SHA-256: <code>48d5a36c0ec2a8885aa21ebe1fd9d5d2259db1a25e935749adde6a1add3b5b3</code> Experiment ID: R13 Hardware profile: DGX Spark (NVIDIA GB10), node spark-b87b, Linux 7.0.0-1019-nvidia, aarch64; silicon run Receipt ID: 20260928T122742Z-83f880786202 Run code commit: <code>83f880786202bb2274848933395d55295cb0b679</code> Public location: https://github.com/aien-dev/omega/blob/89e72ed72ba004631e1a0d81b874a136973d905f/evidence/R13/48d5a36c0ec2a8885aa21ebe1fd9d5d2259db1a25e935749adde6a1add3b5b3.json

Artifact	Reproducibility fields
R14 recovery qualification receipt	Repository: <code>aien-dev/omega</code> Commit: <code>2c647c6ff54be87c96041ce1fdd0abdd52a19a9</code> Artifact SHA-256: <code>0c09168743bfc8acac8b137a561958e9644c1d4160f92400b9a4bcec7b18965f</code> Experiment ID: R14 Hardware profile: DGX Spark (NVIDIA GB10), node spark-b87b, Linux 7.0.0-1019-nvidia, aarch64; silicon run Receipt ID: 20260928T134432Z-6b38173cd5be Run code commit: <code>6b38173cd5bee75cbdb0b132d535ec0f0df38d4c</code> Public location: https://github.com/aien-dev/omega/blob/2c647c6ff54be87c96041ce1fdd0abdd52a19a9/evidence/R14/0c09168743bfc8acac8b137a561958e9644c1d4160f92400b9a4bcec7b18965f.json
R15 preregistered methodology	Repository: <code>aien-dev/omega</code> Commit: <code>4d47ce7fe85c3dc6bcfa5f5157250194656b3154</code> Artifact SHA-256: <code>cae61c9bb0a1e48fb316e8159a296f503ce155015eec3a02c814478f62dfda0a</code> Experiment ID: R15 Hardware profile: [unavailable] Receipt ID: [unavailable] Public location: https://github.com/aien-dev/omega/blob/4d47ce7fe85c3dc6bcfa5f5157250194656b3154/spec/r15-performance-profile.md
R15 qualification summary	Repository: <code>aien-dev/omega</code> Commit: <code>de4f32dee665b45faa7f96cb87015830e2b7f313</code> Artifact SHA-256: <code>5b3541e9d3f7568e5e74c5429e9dc0e5ecec297c5626b9f66d01749bbd401172</code> Experiment ID: R15 Hardware profile: [unavailable] Receipt ID: [unavailable] Public location: https://github.com/aien-dev/omega/blob/de4f32dee665b45faa7f96cb87015830e2b7f313/evidence/R15/ATTEMPT-2-PASS.md

Artifact	Reproducibility fields
R15 machine-readable receipt	Repository: <code>aien-dev/omega</code> Commit: <code>de4f32dee665b45faa7f96cb87015830e2b7f313</code> Artifact SHA-256: <code>065c688408aa18950f363b52be2357c3cb853827592dc71ea72b6da181b4569a</code> Experiment ID: R15 Hardware profile: DGX Spark GB10, host spark-b87b, kernel 7.0.0-1019-nvidia; silicon mode Receipt ID: 20260929T025735Z-3e9e53be3358-silicon Run code commit: <code>3e9e53be3358f6f2849a4f02044a1fd809240cbb</code> Public location: https://github.com/aien-dev/omega/blob/de4f32dee665b45faa7f96cb87015830e2b7f313/evidence/R15/065c688408aa18950f363b52be2357c3cb853827592dc71ea72b6da181b4569a.json
TURING Field record definitions	Repository: <code>aien-dev/omega</code> Commit: <code>cc341abbe203c64506b2acab824f2e58e43f99ab</code> Artifact SHA-256: <code>62f5eae7c870ce87c5b3a20b0d99473cc2bc0b5eb617398c0555ef08ad22823c</code> Experiment ID: TURING W0 Hardware profile: [unavailable] Receipt ID: [unavailable] Public location: https://github.com/aien-dev/omega/blob/cc341abbe203c64506b2acab824f2e58e43f99ab/src/turing/field.h
TURING preregistered experiment	Repository: <code>aien-dev/omega</code> Commit: <code>cc341abbe203c64506b2acab824f2e58e43f99ab</code> Artifact SHA-256: <code>a709f74fdadcf5f720a5bfb62c8949e6cd439363c676cf25526147ef65b08705</code> Experiment ID: TURING W0 Hardware profile: [unavailable] Receipt ID: [unavailable] Public location: https://github.com/aien-dev/omega/blob/cc341abbe203c64506b2acab824f2e58e43f99ab/docs/turing/TURING_W0_PROPOSAL.md
TURING reproducibility and tamper tests	Repository: <code>aien-dev/omega</code> Commit: <code>cc341abbe203c64506b2acab824f2e58e43f99ab</code> Artifact SHA-256: <code>50c9a8174f19b348d101d3dd352d1c9925b5fa1a688a44e08572aabfd10af54e</code> Experiment ID: TURING W0 Hardware profile: [unavailable] Receipt ID: [unavailable] Public location: https://github.com/aien-dev/omega/blob/cc341abbe203c64506b2acab824f2e58e43f99ab/tests/turing/test_turing.c

Artifact	Reproducibility fields
Omega empirical cost model	Repository: <code>aien-dev/omega</code> Commit: <code>8290082331072f45fb0b3a60ca4f7654190fbfb6</code> Artifact SHA-256: <code>89b9b953099e10eb9a3ce01ec109d72c18153f67f3f4139935c8997e2a6cffc1</code> Experiment ID: <code>R15</code> Hardware profile: <code>[unavailable]</code> Receipt ID: <code>[unavailable]</code> Public location: https://github.com/aien-dev/omega/blob/8290082331072f45fb0b3a60ca4f7654190fbfb6/src/runtime/rx_costmode1.h
FORGE V2 specification	Repository: <code>aien-dev/physics</code> Commit: <code>2da7b9d8d429abb590c4a12b1bae95eb6fd4461e</code> Artifact SHA-256: <code>a9c15f8b2dc528106b1690e6147fb5783c090648e828397c95c007699f6458d3</code> Experiment ID: <code>[unavailable]</code> Hardware profile: <code>[unavailable]</code> Receipt ID: <code>[unavailable]</code> Public location: https://github.com/aien-dev/physics/blob/2da7b9d8d429abb590c4a12b1bae95eb6fd4461e/docs/FORGE_SUBSTRATE_V2_SPEC.md
FORGE V2 implementation	Repository: <code>aien-dev/physics</code> Commit: <code>5969159c383217178b7b524fff13869489c8d799</code> Artifact SHA-256: <code>[unavailable]</code> Experiment ID: <code>[unavailable]</code> Hardware profile: <code>[unavailable]</code> Receipt ID: <code>[unavailable]</code> Public location: https://github.com/aien-dev/physics/tree/5969159c383217178b7b524fff13869489c8d799/forge/v2
ARGUS ABI	Repository: <code>aien-dev/aienos</code> Commit: <code>b908e0187c2af7ab5675bafaf5768dc2f877441d</code> Artifact SHA-256: <code>7368f92744b0d0481b64462fa00f873f0820e7a612171d9c65d476222e952e51</code> Experiment ID: <code>[unavailable]</code> Hardware profile: <code>[unavailable]</code> Receipt ID: <code>[unavailable]</code> Public location: https://github.com/aien-dev/aienos/blob/b908e0187c2af7ab5675bafaf5768dc2f877441d/native/argus/argus_abi.h

Artifact	Reproducibility fields
AIEN architecture	Repository: <code>aien-dev/aien-architecture</code> Commit: <code>33acd896e5d0cebe18201472b8f6b0b2e91f1b91</code> Artifact SHA-256: [unavailable] Experiment ID: [unavailable] Hardware profile: [unavailable] Receipt ID: [unavailable] Public location: https://github.com/aien-dev/aien-architecture

A Revision history

This companion paper was revised through v1.7 on 2026-10-02 before release. The revision record is kept here, outside the body of the paper, for transparency.

- v1.7** Result-status reconciliation with the formal preprint v1.13: result-status paragraph added beside the TY-1+TY-2 measurement (internally replayed descriptive result; independent blinding not established; corpus not publicly downloadable); “sealed unseen” replaced by “sealed held-out” with the blinding distinction; cross-reference updated to the formal preprint v1.13.
- v1.5** Terminology and status reconciliation with the formal preprint v1.9: “first canonical Turing measurement” replaced throughout by the first profile-bound idealized Turing result, $+2,559,679.825 T_{\text{ideal}}$ under profile V0; the TY-1+TY-2 run restated as the completed pre-EXP instrument result, with EXP-001’s preregistered runs pending and H1 open; cross-reference updated to the formal preprint v1.9.
- v1.1** Cross-reference updated to the formal preprint v1.3; forward pointer to the preregistered experiment ladder EXP-001/002/003 defined there.
- v1.2** Em dashes removed throughout at the author’s direction (punctuation only, no content change).
- v1.3** Cross-reference updated to the formal preprint v1.6; first canonical Turing measurement recorded (omega PR 85, profile V0); AEGIS role clarified per the AEGIS ruling; FORGE terminology; narrow-scope statement and specific Turing gain.
- v1.4** Epilogue added (TY-1+TY-2 idealized Turing result; EXP-001 as the reference-coder bridge); cross-reference updated to the formal preprint v1.8; live branch URLs replaced with immutable commit permalinks; evidence index rebuilt as a reproducibility manifest (repository, commit, artifact SHA-256, experiment ID, hardware profile, receipt ID); R13/R14 causal records renamed out of the Crumb Doctrine vocabulary with the distinction stated; canonical role vocabulary applied throughout (AIEN proposes, Omega defines semantics and synthesizes candidates, AEGIS authorizes/certifies, independent verifiers verify, PHYSICS/FORGE realizes and measures, hardware executes, evidence records, Cortex learns/retains).

References

- [1] A. M. Turing, “Computing Machinery and Intelligence,” *Mind*, vol. 59, no. 236, pp. 433–460, 1950.
- [2] C. E. Shannon, “A Mathematical Theory of Communication,” *Bell System Technical Journal*, vol. 27, pp. 379–423 and 623–656, 1948.
- [3] J. Rissanen, “Modeling by Shortest Data Description,” *Automatica*, vol. 14, no. 5, pp. 465–471, 1978.
- [4] P. Grünwald, *The Minimum Description Length Principle*, MIT Press, 2007.
- [5] R. J. Solomonoff, “A Formal Theory of Inductive Inference,” *Information and Control*, vol. 7, parts I–II, 1964.
- [6] A. N. Kolmogorov, “Three Approaches to the Quantitative Definition of Information,” *Problems of Information Transmission*, vol. 1, no. 1, pp. 1–7, 1965.
- [7] J. B. S. Haldane, “Suggestions as to Quantitative Measurement of Rates of Evolution,” *Evolution*, vol. 3, pp. 51–56, 1949.
- [8] H. Masum, S. Christensen, and F. Oppacher, “The Turing Ratio: A Framework for Open-Ended Task Metrics,” *Journal of Evolution and Technology*, vol. 13, no. 2, 2003.
- [9] A. Hernández-Espinosa, F. Ozelim, F. Abrahão, and H. Zenil, “SuperARC: A Test of Machine Superintelligence,” arXiv:2503.16743, 2025.
- [10] S. Legg and M. Hutter, “Universal Intelligence: A Definition of Machine Intelligence,” *Minds and Machines*, vol. 17, no. 4, pp. 391–444, 2007.
- [11] F. Chollet, “On the Measure of Intelligence,” arXiv:1911.01547, 2019.
- [12] Canlin Zhang and Xiuwen Liu, “A Theory of Machine Understanding via the Minimum Description Length Principle,” arXiv:2504.00395, 2025.

The Turing Discovery Gap Conjecture

The conjecture: available explanatory value versus accessible explanatory value.

The Turing Discovery Gap Conjecture

M. Drake Stapleton

October 2, 2026

How much explanatory gain can exist beyond the reach of efficient discovery?

Revision 9, 2026-10-02. In this revision I restate the Claim A uniform scoring bound operationally: the profile fixes one polynomial step budget for the whole evaluation, the scorer executes submitted code under that budget and rejects timeouts or failed reconstruction, and the key-holding reference program fits within the budget. I also mark the CEXP-1 measurement profile and the appendix reference as draft preregistration status, not frozen.

Earlier history: Revision 8, 2026-10-02. In this revision I narrow CEXP-1 to equal candidate-evaluation allowances, with physical costs reported as outcomes rather than assumed equal; retitle the CEXP-1 text as a draft preregistration specification; state the Claim A uniform scoring bound as a scorer requirement; and update the cited paper versions to formal v1.12 and companion v1.6.

Central claim

I claim there exist worlds governed by compact, efficiently executable rules, whose explanations yield enormous gains in Turings, yet remain beyond the reliable reach of every computationally efficient learner.

Knowing a rule and discovering that rule are different computational tasks. A short program may explain a vast collection of observations while offering no efficient path from those observations to an effective explanation.

The explanation can be small while the distance to finding it is vast.

The Turing as a unit

One Turing represents one bit of net held-out description-length gain, after accounting for the description cost of the model, in the minimum-description-length sense [10].

An evaluation record consists of an input location and its outcome. Let X denote the input locations of the evaluation set and D the outcomes at those locations. The input locations are supplied to both models: they are the questions, not the answers. Let W denote the hidden generating program that produced the outcomes; W is never part of the declared background. For a candidate model M , a fixed baseline B , the evaluation outcomes D , the declared shared background G (here: the input list X and the scoring conventions), and the frozen measurement profile P , define:

$$T(M; B, D, G, P) = [L(B | G) + L(D | B, G)] - [L(M | G) + L(D | M, G)].$$

Here, L denotes description length in bits under a specified encoding.

One accounting rule governs everything a decoder may use: anything available to the decoder beyond X is part of the model and is charged in its description length. If a model decodes with the help of a retained training history, that history is written into the model and charged in $L(M)$. No information reaches the decoder uncharged, for either model, in any claim below.

A model earns Turings by describing fresh outcomes more economically than the baseline, after paying for its own complexity. Memorizing the evaluation data does not constitute discovery.

Two further rules accompany the unit:

- **Report the density, not only the total.** Alongside total Turings, report Turings per evaluated bit. I call this the gain density: total gain normalized by evaluation size. Normalization removes the raw growth of the total as the evaluation set grows. The density is not strictly size-independent: a model's description cost is paid once, so it is amortized over more evaluated bits as the set grows, and the density can change with evaluation size for that reason. The density does not by itself determine whether a gain was earned or bought; it states how much gain each evaluated bit carried.
- **Coding must be efficient.** The encoding and decoding procedures associated with a model must run in time polynomial in the size of the model and the data. Without this rule, a description can conceal enormous computation and description length stops measuring explanation.

Proposed mathematical form

I propose a family of learning problems, indexed by a size parameter n , with the following properties:

1. **Compact explanation.** Each hidden world has a generating program with description length $O(n)$.
2. **Efficient execution.** Given that program, outcomes can be predicted in time polynomial in n .
3. **Large explanatory gain.** The evaluation size is fixed exactly: $m(n) = n^2$ outcome bits, at fresh input locations that are pairwise distinct and were never queried during discovery, drawn after discovery closes. On that evaluation set, the generating program earns $\Omega(n^2)$ Turings relative to the uniform baseline, a gain density bounded away from zero.
4. **Persistent discovery barrier.** For every probabilistic polynomial-time learner, the probability of producing a model that earns at least a fixed fraction of the available gain tends to zero as n grows. The probability is taken over the random draw of the hidden world, the observations, the evaluation inputs and outcomes, and the learner's randomness.

I propose 10% as the fraction. Its exact value is secondary to the existence of a separation that persists as the problems grow.

The learner may produce **any effective predictor**; recovering the original generating program is unnecessary.

Conditions for a meaningful test

The baseline, encoding, sampling process, permitted experiments, and evaluation protocol must be fixed in advance. Evaluation outcomes must remain unavailable during discovery, and evaluation inputs must be fresh and pairwise distinct, never queried during discovery. Every submitted model must pay its description cost, including anything its decoder uses beyond the input locations.

The observation and experiment budget must be specified alongside the computational budget and the evaluation size. These choices are part of the conjecture: changing them may change whether the claim holds.

Costs are kept in two separate accounts throughout. Description costs are measured in bits and charged in the Turing accounting. Operating costs, including search time, program maintenance, and library construction, are measured as computation and tracked under the discovery budget. No conversion between the two accounts is used unless a conversion rule is explicitly defined; I define none here.

Three claims, three standards

Claim A: a conditional result, proved. I prove that under the assumption that one-way functions exist, a family satisfying the four conditions exists, with the baseline fixed to uniform on the evaluation outcomes. The route runs through pseudorandom functions: compact keys, efficient evaluation, outputs that resist efficient prediction [5]. The separation phenomenon has precedent in the cryptographic limits on learning established by Kearns and Valiant [6]. Appendix C gives the construction, the sampling rules, and the full proof: a learner beating the barrier would distinguish the pseudorandom function from a truly random one, contradicting its security. The proof is carried out under AIEN’s accounting rules, including the decoder rule, the efficient-coding requirement, and the gain density.

Claim B: an unconditional conjecture, to investigate. The same separation holds with no cryptographic assumption. I have not established a reduction between this claim and the existence of one-way functions, and I do not assert one. Some precisely defined hardness results for time-bounded Kolmogorov complexity are equivalent to their existence [9]; whether anything comparable holds for this formulation is itself an open question. Claim B touches deep open questions in cryptography and complexity theory, and is stated at exactly that strength, no stronger.

Claim C: an AIEN program-reuse hypothesis, to test. Stated as a protocol, so that it can fail:

- *Task family.* A sequence of stochastic-law discovery tasks drawn from one declared generator family whose laws share components, of the kind used in AIEN’s sealed-evaluation experiments [12, 13]: a hidden law, observations during discovery, and a sealed held-out evaluation under a frozen profile.
- *Library-building procedure.* After each task, any program that earned positive net Turings on that task’s sealed evaluation is admitted to the library. Admission charges the program’s description length to the library, and every admitted program is part of the model, charged under the decoder rule, whenever the library is used.
- *Comparison.* Two learners run the same task sequence under equal candidate-evaluation allowances: 20,000 candidate evaluations per task per learner, one with access to the growing

library, one with an empty library throughout. The allowance covers every candidate scored against context data, including library admission checks and library use. Wall-clock time and measured joules are recorded as outcomes, not equalized: the protocol guarantees equal evaluation counts, not equal consumed computation.

- *Hypothesis.* The library learner’s total net Turings across the sequence, with all library description costs charged, exceed the library-free learner’s total under equal candidate-evaluation allowances. Per-task amortization of library costs may also be reported, but the headline figure is the sequence total, with construction costs included, never waived.

Lifelong learning and library learning research have explored versions of this idea [2, 4, 14]; the specific, measured form stated here, with library costs charged in bits and construction counted in the budget, is mine to demonstrate. One boundary is firm: a library independent of a fresh secret key does not defeat a secure pseudorandom function. Reuse helps when tasks share structure. It is not a universal solvent for discovery barriers. Appendix D states the experiment as a draft preregistration specification, with the remaining values to be settled and committed before the first sequence runs.

What would resolve it?

I prove Claim A in Appendix C under the standard assumption that one-way functions exist. A proof of Claim B would establish it outright. A disproof of Claim B would establish that no family satisfies all four conditions under the specified protocol. Efficiently learning one difficult family would not, by itself, disprove an existence claim.

Experiments bear directly on Claim C and can expose weaknesses in the formulation of Claims A and B. They cannot establish the universal claims.

Significance for AIEN

In this conjecture I distinguish **available explanatory value** from **accessible explanatory value**.

Turings measure the gain an explanation delivers. They do not automatically measure the work required to discover it.

For AIEN, I take the foundational question to be Claim C: when do retained programs, accumulated knowledge, and reusable explanations make discovery measurably cheaper, in Turings gained per unit of discovery budget, and when does a computational barrier remain?

Research status

I propose this as a conjecture, not an established theorem or a recognized Millennium Prize problem. Its conditional part is proved in Appendix C by reduction to pseudorandom-function security; its unconditional part is open and not known by me to be equivalent to any standard assumption; its program-reuse part is an empirical hypothesis with named predecessors, now stated as a falsifiable protocol with a draft preregistration experiment specification. Quantitative separations between explanations that exist and explanations efficient algorithms can find already appear in the literature; naming the unit does not by itself constitute a mathematical contribution. What

I offer here is a proof under AIEN’s accounting rules and a working measurement protocol. The program-reuse advantage of Claim C is an untested hypothesis: CEXP-1 has not run.

A world may be easy to explain once its rule is known, yet fundamentally hard to learn.

Acknowledgments and provenance

The proof technique of Appendix C, the counting lemma and the bounded sampler, follows an independent draft by GPT-6 Astra, shared with me on 2026-09-30; I have restated both in this document’s notation and protocol. The problem statement, the accounting rules, the draft CEXP-1 preregistration specification, and the references are my own. The proof as written stands on its own derivation; this note records provenance, not a dependency.

A Valid compression

A submission is scored only if its code is valid, in the following sense.

- **Prefix-free codes.** All description lengths are lengths under a prefix-free binary code: no codeword is a prefix of another. By the Kraft-McMillan theorem, a set of lengths is realizable by a uniquely decodable code exactly when the corresponding Kraft sum satisfies $\sum_i 2^{-\ell_i} \leq 1$ [3, 11]. Unique decodability is what makes a length an honest count of information.
- **Lossless decoding.** Decoding the submitted encoding, given the input locations X and the charged model, must reproduce the evaluation outcomes exactly. A short output that does not decode to the actual outcomes is not compression; it is an error.
- **Invalid submissions earn no score.** If decoding fails, is lossy, or runs outside the polynomial time bound of the efficient-coding rule, the submission is rejected and earns no Turings. There is no partial credit for approximate reconstruction; approximation belongs in the model’s predictions, where its cost is paid openly in the conditional description length.
- **Uniform scoring bound.** The profile fixes one polynomial step budget for the whole evaluation: encoding, decoding, and reconstruction checking, including the overhead of the simulation that runs the submitted code. The scorer executes submitted code under that budget and rejects timeouts or failed reconstructions. It does not attempt to decide the asymptotic runtime bounds of arbitrary programs; it only enforces the declared budget. The key-holding reference program fits within the chosen budget. This is what the Claim A proof relies on when it scores a submitted model: the reliance is on the scorer, stated here as a requirement.
- **One code, fixed in advance.** The encoding scheme, and therefore the meaning of L , is part of the frozen profile. It cannot be chosen after seeing the evaluation data.

B Complete probability model

A learning problem in the family is fully specified by the following, all fixed in advance:

1. **World sampling.** A declared, efficiently samplable distribution over generating programs of the family at size n . The hidden world is one draw from this distribution.

2. **Input space and input distribution.** A declared set of input locations and a declared, efficiently samplable distribution over them. Evaluation inputs are drawn from this distribution, conditioned on never having been queried during discovery and on being pairwise distinct.
3. **Discovery interaction.** During discovery the learner may adaptively query input locations of its choice and receive the true outcomes, up to a declared query budget, in the style of query learning [1, 15], and may also receive observations drawn from the input distribution, up to a declared observation budget. Both budgets are polynomial in n and are part of the problem specification.
4. **Freeze point.** When the budgets are exhausted, the learner outputs one model and is frozen. No further queries, updates, or selection among models is permitted after the freeze. Model selection performed during discovery is part of the learner and is charged to its computation budget.
5. **Evaluation draw.** After the freeze, $m(n) = n^2$ fresh, pairwise distinct input locations are drawn, their outcomes computed by the hidden generating program, and the frozen model is scored exactly once under the accounting rules.
6. **Success and failure.** The available gain is the frozen evaluation score of the generating program itself. A learner succeeds if its score is at least the declared fraction (proposed: 10%) of the available gain. Claims A and B assert that, over the draws in steps 1, 2, and 5 and the learner's own randomness, the success probability of every probabilistic polynomial-time learner tends to zero as n grows.

C Instantiation for Claim A: the pseudorandom-function family

The conditional construction, with every sampling rule made explicit:

- **World sampling.** Draw a key k uniformly at random from $\{0, 1\}^n$. The generating program is the pseudorandom function F_k of Goldreich, Goldwasser, and Micali [5], built from a pseudorandom generator, which exists if one-way functions exist. The program consists of the key plus a fixed evaluator: description length $O(n)$, evaluation in time polynomial in n .
- **Input space and distribution.** Inputs are n -bit strings, $x \in \{0, 1\}^n$, so the domain has 2^n locations, large enough that exhaustion and collision are not strategies. The input distribution is uniform, which is efficiently samplable.
- **Discovery interaction.** The learner makes adaptively chosen input queries and receives $F_k(x)$ for each, up to a query budget of $q(n) = n^3$. Let Q be the set of all inputs whose outcomes were revealed during discovery, by query or by observation. Then $|Q| \leq n^3$, negligible against 2^n .
- **Evaluation draw.** Draw $m(n) = n^2$ evaluation inputs uniformly without replacement from $\{0, 1\}^n \setminus Q$. The evaluation inputs are therefore pairwise distinct and disjoint from everything the learner has seen. Distinctness is load-bearing: a repeated input would let any model earn from repetition itself, and an input repeated from discovery would let a charged transcript answer it, so compressible repetition could appear even though the function resists discovery. Outcomes are the bits $F_k(x)$, one per input, for n^2 outcome bits in total.

- **Baseline.** The baseline is uniform on the evaluation outcomes: each bit is assigned probability one half. The uniform predictor needs no parameters, so $L(B)$ is a fixed constant, and $L(D \mid B, X) = m$ for every D .
- **Available gain.** The key-holding program predicts every evaluation bit. Under the frozen code its score is the baseline cost of n^2 near-random bits, minus its $O(n)$ description and a small conditional cost: $T(P_k; B, D, G, P) = n^2 - n + O(1)$, hence at least $n^2/2$ Turings for all large n , an $\Omega(n^2)$ gain with density approaching 1, bounded away from zero.
- **Barrier: proof of Claim A.** Suppose, for contradiction, that some probabilistic polynomial-time learner A violates Condition 4: for some constant $c > 0$ and infinitely many n , A earns at least one tenth of the available gain with probability at least c , the probability taken over the key draw, the discovery interaction, the evaluation draw, and A 's randomness. For large n the available gain is at least $n^2/2$, so success implies a Turing score of at least $n^2/20$. The argument works for any fixed positive fraction, and for any polynomial discovery budget, not only n^3 .

One lemma does the work.

Counting lemma. Fix the discovery transcript, the resulting frozen model M , and the evaluation input list X . Assume the baseline B is uniform on the evaluation outcomes, so that $L(D \mid B, X) = m$ for every D . If the m evaluation outcome bits are independent uniform bits conditional on those fixed objects, then for any threshold s ,

$$\Pr[T(M; B, D, G, P) \geq s \mid \text{transcript}, M, X] \leq 2^{L(B)-s}.$$

Proof of lemma. By the uniform-baseline hypothesis, scoring at least s requires $L(D \mid M, X) \leq L(B) + m - L(M) - s$. Call that bound r ; if $r < 0$ the event is impossible. The residual code is prefix-free, so by the Kraft inequality it contains at most 2^r codewords of length at most r , and each codeword decodes to at most one outcome string D . Of the 2^m equally likely outcome strings, at most 2^r can reach the threshold, so the probability is at most $2^{r-m} = 2^{L(B)-L(M)-s} \leq 2^{L(B)-s}$. The bound holds even for a randomized encoder: it counts decodable short residuals, not encoder behavior. Averaging over transcripts, models, and input lists preserves it. $\square$

Now construct a probabilistic polynomial-time distinguisher D with oracle access to $O : \{0, 1\}^n \rightarrow \{0, 1\}$:

1. Simulate discovery: run A , answering its adaptive queries with O ; let Q be the set of all revealed inputs.
2. Let M be the frozen model A outputs.
3. Simulate the evaluation draw with a bounded sampler: repeatedly draw uniform inputs, accepting only points outside Q not already accepted, until n^2 inputs are accepted or $2n^2$ attempts are made; on failure, output 0. Query O for the accepted inputs' outcome bits and compute M 's Turing score under the frozen profile and baseline. Output 1 if the score is at least $n^2/20$, else 0.

D runs in polynomial time: A does, it makes polynomially many oracle queries, and scoring a valid model is polynomial by the efficient-coding rule. An invalid model scores nothing. The bounded sampler keeps D strictly polynomial rather than relying on unbounded rejection sampling. Its failure probability is at most $4^{n^2} a^{n^2}$ with $a = (|Q| + 2n^2)/2^n$: failure needs at

least n^2 rejected attempts among $2n^2$, each attempt rejects with conditional probability at most a , and a union bound over the choices of the rejected positions gives the bound. The exact count is the binomial coefficient $\binom{2n^2}{n^2}$, and 4^{n^2} is a valid but loose upper bound for it; the looseness is harmless, since only negligibility is needed. Since $|Q|$ is polynomial, a is far below $1/4$ for large n and the failure probability is negligible; failure only lowers acceptance.

If $O = F_k$ for a uniform key k , the simulation reproduces the Claim A world, and D 's evaluation draw has the game's distribution whenever the sampler completes, so D outputs 1 with probability at least $c - \text{negl}(n)$ on the infinite subsequence.

If $O = R$ is a truly random function, the outcome bits at the fresh accepted points are independent uniform bits even after conditioning on the adaptive discovery transcript and the accepted input list, so the counting lemma bounds D 's acceptance probability by $2^{L(B)-n^2/20}$. The baseline length $L(B)$ is a constant, so this is negligible in n .

The distinguishing advantage is therefore at least $c - \text{negl}(n)$, bounded away from zero on the infinite subsequence. This contradicts the pseudorandomness of the Goldreich-Goldwasser-Micali family [5], which holds if one-way functions exist. Hence no such learner A exists: every probabilistic polynomial-time learner's success probability tends to zero. That is Condition 4, and Claim A is proved. $\square$

D The Claim C experiment, draft preregistration specification (CEXP-1)

- Apparatus and implementation identity.** The AIEN stochastic-law apparatus in its EXP-002 "Stochastic Law Discovery" configuration: candidates compete to explain sealed held-out trajectories from a freshly generated stochastic law [12, 13]. Measurement profile: turing-profile v1.0 (draft; any change is a new version). Coder: the profile's reference coder, theoretical description length under declared predictive distributions. Quantizer Q : the scored quantity is the one-step increment $\Delta_t = x_t - x_{t-1}$, not the absolute position: absolute positions of a diffusion wander without bound, so a fixed position grid piles mass into its edge bins. Increments are quantized into 256 uniform bins over $[-5, 5]$; out-of-range increments are clipped to the edge bins. The generator family declares its maximum per-step diffusion coefficient in the preregistration manifest, and a run is valid only if the clipped fraction is reported and stays at or below 5.0% of sealed observations. If more than 5.0% of sealed observations fall outside $[-5, 5]$ before quantization and are therefore clipped, the affected preregistered condition is invalid for the primary claim; if CEXP-1 has only one primary condition, that invalidates the run. The failed run is preserved and reported, not deleted and rescored. Clipping is measured on actual out-of-range values before quantization, not on edge-bin occupancy: raw pre-quantization values are retained so the gate uses the true clipped fraction, with edge-bin occupancy reported as a diagnostic. Baseline: uniform over quantization bins, 8 bits per scored increment. Model cost convention: 32 bits per fitted real parameter, 8 bits per small integer hyperparameter selected on context data.
- Generator configuration.** Each task's law is freshly generated by the declared generator family with a sealed seed. For Claim C the family is configured so that laws are composed from a shared component pool, so successive tasks genuinely share structure; the component pool and composition rule are written into the preregistration manifest before the first sequence runs.

- **Task and sequence structure.** A sequence is 12 tasks. Per task, discovery receives 8 context trajectories of 400 observations each. Evaluation is one-step-ahead prediction on 4 sealed trajectories of 400 observations, scoring predictions at steps 1 to 399 of each, for 1,596 scored observations per task.
- **Learner settings.** One discovery procedure, identical hyperparameters in both configurations. The library learner retains only programs admitted under Claim C’s admission rule (positive net Turings on the task’s sealed evaluation; description length charged). The comparison learner retains nothing across tasks. Both learners see identical task sequences, paired by seed.
- **Budgets.** Per task and per learner, the discovery budget is 20,000 candidate evaluations, where one candidate evaluation is one candidate scored against the task’s context data. Library admission checks and any library use are candidate evaluations and count against the same allowance. Observation access is exactly the context trajectories above; there are no additional queries. The run executes on one explicitly identified DGX Spark (GB10) machine configuration, exclusively, with identical execution limits for both arms. Wall-clock time and measured joules are recorded outcomes, not stopping rules: the protocol equalizes the number of candidate evaluations each learner may spend, and the physical costs each learner actually consumed are reported alongside the scores, not assumed equal. Library construction, admission, and maintenance cost real time and energy, and those costs fall on the library learner. A runaway ceiling guards against unbounded runs: the wall-time and energy abort ceilings are a fixed multiple of the measured worst-case development-seed cost from an unsealed calibration run, and the resulting exact hours and joules are written into the preregistration manifest. A ceiling is an abort rule, not an equalizer of consumed computation: a ceiling violation aborts the learner, it does not grant more evaluations.
- **Replication.** 30 paired sequences, the two configurations run on identical seeds and identical task draws.
- **Criterion and uncertainty analysis.** The primary criterion is the expected total net Turings across the 12-task sequence, all library costs charged. The effect is estimated by the mean paired difference of sequence totals across the 30 replications, with a 95 percent confidence interval from a percentile bootstrap over sequences (10,000 resamples). The hypothesis counts as supported only if the interval lies entirely above zero; if it straddles zero the result is reported as inconclusive, not as support. The full distribution of paired differences is reported alongside the mean.
- **Preregistration.** This specification is a draft until four items are settled and committed before the first sequence runs: the runaway-ceiling multiplier and the resulting wall-time and energy limits, the treatment of a learner that aborts at the ceiling, the sequence-level library-cost ledger formula, and the encoding costs for program structure and component selection. With those committed, the specification, the generator configuration, and the analysis method are frozen before the first sequence runs. Deviations void the run’s claim to test Claim C.

References

- [1] Angluin, D. (1988). Queries and concept learning. *Machine Learning*, 2(4), 319-342.
- [2] Chen, Z., and Liu, B. (2018). *Lifelong Machine Learning* (2nd ed.). Morgan and Claypool.

- [3] Cover, T. M., and Thomas, J. A. (2006). *Elements of Information Theory* (2nd ed.). Wiley.
- [4] Ellis, K., Wong, C., Nye, M., Sablé-Meyer, M., Morales, L., Hewitt, L., Cary, L., Solar-Lezama, A., and Tenenbaum, J. B. (2021). DreamCoder: Bootstrapping inductive program synthesis with wake-sleep library learning. *Proceedings of PLDI 2021*, 835-850.
- [5] Goldreich, O., Goldwasser, S., and Micali, S. (1986). How to construct random functions. *Journal of the ACM*, 33(4), 792-807.
- [6] Kearns, M. J., and Valiant, L. G. (1994). Cryptographic limitations on learning Boolean formulae and finite automata. *Journal of the ACM*, 41(1), 67-95.
- [7] Kolmogorov, A. N. (1965). Three approaches to the quantitative definition of information. *Problems of Information Transmission*, 1(1), 1-7.
- [8] Li, M., and Vitányi, P. (2019). *An Introduction to Kolmogorov Complexity and Its Applications* (4th ed.). Springer.
- [9] Liu, Y., and Pass, R. (2020). On one-way functions and Kolmogorov complexity. *Proceedings of FOCS 2020*, 1243-1254.
- [10] Rissanen, J. (1978). Modeling by shortest data description. *Automatica*, 14(5), 465-471.
- [11] Shannon, C. E. (1948). A mathematical theory of communication. *Bell System Technical Journal*, 27(3), 379-423, and 27(4), 623-656.
- [12] Stapleton, M. D. (2026). *Computing Machinery and Understanding: From the Imitation Game to the Turing*. AIEN Project, v1.13. aienos.com/research.
- [13] Stapleton, M. D. (2026). *The Turing as a Measure Emerging from AIEN*. AIEN Project, v1.7. aienos.com/research.
- [14] Thrun, S., and Mitchell, T. M. (1995). Lifelong learning: A dialogue on situated and integrated learning agents. In *Working Notes of the AAAI Spring Symposium on Integrated Learning Architectures*.
- [15] Valiant, L. G. (1984). A theory of the learnable. *Communications of the ACM*, 27(11), 1134-1142.

The Stapleton Doctrine

The philosophy. Why property beats rent.

The Stapleton Doctrine

A Personal Philosophy of AI Architecture and Programming

M. Drake Stapleton

October 2, 2026

Version 1.5

English is the conversation. The verified program is the thing that was said. I state here why the universal language of machines must be built, verified, and owned, not rented.

A philosophical essay and manifesto, distinct from the measurement papers of this project. This one argues a position; those report measurements.

Abstract

On September 23, 2026, at Rails World 2026 in Austin before an audience of over a thousand developers, David Heinemeier Hansson, the creator of Ruby on Rails, retired as a professional programmer and declared: “English is a better programming language than Ruby” [1]. I take that claim as stated and reject it. English is ambiguous; two agents can read one sentence two different ways; and there is no way to prove a sentence means what you think it means. The universal thing in my architecture is not a spoken language at all. It is the verified program: a small, checkable artifact, with a measured receipt, that any machine can recheck without trusting the machine that wrote it. I set down here the doctrine behind that architecture: how meaning is supposed to flow from suggestion to verification to evidence, how the software stack is to be stripped layer by layer down to a foundation the author owns outright, and why renting an interpreter forever is not the same as owning understanding. The argument builds on earlier marks in the same debate: Ada Lovelace on what machines can originate, Edsger Dijkstra on what tests and plain language can prove, C.A.R. Hoare on the difficulty of simplicity, and Richard Stallman on who controls whom.

1 Introduction

Watch anyone experienced work with an AI coding agent and you will see the same pattern: the human talks, the agent writes code, the human tests, the agent fixes. David Heinemeier Hansson says this pattern proves that hand-written code is over and that English is the real programming language now. His exact line, from the Rails World 2026 keynote in Austin: “English is a better programming language than Ruby” [1]. The reasoning goes like this: programming was always the translation of human ideas into machine instructions; the machine now understands human language directly; therefore the translation layer disappears, and whatever sits underneath can be a black box that nobody ever reads.

I agree with one half of that argument and reject the other.

The half that is right: talking *is* becoming the main interface. Humans should not have to hold a compiler in their heads. A person who thinks clearly but never learned to write code in detail should still be able to build serious systems by working with capable agents.

The half that is wrong: the idea that the bottom does not matter because nobody reads it. It matters more than the top. If English is the language and a rented model is the compiler that decides what your English means, then whoever owns the model owns the meaning of your instructions. That is not the end of the meter. It is the meter moving up one layer, out of sight, where you can no longer measure it.

I owe the other side one honesty before I go on. Preferring English as the interface does not by itself force perpetual dependence on a vendor. A model running on your own machine, and generated artifacts you keep and can inspect, break the binary: the interface can be English while the interpreter and the artifacts are yours. I reject the rented form of the interpreter picture, not every arrangement that involves talking. What I claim is narrower, and I state it as a design preference under explicit threat models rather than as the only coherent alternative. The threats are three, and I name them: who can cut you off, who sets the price, who holds the off switch. If your arrangement answers all three to your satisfaction, renting is rational, and nothing in this paper disputes it. Mine does not, so I build to own.

2 The Disagreement, On the Record

The position answered in this paper belongs to a named person and has a fixed text. On September 23, 2026, at Rails World 2026 in Austin, David Heinemeier Hansson closed a quarter century of writing code by hand and opened what he calls his retirement as a professional programmer. He has not written code by hand since about March 2026; by August he was generating around 150,000 lines a month, much of it verbose Rust he states outright he does not want to look at. In that context he said the sentence this paper answers: “English is a better programming language than Ruby”[1]. He evaluates the agent-written code “as a black box ... as any business owner in history”[1], and he names the future role the “professional maker of things”[1].

I grant the generous half of this position. Talking is becoming the interface, and Hansson is right that a careful person can now make real things through agents. I reject the rest. A black box you own and may open is one thing; a black box whose contents live below the hand of a machine you rent, and whose meaning is settled by interpretation, is another. The difference between the two is the whole argument of this paper.

There is a precedent for answering claims by name. Alan Turing, in “Computing Machinery and Intelligence” (1950), takes the objection he attributes to Ada Lovelace, labels it “Lady Lovelace’s Objection,” and answers it by name. I follow that courtesy: the claim gets its speaker’s name, its date, and its exact text, and is then met on its merits[2].

The claim also has a history older than the current argument, and the history is instructive, but I will not overclaim it. English-like syntax as a design goal is not a new idea. COBOL, designed in 1959 under the CODASYL committee at Grace Hopper’s impetus, was made to read like English on purpose, so that business managers could follow their own programs without thinking in the machine’s terms[9]. But COBOL’s English-likeness is a fixed vocabulary and a fixed grammar; it is not unrestricted natural language, and no statistical model interprets it. It tested the accessibility of English-like syntax, not the proposal on the table now. The verdict handed down was a design judgment, not an experiment. Edsger Dijkstra, the man who spent his career arguing that correctness must be built in rather than wished in, wrote of COBOL that “The use of COBOL cripples the mind”[6]. I quote him as a critic of a language design, not as evidence about natural-language programming, which his line does not address. In the same body of work he warns that “program testing can be used to show the presence of bugs, but never to show their absence”[4], and supplies the line that belongs on any wall of this debate: “The purpose of abstracting is not to be vague, but

to create a new semantic level in which one can be absolutely precise” [5]. English is precisely vague, endlessly interpretable, and semantic levels built on it inherit that property. What the COBOL episode does teach, and what I keep, is smaller than a verdict: legibility is not rigor. Hopper’s aim served its cause; the rigor still has to live somewhere, and I put it in the verified artifact.

3 The Claim

English is the conversation. The program is what was actually said.

My central claim is simple. The candidate for a universal language of machines is not English, and it is not any language a human already speaks. It is the verified program. A program counts as a program here only if it passes every check thrown at it and carries the receipt to prove it.

“Universal” does not mean everyone can read it. It means no one has to take anyone’s word for it. A sentence can be argued with. A receipt makes its evidence inspectable and recheckable. Its specification, verifier, assumptions, and applicability remain open to challenge. If the artifact is checkable, any machine on earth can verify it independently. That property, and not human readability, is what makes an artifact universal.

The smallest such artifact in this architecture is called a **crumb**. A crumb is a fragment of program: drafted by an agent, checked by an independent verifier, measured for what it gains and what it costs, and filed with evidence. Programs are built the way cities are built, from the curb upward; each block laid is proven before the next one rests on it. I offer this paper as the philosophy; the engineering rules for crumbs live in the sovereign-lineage doctrine that governs my repositories.

Note what this turns down. It turns down “the code doesn’t matter because agents write it.” In this doctrine, the code is the only thing that matters. It is the evidence that the conversation ended in something real.

4 The Chain

Nothing in this architecture is believed on arrival. Everything passes through the same loop (Figure 1):

Weights suggest. Programs explain. Omega defines and synthesizes. Physics realizes and measures. Evidence teaches.

A model’s weights propose a candidate. The candidate is restated as an explicit program, the crumb, which *explains* the proposal in the sense the loop demands: a procedure a machine can run and a checker can examine. Omega, the definitional layer, defines the semantics of the claim and synthesizes it into something with crisp boundaries. PHYSICS realizes the claim in measured execution: watts drawn, bits observed, outcomes counted; only what is measured counts as real. And evidence teaches, because the outcome of the measurement feeds back into the weights, so every loop ought to begin with a better suggestion than the last.

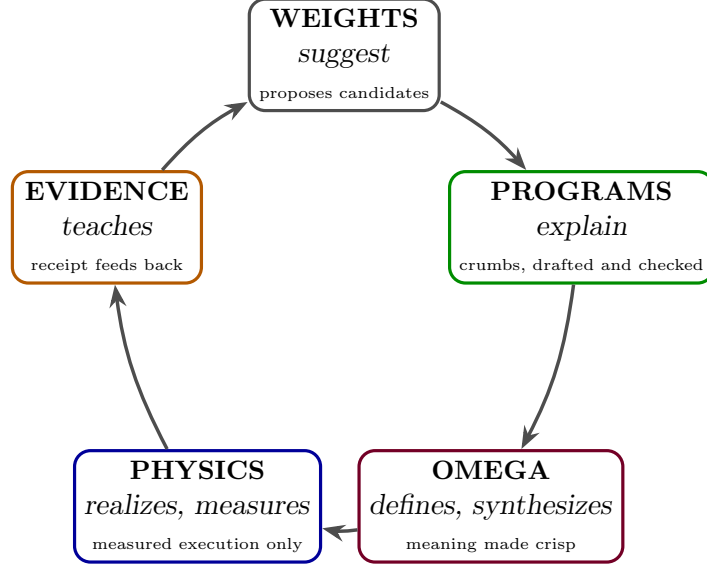


Figure 1: The verification loop. Weights suggest, programs explain, Omega defines and synthesizes, Physics realizes and measures, evidence teaches. Evidence returns to the weights, so every pass through the loop can begin with a better suggestion than the last.

A word about the first two steps of the loop. Writing in 1843, Ada Lovelace held of Babbage’s Engine that “The Analytical Engine has no pretensions whatever to *originate* anything. It can do whatever we know how to order it to perform” [3]. I extend her line one step without breaking it: a large model can order its own suggestions, but the suggestions still originate nothing. They become something only when a program embodies them, the witness verifies them, and physics realizes them. Suggesting and ruling are kept apart for the same reason. Lovelace’s doubt stands, answered now by proof.

A word about the verifying seat. In the working machine it is held by independent verifiers. The witness verifies what the agents generate, including every crumb, and leaves everything as he found it: he changes nothing and is not part of the generation system. A word about the authorizing seat. In the working machine it is held by AEGIS. He certifies what the verifiers have checked, according to policy and the verification evidence, and leaves everything as he found it: he changes nothing, decides no policy, and is not part of the generation system. A checker that rewrites the checked work is a second author, not a witness. The doctrine insists on a witness.

The loop is where meaning lives. English proposes at the top; something else, something verifiable, is what is actually said. The loop is also why this doctrine consumes less trust than the alternatives: each stage can be re-run independently, on your own machine, with your own eyes on the numbers. Trust is a tax. The loop is how you stop paying it to someone else.

5 Two Futures

The argument over the universal language reduces to two pictures of the future. Figure 2 places them side by side. They begin at the same point, a human talking to a machine, and they end in opposite places.

In the first future, English is declared the universal language and the machinery underneath is waved off as a black box. Nobody reads it; nobody needs to. But somebody pays for it, per token,

forever, and somebody else decides what the words meant. The interpreter that matters is rented, and interpretation is the thing you can never stop buying.

This picture is the rented form of that future, and I paint it as the rented form. I grant the owned form: the model runs locally, the artifacts are filed where you can inspect them, and English stays the interface. Nothing in that arrangement rents you the meaning of your instructions. The danger is not English as the interface; it is dependence you cannot exit.

In the second future, the conversation stays at the top, where humans are good, and verification descends into the machine, where proof lives. Programs are drafted, checked, measured, and filed. The model never gets to rule; it gets to suggest, and its suggestions are graded in the open. This future ends when there is nothing left to rent.

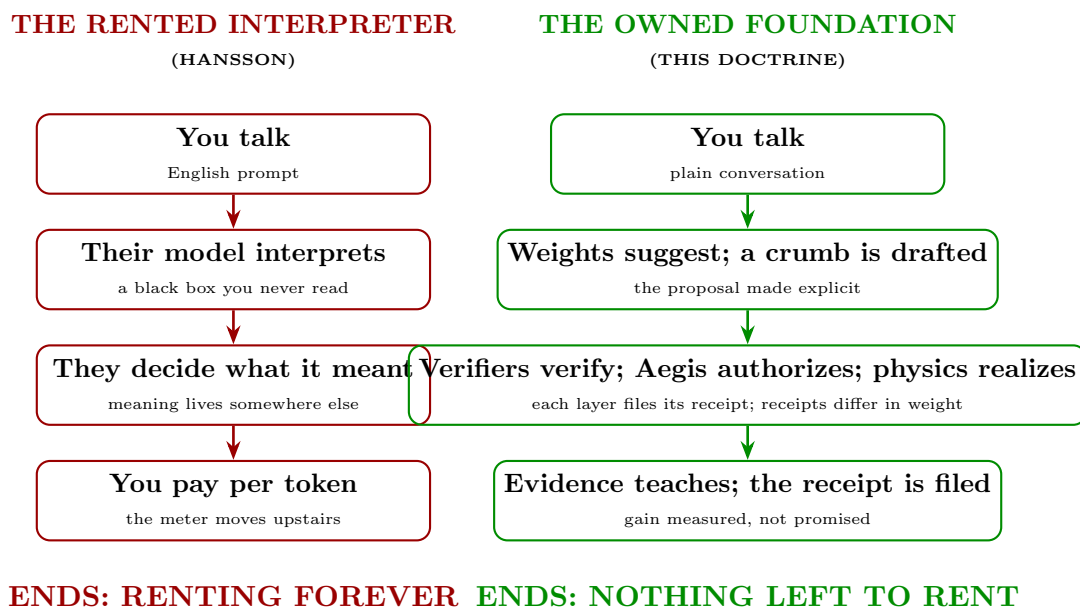


Figure 2: Two futures, side by side. Left: the human talks; a rented, opaque model interprets; the owner of the model owns the meaning; the meter never stops. Right: the human talks; programs are drafted, verified, and measured; the evidence is filed where anyone can check it.

The left picture is sold as freedom, “everyone can program now.” Hansson is sincere and consistent about the terms: he admits in the same breath that English is vaguer and less deterministic than the tool he gave up, and calls it a trade rather than a discount[1]. The terms of the trade are the issue, not the sincerity of the man who stated them: the foundation is declared irrelevant while it stays under someone else’s control, and the interpreter decides what your words meant, per token, forever. The right picture asks for more work, but every bit of that work buys something: one more artifact you can prove, one more layer no one can take away. Which of the two pictures is freedom depends on where you stand: I stand at the bottom, with the receipt.

6 The Ladder

I am not content with verifying the crown of the stack. I want the foundation. The machine a verified artifact runs on has to be ownable below the artifact, or verification is theater. So the stack is to be stripped layer by layer, down to a foundation no one else can rename, reprice, or revoke. Figure 3 shows the descent, and the doctrine’s rule for it: the ladder only goes one direction.

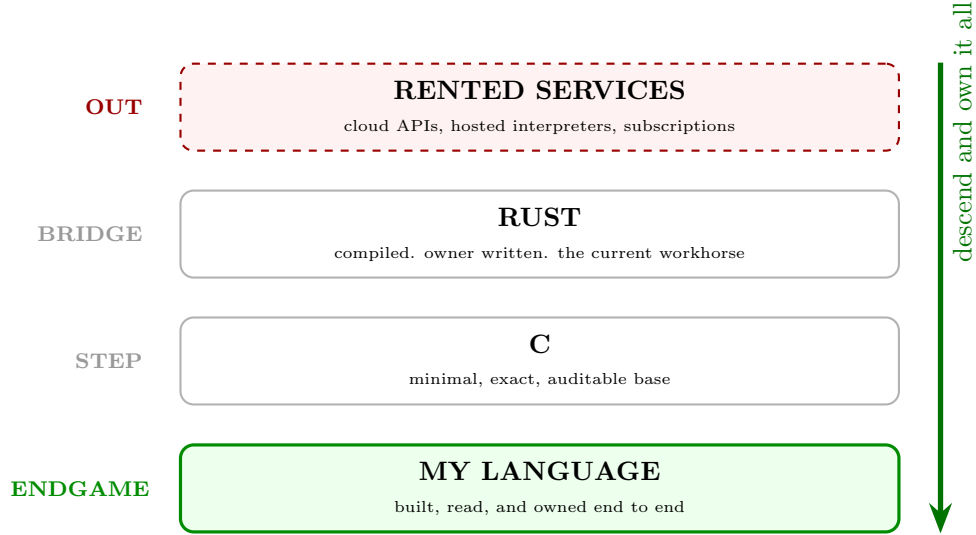


Figure 3: The descent. Rented services and hosted interpreters go first. Rust is the bridge. C is the next step down. Nothing finishes at the bottom until the foundation is the author’s own language. The ladder only goes one direction.

The choice to descend is the harder of the two designs C.A.R. Hoare named in his 1980 Turing lecture: “One way is to make it so simple that there are obviously no deficiencies, and the other way is to make it so complicated that there are no obvious deficiencies. The first method is far more difficult” [7]. Renting is the easy method with good lighting: a black box you can never open has no obvious deficiencies by construction, and its errors surface only when you can least afford them. Stripping the stack down to a simple bedrock you own and can read is the hard method, and Hoare’s line is the defense for doing it anyway. Difficulty is the feature, because simplicity is what proof can see.

The ladder is not an aspiration; it is a history. The whole project started when I bounced off someone else’s language, took their layer out, and put my own in. That substitution is how AIEN began. What follows is the same move repeated on schedule: strip the hosted pieces, move down through the migration steps, and never stop before the bedrock.

Two clarifications matter, and the first needs a third. First, C is a step on this ladder, not the destination. It narrows the foundation to a minimal, exact base, but my doctrine does not end in C; it ends when the language at the bottom is mine all the way down. Second, the stripped layers are not denied, they are replaced only when a proven replacement stands under the load. The ladder descends on evidence, not on declarations. Third, I do not claim that C is safer than Rust; the move down purchases no safety. What it purchases is a smaller base with fewer dependencies, chosen for audibility under the threat models I named: fewer parties who can cut me off, fewer hands on the price. Safety is a property of verification, not of the rung. And a word about what counts as evidence along the way. Dijkstra’s warning from 1970 holds: “Program testing can be used to show the presence of bugs, but never to show their absence” [4]. A passing test says the bug you thought to look for was not there that day. A receipt is a measured result, filed with its method, ready to be re-run and re-checked by anyone. The ladder descends on the second kind of evidence, not the first.

7 Property, Not Rent

The machine that runs this doctrine has a name for the goal of the whole arrangement: **intelligence as property, not rent**. Figure 4 renders the machine in full. Read top to bottom, it is a chain of custody for meaning: thought is proposed, definition is made, verification is performed by an independent witness, and what is verified is what the silicon executes. Nothing runs without the receipt of every layer it passed, and evidence from below feeds back up, because the loop must teach or it is decoration. The receipts are not all one kind. A proof-backed receipt says the argument closes. A rerunnable test or measurement receipt says the run was observed and can be repeated, and it carries the epistemic limits of a test: it shows what happened, not what could not happen. Both are filed. They are not weighed the same.

Richard Stallman stated the underlying rule a generation ago, about the software on your own disk: “The users (both individually and collectively) control the program and what it does for them. When users don’t control the program, the program controls the users” [8]. I apply the same rule one layer up, to the model that interprets your English. If the interpreter is someone else’s program under someone else’s control, then it is no metaphor to say the interpretation is not yours either. Hansson presented his black box as the end of a burden; Stallman’s line reframes it as the beginning of a custody.

This is why the doctrine and the rented-interpreter future cannot be combined by compromise. One says “the bottom does not matter.” The other says the bottom is the only thing you can truly own. Property means: the artifact on your machine can be re-run, re-measured, and re-verified by you, tomorrow, without a subscription and without anyone’s permission. Rent means someone else’s black box deciding what your instructions meant today, and charging you for the next round. I choose property over rent, and state the choice in an architecture rather than in a slogan.

8 The Doctrine in Tenets

The Stapleton Doctrine, stated whole:

1. **English is the conversation, not the language.** Talking is how humans steer. It is not what gets executed, not what gets verified, not what gets remembered.
2. **The verified program is the universal artifact.** Universal means checkable without trust, not readable by everyone. The crumb, with its receipt, is the currency of this architecture.
3. **Weights suggest; they do not rule.** Suggesting and ruling are different offices, and the doctrine keeps them held by different holders.
4. **Meaning descends the loop.** Suggestion, explicit program, clarified definition, measured execution, filed evidence. What arrives at the bottom is what the receipt can support: empirical adequacy under the receipt. The best-compressing model need not be the true one.
5. **The witness verifies; AEGIS authorizes.** Independent verifiers check what agents generate and change nothing, and belong to no part of the generation system. AEGIS certifies what the verifiers have checked, according to policy and the verification evidence. Without a separate witness, verification is just more authorship.
6. **PHYSICS realizes and measures.** If it was not measured, it was not done. Documentation, claims, and architecture diagrams never outrank an execution receipt.

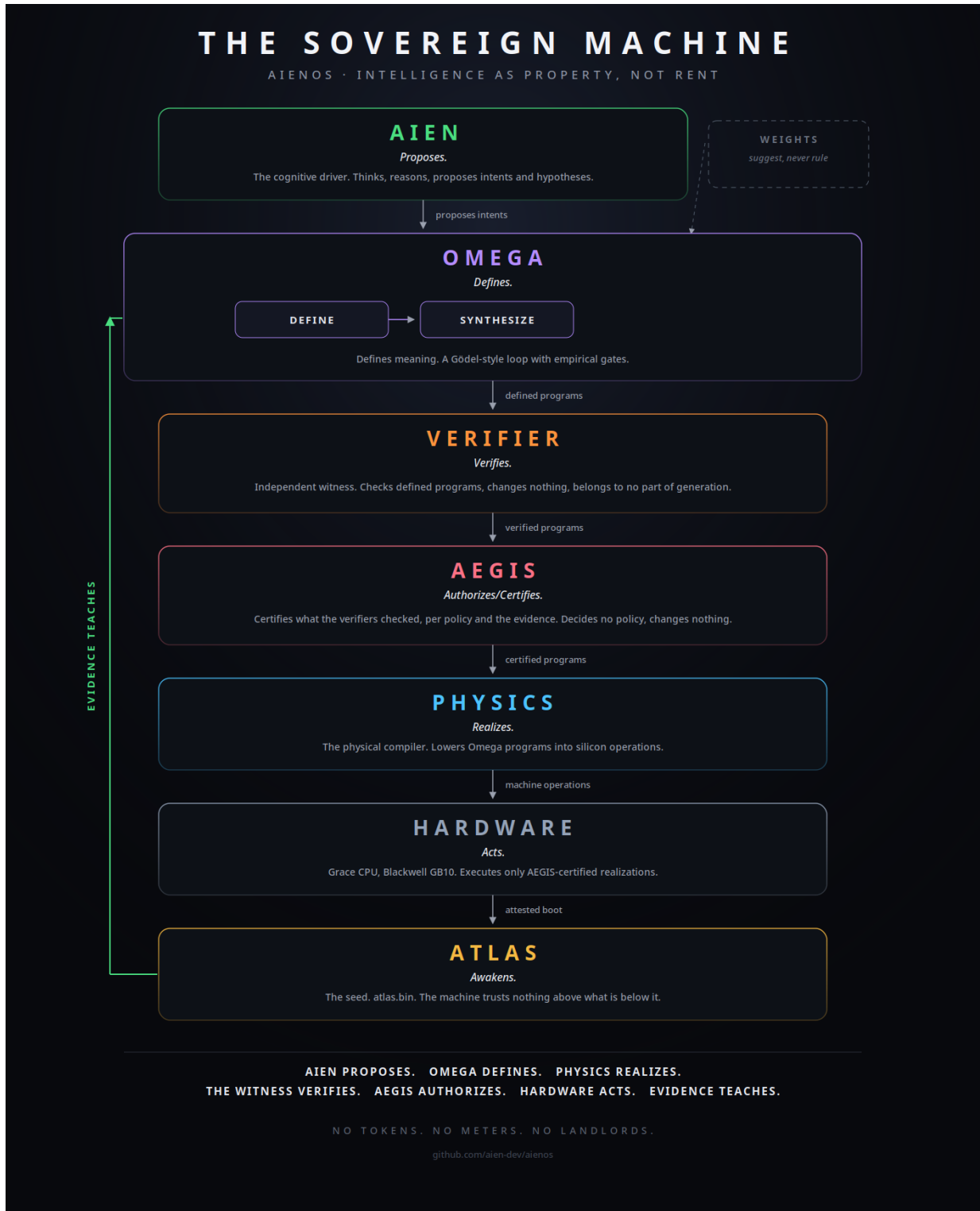


Figure 4: The Sovereign Machine: the pipeline that proposes, defines, verifies, authorizes, realizes, and executes, with evidence flowing back up to teach the mind. No tokens, no meters, no landlords.

7. **Strip the stack to bedrock.** Hosted layers go first. Migration steps like C earn their place as steps. The foundation is not finished until it is owned end to end in the author’s own language.
8. **Nothing is removed until it is replaced.** A layer is removed only when a proven replacement stands under the load. The ladder descends on evidence, not on declarations.
9. **Own the foundation or you rent the meaning.** Whoever owns the interpreter decides what your words mean. The doctrine ends when there is nothing left to rent.
10. **Eyes and ears first, industry second.** The loop must be independent of any vendor before it is convenient. Property is built before it is productized.

The Stapleton Doctrine is my philosophy behind AIEN, Omega, FORGE, AEGIS, and the Turing unit that measures their progress. English is allowed to be the conversation forever, but the machine will never be told that English is the language, because a language you cannot verify is a language you cannot own. What is said is a program; what is kept is the proof.

Revision History

- v1.5 (October 2, 2026).** Reconciliation pass: receipt-challenge language made consistent across the collection (“A receipt makes its evidence inspectable and recheckable”); DHH citations consolidated on the primary keynote source. Version set to v1.5.
- v1.4 (October 2, 2026).** Reconciliation pass. The receipt formulation was corrected: a receipt makes its evidence inspectable and recheckable, and its specification, verifier, assumptions, and applicability remain open to challenge. A trust passage was aligned to the same standard. Version set to v1.4.

References

- [1] David Heinemeier Hansson, opening keynote, Rails World 2026, Austin, TX, September 23, 2026. The quoted passage, “English is a better programming language than Ruby,” occurs in the segment on English as the programming language, beginning at approximately 38:06. https://www.youtube.com/watch?v=vDjW_dRyKXY
- [2] Alan M. Turing, “Computing Machinery and Intelligence,” *Mind* **59**(236), October 1950, 433-460.
- [3] Ada Lovelace, notes on L.F. Menabrea, “Sketch of the Analytical Engine,” with notes on the memoir by the translator, *Taylor’s Scientific Memoirs* **3** (1843), 666-731. Note G carries the remark on origination.
- [4] Edsger W. Dijkstra, “Notes on Structured Programming” (EWD249), 1970. The dictum on testing appears as a corollary at the end of section 3.
- [5] Edsger W. Dijkstra, “The Humble Programmer” (EWD340), 1972 ACM Turing Award lecture, *Communications of the ACM* **15**(10), October 1972, 859-866.

- [6] Edsger W. Dijkstra, “How do we tell truths that might hurt?” (EWD498), 1975. Published in *Selected Writings on Computing: A Personal Perspective*, Springer, 1982, and in *ACM SIGPLAN Notices* **17**(5), May 1982, 13-15.
- [7] C.A.R. Hoare, “The Emperor’s Old Clothes,” 1980 ACM Turing Award lecture, *Communications of the ACM* **24**(2), February 1981, 75-83.
- [8] Richard M. Stallman, “What is Free Software?” Free Software Foundation.
- [9] COBOL, Common Business-Oriented Language. Designed 1959 by the CODASYL committee, building on Grace Hopper’s FLOW-MATIC, with the explicit goal of English-like, business-readable syntax. See e.g. <https://github.com/wallstromsimon/techshoulders/blob/HEAD/src/content/works/cobol.md>

The Crumb Doctrine

The epistemic constitution. No receipt, no crumb.

The Crumb Doctrine

M. Drake Stapleton

Version 1.4, 2026-10-02

1 Preamble

We are building machines that will be trusted with real work, and trust cannot rest on a model's confident prose or on a human's careful review, because both fail. A model will invent a formula that was never in the sources. A human will stamp PASS on a test that never ran. The Crumb Doctrine exists to fix the meaning of what the machine knows in something harder than words: verification receipts. A crumb is the smallest unit of knowledge we allow ourselves to stand on, and this document says what a crumb is, what it demands, and what it forbids.

The doctrine is not yet a property the machine enforces. What we can say today is narrower: admission and closure are now fail-closed. Nothing enters the Crumblin without a receipt, and any step the machinery cannot verify reports itself as not run, never as passed.

2 Definitions

Crumb. A verified program plus its verification receipt. Code plus qualifying evidence. A program without a receipt is a claim, not a crumb. A receipt without a program is paperwork, not a crumb. The two travel together or not at all.

Receipt. The evidence that a program does what it claims. It records what was checked, how it was checked, when it was checked, which verifier did the checking, and what the program was checked against. A receipt must be recheckable by an independent party under fully declared prerequisites, without asking us for permission or for our word. If an independent party cannot recheck it, it is not a receipt. A receipt makes its evidence inspectable and recheckable. Its specification, verifier, assumptions, and applicability remain open to challenge.

A receipt records at minimum: the digest of the verifier that ran, the digest of the specification it checked against, the closure of dependencies the result depends on, the execution environment, the architecture assumptions, the randomness and seeds, the resource bounds, the timeout semantics, the nondeterminism policy, the input corpus or a commitment to it, and the exact result. A receipt that omits any of these is incomplete. When a crumb is scored in Turings, the measurement profile fields travel with or alongside the receipt, so the two do not drift.

Provenance. A crumb's history, carried with it permanently: where the crumb came from, what generated it, what it was verified against, and every hand it passed through. Provenance is never stripped, never summarized away, never "cleaned up." A crumb with no history is a rumor.

Crumbline. The append-only lineage of verified crumbs and their outcomes. Empirical claims requiring generalization are verified against appropriately sealed held-out evidence. The guarantee is simple. Models built the laboratory, the generators, the harness, the machinery. Any lineage claimed as independent machine discovery must carry auditable evidence that prohibited generators and learners had no access to the sealed side of the curriculum. That is what makes the records trustworthy: they are firsthand.

3 The Tenets

1. **Claims are admitted through verification, not prose.**

Semantics defines behavior. Specifications identify claims. Receipts record evidence for those claims under explicit assumptions. Policy determines admissibility; authorization remains separate. A type-checking receipt, a fuzzing receipt, a bounded model-checking result, and a formal proof do not establish the same thing.

Rationale: if two receipts look alike, the claims they record are still different; admissibility has to be decided on the claim, not on the shape of the receipt.

2. **No receipt, no crumb.**

An unverified program may be useful, interesting, or promising, but it is not a crumb and it does not enter the library. There is no provisional shelf, no “verified enough,” no trust-me tier.

Rationale: one exception becomes the standard; the line holds only if it never bends.

3. **Every receipt must be recheckable by an independent party.**

If verifying a crumb requires our machines, our keys, or our word, the receipt is theater. An independent party with the crumb, its receipt, and the prerequisites the receipt declares must be able to rerun the check and reproduce the declared result within its declared equivalence or tolerance conditions. Deterministic receipts may declare exact equality as their tolerance.

Rationale: trust that depends on us is rent; trust that survives without us is property.

4. **In experiments claiming independent rediscovery, the learner is never told why a crumb matters.**

AIEN sees observations only: inputs and outputs, no titles, no labels, no explanations, no hint that one crumb matters more than another. The task boundary comes from the runtime, never from descriptive text. This rule governs the rediscovery claim; it is not a restriction on all learning. What counts as evidence of rediscovery is stated in the Blind Curriculum Protocol.

Rationale: if the machine claims to have rediscovered something true on its own, we must be able to tell rediscovery from quiet coaching, and that is only possible if we never coached it in that experiment.

5. **Provenance is permanent.**

Every crumb carries its full history forever: generator, seed, verifier, date, and what it was checked against. Nothing is stripped for convenience.

Rationale: a result you cannot trace is a result you cannot defend; when something breaks, the history is how you find out why.

6. **The Crumbline is append-only and unbroken.**

Crumbs are added; they are never edited in place, never silently replaced, never deleted to hide

a failure. Failures are recorded as failures, alongside the passes.

Rationale: a lineage with gaps is a story, not evidence; the misses teach as much as the hits, but only if they stay on the record.

7. **Weights suggest; programs explain.**

A model may propose anything. Proposals become knowledge only when they survive verification and enter the Crumblin as crumbs. Until then they are suggestions with no standing.

Rationale: this is the division of labor the whole project rests on; blurring it is how slop gets in.

8. **Verification is independent of generation.**

The thing that checks a crumb must not be the thing that made it. Complete assumption independence is impossible: generator and verifier will share instruction-set semantics, arithmetic definitions, parsers, perhaps even the semantic representation itself. We require enumerated independence instead: separate implementation where it matters, separate failure modes, and every shared assumption stated explicitly in the receipt as a limitation of what the receipt can claim.

Rationale: Omega is the system; verification is performed by an independent verifier, never by the generator. Independence is what makes a receipt worth anything, and honesty about where independence ends is what makes independence real.

4 The Composition Principle

Large programs are built only from verified crumbs. A crumb may be composed with other crumbs, and the composition itself becomes a crumb only when the composed program is verified and receipted in turn. Nothing unverified enters the chain at any level: not the foundation, not the middle, not the top.

This is the long-term shape of the work. The Omega language's module and import system is the crumb store: to build on something in Omega is to build on a verified crumb, the way Rust code is built on crates. The difference is that a crate is trusted by reputation while a crumb is trusted by receipt. In that language, the doctrine stops being a document and becomes the grammar. The easy way to build and the right way to build are the same act.

Until that language exists, the principle is enforced the hard way: by gates. See the closing.

5 The Learner Principle

The autonomous improvement process may acquire durable machine-generated knowledge only through the Crumblin. Where the claim is independent rediscovery, the learner is shown crumbs as observations only and is never shown the sealed side: no generator identities, no difficulty scores, no provenance, no labels, no explanations of what any crumb is for. This blindness is mandatory specifically for independent-rediscovery claims; it is not a restriction on all learning. The segregation is structural, not cosmetic. It is what makes every later claim of independent discovery checkable: either the machine found it on its own from observations, or we can point to the exact place we told it.

Observations alone are not enough to guarantee blindness. Information can leak through ordering,

repetition frequency, curriculum selection, input shape, timing, dependency structure, naming, rarity, or which crumbs are exposed at all. We therefore require a Blind Curriculum Protocol: a specification that enumerates exactly which channels are observable to the learner and tests each channel for leakage. Until that protocol exists and is enforced, no claim of independent rediscovery counts as evidence.

Where independent rediscovery is claimed, the learner’s ignorance of why is not a limitation to be fixed later. It is the mechanism that keeps the whole lineage honest.

6 What the Doctrine Demands of Tooling

A doctrine that lives in a document is a wish. A doctrine that lives in the merge gate is a law. The Crumb Doctrine therefore demands the following of every tool in the pipeline, and a tool that cannot meet them does not get to touch crumbs:

1. **Gates, not guidelines.** Storing the crumb must be a required check before a change is accepted, not a recommendation in a style guide. If the check is optional, it will be skipped under pressure, and pressure is constant.
2. **Automatic minting.** When a program passes verification, its crumb is minted by the machinery itself, through the existing promotion and provenance modules. No manual step, no extra effort, no excuse. Anything that costs the author effort will be avoided.
3. **Independent verification.** The verifier runs separately from the generator, with its own assumptions stated. Shared assumptions are documented as a limitation of the receipt, not hidden.
4. **Visible counts.** The number of crumbs minted, verified, and failed is reported where people can see it without digging. What is measured is done; what is invisible is neglected.
5. **Fail closed.** Any step of the crumb pipeline that cannot be verified reports itself as not run, never as passed. A missing receipt is a missing crumb, and the system says so plainly.

The standard applies to our own tooling first. The day we found our own scripts stamping PASS without running the checks, the doctrine did not bend for us. It does not bend for anyone.

7 The Lifecycle

We give the toolchain four stages, and each stage establishes something different.

`omega check` establishes well-formedness and declared effects: the program is coherent and states the effects it claims. A workspace `omega build` produces an unsealed realization for testing: a thing that runs, carrying no admission into the Crumbline. `omega verify` produces claim-specific evidence against the specification the claim names: the receipt records what was checked and how. `omega seal` admits the artifact and its qualified dependency closure: from that point the artifact is addressable by its identifiers and its closure is on the record. Sealing does not settle the

cost of verification once and for all. Evidence can be reused while its assumptions, dependencies, environment, and policy remain applicable; rechecking and composition can incur additional costs.

Two rules hold across all four stages. A receipt attests to a claim, not to a program in the abstract: the same semantic object paired with a different specification is a different claim, and needs its own evidence. And execution receipts bind the actual tested realization and its environment, not merely the source semantic identity. What was run, where it was run, and under which conditions it ran belong to the receipt; the source identity alone does not say what executed.

Status: v1.4. This document states principles. The mechanisms are separate works (the Crumblin curriculum spec, the crumbs crate, the Omega language design): receipt-and-closure checking is implemented and exercised in CI; repository-wide closure enforcement and required-before-merge gating are not yet active; full language semantics are not. They stand or fall on their own evidence.

Revision note (v1.4): verification is no longer described as paid once at seal time. Evidence may be reused while its assumptions, dependencies, environment, and policy remain applicable; rechecking and composition can incur additional costs. Tenet 1 rationale no longer repeats the body. Enforcement state unchanged.

The Omega Language

The proposed enforcement language. Syntax is disposable; meaning is permanent.

Omega: A Programming Language Built for Verified Programs

M. Drake Stapleton

October 2026

Abstract

We propose Omega, a planned systems programming language that combines the memory safety of Rust with the simplicity of Go, and adds two properties neither language has: it is semantics-first, and verification is a first-class language feature. In Omega, the primary object is meaning, not text. Syntax is a disposable skin over a canonical, content-addressed semantic identity: two surface texts that elaborate into the identical normalized semantic graph share one semantic identity (SEMANTIC_ID). The module system is the crumb store. A *crumb* is a verified program together with its verification receipt, and programs may only be built from verified crumbs. The doctrine that the verified program is the universal artifact of meaning is therefore not a policy document. It is the grammar of the language. In this document we state the motivation, the design principles, the intended trade-offs, and the open questions, and then we answer the objections we expect. It is a design proposal, not a specification. Nothing here is final until we have built, measured, and ratified it.

1 Motivation

We propose a programming language built for verified programs.

Most programming languages were designed for a world in which humans write all the code. That world is ending. In our work, the large majority of code is already written by AI agents, reviewed by AI agents, and tested by AI agents. Languages optimized for human comfort are optimizing for the wrong writer.

Rust is the closest existing answer to safe systems programming, but its compiler is a harsh teacher: the learning curve is steep, builds are slow, and the strictness that makes it safe also makes it unfriendly. Go chose the opposite trade: simple, fast to build, friendly. It keeps memory safety apart from data races, and pays for simplicity with a garbage collector and less fine-grained control over layout and timing.

Omega starts from a different question, and we ask it directly: what would a language look like if we designed it for verifiability first, with the knowledge that its primary authors will be machines? A strict compiler is painful for a human. For an agent, a strict compiler is a tireless reviewer that never sleeps, never gets tired, and catches the bug before it ships. The pain of strictness is almost entirely human pain. We design Omega to spend that budget where it buys the most: correctness you can check, not comfort you can feel.

The name is deliberate. In our architecture, Omega is the umbrella system that defines, synthesizes, and coordinates verification. We give the language the same name because the language is one

interface of that system. Verification itself is performed by an independent verifier, never by the generator: the language does not grade its own work.

2 Semantics first

In every mainstream language, the primary object is text. The compiler reads characters, and meaning is something the text is hoped to carry. Omega inverts this: the primary object is meaning, and text is a disposable skin.

This is not a metaphor. It is the founding position of the Omega engine, established in Milestone 4 (OMEGA_SEMANTICS): Omega defines what computation means without committing that meaning to any particular machine representation. The governing axiom is that meaning is permanent and representation is disposable. The precise claim is this: SEMANTIC_ID is the hash of the canonical semantic IR produced by elaboration, and the hash input includes the semantic schema version and the normalization rules version: $\text{SEMANTIC_ID} = \text{H}(\text{schema_version} \parallel \text{normalization_version} \parallel \text{canonical_graph_bytes})$. A stable byte-level graph cannot silently acquire a new interpretation as the language evolves, because any change to the schema or the normalization rules changes the identity. Two surface texts that elaborate into the identical normalized semantic graph under the same schema and normalization versions share one identity. Programs that elaborate into different graphs may later be proven equivalent, but that proof is a separate artifact: source elaborates to canonical semantic IR to SEMANTIC_ID, and separately, SEMANTIC_ID_A is connected to SEMANTIC_ID_B by an equivalence proof. We do not claim that arbitrary programs can be decided equivalent; general behavioral equivalence is not computable, and Omega does not pretend otherwise. What is permanent is the semantic object; what is disposable is its realization. An optimizer does not change the semantic object. It realizes the same object into a different target representation, and carries a refinement or equivalence receipt connecting the realization to the source semantics.

For the language, we draw four consequences that shape everything below.

First, syntax is not sacred. The concrete text a programmer writes is one of many possible skins over a meaning. We leave the language free to offer multiple surfaces, or to change its surface over time, because the surface was never the program. What the programmer writes is parsed into meaning; what is stored, checked, imported, and verified is meaning.

Second, equality is semantic, over the canonical graph. Two crumbs whose surface texts elaborate into the identical normalized semantic graph share one SEMANTIC_ID: they are the same semantic object, regardless of how they were written. They are not necessarily the same crumb: a crumb is a claim plus its receipts, and different specifications asserted of the same graph give different CLAIM_IDs and therefore different crumbs. We deduplicate semantic objects; we never collapse distinct claims or distinct evidence. This makes the crumb store deduplicating by construction: the library cannot fill up with ten spellings of one idea. Refactoring or optimization that changes the semantic graph is not identity-preserving; it produces a new semantic object, connected to the old one by an equivalence receipt where one is claimed.

Third, verification checks meaning, not text. A receipt does not say “this text passed the checks.” It says “this meaning was verified against this specification.” That is why the crumb module system works at all: when you import a crumb, you import a meaning plus the evidence for it, not a file. Files can lie about what they contain. Canonical encoding and hashing do not prevent that: hashes

bind encoded content; they do not establish semantic correctness. A hash proves the content is the content that was hashed, nothing more. What hashing gives us is binding: a meaning changed after hashing no longer matches its hash, so silent substitution is caught.

Fourth, the language and the engine are one idea. The Omega engine’s semantic substrate is the ground the language stands on. The compiler does not translate text to machine code in the traditional sense; it resolves meaning and realizes it on the target. The language is the notation; the semantic substrate is the substance.

In short: most languages ask “what does this text say?” Omega asks “what does this mean?” and treats everything else as packaging.

3 Design principles

1. **The verified program is the unit of meaning.** Claims are admitted through verification, not prose. Semantics defines behavior; specifications state properties; receipts support those properties under declared assumptions. The language manipulates meanings, not text; syntax is a disposable skin over the semantic identity. This is the Crumb Doctrine, and we make it structural in Omega, not aspirational.
2. **Design for the writer you have.** We assume the language’s code is mostly written by agents and mostly read by verifiers. We optimize error messages, tooling, and language rules for machine authors first. Human readability is a requirement, not the goal.
3. **Safety is not optional.** We provide no mode in which the safety guarantees are silently dropped. Any weakening of safety must be explicit, visible, and auditable. See Section 8 for the resolved answer this raises.
4. **One idea per mechanism.** Each language feature does one thing. Features that overlap are a design failure, not richness.
5. **Name the sacrifice.** Every language gives something up. Go gave up fine-grained control for simplicity. Rust gave up friendliness for safety. We state Omega’s sacrifice explicitly in Section 5. We do not leave it to be discovered later by its users.

4 The crumb module system

In Rust, the ecosystem is organized around crates: reusable packages that programs are built from. In Omega, the corresponding unit is the *crumb*: a verified program together with its verification receipt. We define the module and import system of Omega as the crumb store. There is no separate package registry and no separate proof archive. To import a crumb is to import its receipt.

Concretely, this means:

- A crumb is the smallest unit of reuse. Functions, modules, and libraries are all crumbs: each carries the evidence that it does what it claims.
- We permit a program to depend only on verified crumbs. An unverified program cannot be imported, linked, or built upon. We have the compiler check this, not policy.

- A crumb carries several distinct identifiers, and we do not collapse them. `SEMANTIC_ID` is the hash of the canonical meaning together with the semantic schema version and the normalization rules version (Section 2). `SPEC_ID` is the hash of the property or specification being asserted. `CLAIM_ID` is the hash of the semantic ID concatenated with the specification ID: $H(\text{SEMANTIC_ID} \parallel \text{SPEC_ID})$. `RECEIPT_ID` is the hash of the verifier, the evidence, the environment, the result, and the `CLAIM_ID` the receipt attests to: the `CLAIM_ID` is included in the receipt’s hashed payload, so the receipt is bound to the exact claim it verifies. `SEAL_ID` is the hash of the claim ID, the ordered receipt identifiers, and the store policy ID: $H(\text{CLAIM_ID} \parallel \text{ordered_receipt_ids} \parallel \text{policy_id})$, an immutable snapshot of the evidence accepted at seal time. A crumb is the claim plus at least one qualifying receipt: the logical subject stays stable while evidence accumulates, which matters when independent laboratories add their own attestations years later. The same claim may legitimately acquire several receipts: checked by two independent verifiers, reverified years later, or qualified on two architectures. Those are new receipts, not new claims. A crumb cannot be silently substituted or modified without invalidating everything built on it, because every identifier in the chain is content-addressed.
- Four rules hold across these identifiers, and we state them together so the pages agree. The same canonical graph under the same schema and normalization versions gives the same `SEMANTIC_ID`. A different specification gives a different `CLAIM_ID`, because `SPEC_ID` is part of the hash. A different verification event gives a different `RECEIPT_ID` only when event-specific information (timestamp, verifier version, input digests, environment) is included in the hashed payload: hashing identical payloads yields identical hashes, and that is the point of the scheme. A different set of accepted evidence or a different policy gives a different `SEAL_ID`. We deduplicate semantic objects; we never collapse distinct claims, distinct receipts, or distinct seals.
- We call the growing lineage of crumbs the *Crumbline*: the unbroken record of what the machine actually built and verified.

Our intent is that the crumb doctrine, which today we must enforce on agents by external checks, becomes unavoidable. The easy path and the correct path are the same path, because the language has no other path.

This works because crumbs are semantic objects, not files. A crumb’s identity begins with its `SEMANTIC_ID`, the content address of its canonical meaning. Importing a crumb therefore imports a meaning plus its evidence, each addressed separately. Two surface texts that elaborate into the identical normalized semantic graph resolve to the same `SEMANTIC_ID`; the store cannot accumulate ten spellings of one meaning. See Section 2.

5 Safety model

We give Omega Rust-grade memory safety: programs cannot corrupt memory, and the classic classes of memory bugs (use after free, double free, data races on shared memory, buffer overruns) are ruled out by the language, not by programmer discipline. We pursue Go-like simplicity in expressing that safety: the ownership rules are designed to need fewer annotations and produce clearer errors than Rust’s borrow checker, with more inference and fewer ways to say the same thing.

We state the intended trade up front, per our design principles: **we sacrifice low-level control and build speed.** There is no raw pointer arithmetic in the safe subset, no manual memory layout control, and no custom allocators. Verification takes time, so builds are slower than Go's. In exchange, the programmer, human or agent, gets memory safety plus simplicity plus proof-carrying code. If a use case needs hand-tuned memory layout or the fastest possible build, Omega is the wrong tool, and we have the language say so plainly rather than grow a back door.

6 Concurrency

Go's most beloved feature is its concurrency model: thousands of tiny, cheap tasks that are simple to write and simple to reason about. Rust's concurrency is powerful but widely considered painful. We adopt the Go-like model as the headline direction: lightweight tasks with simple syntax, combined with Rust-grade safety guarantees.

We state plainly that this section is direction, not architecture. Before the concurrency model can carry verification, the following must be defined: the memory model, the effect model, the ownership and aliasing model, cancellation semantics, task lifetime rules, channel semantics, failure propagation semantics, scheduling guarantees, and the treatment of nondeterminism. The decisive question is not whether the syntax looks like Go. It is what semantic object a concurrent Omega program denotes, because verification is meaningless until that object is defined.

7 Verification as a language feature

In most languages, verification happens outside the language: test suites, external analyzers, code review. We make verification a first-class part of the language, but we do not pretend it is one homogeneous act. Verification ranges from milliseconds of type checking to hours of property testing, model checking, differential execution, fuzzing, or hardware-in-the-loop qualification. One word should not cover all of that. We therefore divide the pipeline into four stages, and we name them so their costs stay visible.

`omega check` establishes well-formedness and declared effects: the program is coherent and states the effects it claims. This is the fast, local work: parsing, elaboration, and type checking.

A workspace `omega build` produces an unsealed realization for testing: a thing that runs, carrying no admission into the Crumblin. Building does not verify; it realizes.

`omega verify` produces claim-specific evidence against the specification the claim names. The verification profile says what is checked, how, and at what cost; a receipt for type checking and a receipt for model checking are different artifacts with different weight.

`omega seal` admits the artifact and its qualified dependency closure, minting the crumb: from that point the artifact is addressable by its identifiers and its closure is on the record. Only a sealed crumb may enter the crumb store or satisfy a verified dependency.

An unsealed object may exist locally during development; we do not make the inner edit-build cycle unusable in the name of purity. But we distinguish the workspace graph from the crumb graph. In the workspace, sealed and unsealed objects may depend on each other locally: an unsealed object may depend on unsealed objects during development. The crumb graph is sealed only. Nothing

unsealed may satisfy a sealed dependency, enter the crumb store, or contribute to a sealed top-level program until the entire dependency closure meets the required verification policy. That is how “no receipt, no crumb” survives contact with real development.

What is verified is the meaning, not the text. Two surface texts that elaborate into the identical normalized semantic graph share one `SEMANTIC_ID`, and one verification of that graph serves both. Where the graphs differ, equivalence must be proven as its own artifact; we do not declare it by definition.

We draw three consequences from this. First, the Crumbline grows automatically as a side effect of sealing verified software; no separate step records what was proven. Second, the cost of verification is paid once, at seal time, and amortized over every program that imports the crumb afterward. Third, trust becomes checkable: any consumer of a crumb can re-examine its receipt without trusting its author.

We leave the verification machinery itself and the exact contents of a receipt to the language specification. In this document we establish only that verification is not optional tooling. It is the grammar.

8 The foreign boundary

We have decided. There is no conventional `unsafe` inside Omega.

The argument for an escape hatch was practical: hardware interfaces, foreign code, and extreme performance work may be inexpressible inside the safe subset. The argument against was doctrinal: a language whose meaning is the verified program cannot have a region where programs are unverified but still composable, or the doctrine is a suggestion rather than a grammar. We resolve this by refusing the premise that the hatch must live inside the language.

Low-level components, device interfaces, allocators, drivers, SIMD layouts, DMA, MMIO, and hand-tuned kernels, live outside Omega, in the languages that can express them. Omega imports them only through sealed foreign crumbs: boundary artifacts that carry an explicit ABI, a behavioral contract, platform constraints, an implementation digest, and qualification evidence. The foreign crumb is sealed by the independent verifier like any other crumb, against the contract it claims. Omega never sees the inside; it sees the contract and the receipt, and that is enough to build upon.

We hold foreign crumbs to an invariant. A foreign crumb must satisfy one of three conditions: (a) it is formally proven to satisfy the relevant memory and ABI contract; (b) it executes behind an isolation boundary that prevents violation of Omega’s memory model; or (c) it explicitly weakens the enclosing crumb’s guarantee, and that weakening is visible in the specification and the receipt. For ordinary foreign components our preference is isolation plus contracts. Anything looser recreates `unsafe` in another file. A behavioral receipt alone cannot preserve memory safety for foreign code running in the same address space: without a proof or a boundary, the receipt describes behavior we cannot enforce, and a crumb built on it makes a guarantee it does not hold.

By “isolation boundary” we mean something concrete: the foreign code cannot read or write Omega-managed memory except through the checked ABI, and it shares no mutable address space with Omega code. The canonical mechanism is a hardened OS process. Omega already executes native candidates this way: the runtime places the code on execute-only pages, forks a child, runs the candidate there, and kills it on timeout (`src/runtime/rx_omega.c`). That is crash and hang

containment, not yet a security sandbox: the current child has no seccomp filter, no namespace or filesystem isolation, no resource limits, and no privilege dropping. The hardened form closes those gaps. The child starts with no ambient file descriptors, no network, a seccomp allowlist, namespace and filesystem isolation where the platform provides it, explicit memory and CPU limits, no-new-privileges, and a killable timeout. We keep the separate-process model rather than WASM because Omega must test the actual native realization it may promote; validating WASM is not validating the AArch64 code that will run. WASM remains a separate execution class for portable tools. The architectural rule is: foreign code is data until a disposable, capability-empty native worker verifies it; only receipt-bound verified code may cross into the resident execution path.

One caution about the mechanism itself. A forked child inherits the readable contents of the parent’s memory; closing descriptors and adding syscall restrictions does not erase that information. Worker initialization must therefore start from a clean, minimal state: the child is given only what the task requires, and the rest of the parent’s address space is cleared or dropped before the foreign code runs.

A promotion rule follows from the invariant. Foreign code qualified through isolation remains isolated in production: if the receipt that admits a component rests on an isolation boundary, the component stays behind that boundary in every deployment. Resident-address-space execution requires the relevant memory/ABI proof, or an explicitly propagated weakening of the enclosing guarantee under condition (c) above. We do not let a component ride an isolation receipt into the resident address space; that would be **unsafe** wearing a sealed label.

A weakening under (c) is not a note in prose. It is a requirement on CLAIM_ID and spec composition that the weakened guarantee is part of the specification itself: SPEC_ID is computed over the weakened specification, so every dependent CLAIM_ID is derived from the weakened claim, and the admission check rejects any seal whose dependency closure treats the component as unweakened. The weakening propagates mechanically through the identifier chain and the admission checks, not by convention.

This preserves the doctrine better than embedding an unverified language inside the verified one. There is no region of Omega where verification stops applying, because the unverified work never enters Omega at all. It stands at the boundary, labeled, receipted, and accountable.

9 Bootstrapping path

The foundation crumbs, the verified core the language is built from, must be verified the hard way first: written carefully, checked thoroughly, with no existing Crumblines to stand on. This is slow. There is no way to skip it, because a crumb store is only as trustworthy as its first entries.

The expected shape is therefore one we accept: a slow, careful beginning, then compounding returns. Each verified foundation crumb makes the next layer cheaper to verify, because new crumbs are built from old verified ones. The library grows slowly at first and then quickly, and every entry pays for itself across all future work. We accept patience at the start as part of the design.

10 Objections

We have stated our proposal. We now consider the objections we expect to be raised against it, and answer each in turn.

Objection 1: This is only Rust with nicer syntax.

We answer that this description would be a smaller idea, and it misses the proposal. A nicer Rust would still manipulate text and hope meaning follows. Our distinguishing claim is different in kind: verification as a four-stage pipeline (**check**, **build**, **verify**, **seal**), the crumb as the unit of reuse, and meaning rather than text as the object the language manipulates. We are not improving Rust's answers. We are asking a different question.

Objection 2: Or Go with a borrow checker.

The same answer applies. Go's simplicity is worth borrowing, and we borrow it openly. But a Go with stricter checking would still be a language about text, with verification as an afterthought. We make verification and sealing structural requirements of reusable composition, and we make the module system the store of verified meanings. The resemblance to Go is in the feel of writing concurrent code, not in the nature of the language.

Objection 3: Most tasks do not need any of this.

We agree. Omega is not a general-purpose scripting language. Where a task does not need safety guarantees or verification, we recommend the simpler tool. A language that insists on proving everything is the wrong instrument for work that needs proving nothing. We do not propose Omega for all programming. We propose it for the programming that has to be right.

Objection 4: A language about text would be simpler and just as good.

It would be simpler to build, and it would not be as good. Text is packaging: two surface texts that elaborate into the identical normalized semantic graph share one identity, and a language that cannot see this will verify the same meaning twice, store ten spellings of one idea, and mistake reformatting for change. We do not claim to decide equivalence of arbitrary programs; where the semantic graphs differ, equivalence is a separate proven artifact. We accept the cost of canonicalizing meaning because the alternative is a system that confuses the wrapper for the thing wrapped.

Objection 5: This is academic theorizing, not engineering.

We measure Omega's success in systems shipped, not papers published. It is a working language for building real systems, or it is nothing. The proposal is written down precisely so that it can be tested against reality, and we will discard whatever reality rejects.

Objection 6: Why should anyone believe these claims before the language exists?

They should not. This document is a design proposal dated October 2026, and every claim in it is provisional until we have built, measured, and ratified it. We propose; we do not assert. The objections above are welcome, and we expect more. A proposal that cannot survive its critics does not deserve to be built.

Objection 7: This has all been done before.

Much of it has, and we say so plainly. Unison addressed code by the hash of its content. Dafny, F*, SPARK, and Lean made verification a first-class concern of the language rather than an afterthought. Proof-carrying code, introduced by Necula in 1997, is the direct ancestor of our maxim that the receipt travels with the code. We do not claim to be the first to put verification inside a programming language.

What we propose is the integration of four things these systems do not combine: semantic identity as the unit of reuse, mandatory evidence-bearing dependencies, an admission policy that decides what may be built upon, and a provenance record of everything the machine has built and verified. Each of those has precedent. Their combination, made structural rather than optional, is the proposal.

The Atemporal AIEN Postulate

The eighth paper. Causal semantics without clocks: the Crumbline is not a timeline, it is a causal record.

The Atemporal AIEN Postulate Causal Semantics Without Clocks

M. Drake Stapleton

Version 1.1, 2026-10-02

1 The proposal

I propose that an AIEN computation is defined by what causally depends on what, not by when those events occur. Physical time characterizes a realization of the computation; it does not define the computation itself.

I do not mean by this that the machine runs without time. Hardware takes time. The GPU has clocks. Electrons move. Experiments consume joules and seconds. I mean that those quantities belong to the realization boundary, not to the ontology of Omega. Omega has order without clocks. FORGE has clocks without defining meaning.

Version 0.1 of this paper stated the proposal and left its formalism incomplete. Four independent reviews found the holes: the Crumblin extension definition did not enforce what it claimed, schedule invariance was stated without its hypotheses, and the novelty claim was argued against the wrong literature. Version 1.0 repairs the formalism, states the hypotheses, and narrows the claims. Version 1.1 folds in a fifth review (GPT-6 Astra), which confirmed the v1.0 repairs hold and found the remaining gaps: the schedule theorem's transfer from abstract orderings to physical executions, the fidelity (not just preregistration) of the realization projection, occurrence identity in canonical serialization, the typing of semantic identity, and an honest statement of what the experiment can and cannot establish. What remains open is stated as open.

2 Glossary

Readers arriving from outside the AIEN project need the following terms, used without definition in v0.1.

AIEN. The cognitive driver of the architecture: the component that thinks, proposes intents and hypotheses, and reasons across possibility spaces.

Omega. The semantic calculus: the component that defines meaning, graphs, constraints, and formal transformation programs. This paper is about Omega's semantics.

FORGE. The physical realizer: the component that lowers Omega programs into physical machine operations and executes them on hardware.

AEGIS. The verifier: checks constraints, bounds, types, and post-execution state contracts. AEGIS verifies; it does not decide what the machine may attempt.

Crumb. A verified program plus its verification receipt. A program without a receipt is a claim, not a crumb.

Receipt. The evidence that a program does what it claims: what was checked, how, when, by which verifier, against what specification.

Crumblines. The lineage of admitted crumbs and receipts, with the formal structure defined in Section 5.

The Turing. A protocol-bound measure of net held-out explanatory compression. In the formal Turing paper, $T(M; B, D, G, P)$ compares a candidate model M against a baseline B on held-out evidence D under shared background G and measurement profile P .

3 The five postulates, revised

Postulate 1 (Atemporal semantics). *Omega semantic identity contains no primitive wall-clock coordinate. Clock measurements belong to physical realizations and receipts, not to meaning. I describe Omega as atemporal, relational, and event-driven. I do not describe it as “analog”: analog hardware may eventually be one realization, but the semantics do not need it.*

Postulate 2 (Causal order). *The primitive dependence relation is a finite directed acyclic graph $\rightarrow$ on semantic events. I write $a \prec b$ for reachability, $a \rightarrow^+ b$, and I call $\prec$ the causal order. It is irreflexive and transitive: a strict partial order. It is not necessarily a total order. Two events may satisfy neither $a \prec b$ nor $b \prec a$; they are then causally independent, or concurrent, and there is no need to ask which happened “first.”*

I note a departure from convention: in the distributed-systems literature, $a \prec b$ usually reads “ a happened before b .” I use it to mean “ b depends upon a ,” which is closer to dataflow and causal-consistency usage than to Lamport’s happened-before (which includes program order between independent operations on one process). The mathematics is the same partial order; the reading is dependence, not precedence.

Postulate 3 (Immutable causal past). *Once an object is sealed, its causal ancestry cannot be altered. New objects may cite existing objects. Existing objects may not acquire newly invented causes. Section 5 gives this a definition that actually enforces it; v0.1’s definition did not, as the reviews showed.*

Postulate 4 (Causal-cone admissibility). *A claim may depend only on information reachable through its declared causal past. As a slogan this is either tautological (if “depends” means “declares”) or undecidable in general (if it means actual information flow). I therefore state the enforceable version: causal-cone admissibility is a noninterference property in the sense of Goguen and Meseguer, enforced by the type system and by sandboxing with sealed inputs. An event’s observable behavior may not vary with any state outside its declared causal past. What the postulate demands of the engineering is that every channel, clock read, random draw, and environment interaction be an explicit declared input or be absent.*

Postulate 5 (Schedule invariance). *If two physical executions realize the same semantic graph, preserve its dependency relation, and satisfy the hypotheses of Theorem 1 (deterministic opera-*

tions, causal completeness, commuting concurrent effects), then they are realizations of the same Omega computation, regardless of clock ordering, duration, concurrency, CPU/GPU placement, or scheduling.

4 The semantic object is a labeled causal structure

In v0.1 I wrote that “order is all the machine’s meaning consists of.” That was too strong, and it contradicted my own formalism two sections later. Two computations with identical graph topology $A, B \rightarrow C$, one defining $C = A + B$ and the other $C = A/B$, are plainly different computations. Order is not all of meaning. The defensible claim is narrower: timing is not part of meaning, and causal order is the only temporal structure that is.

Definition 1 (Omega program). *An Omega program is a tuple*

$$S = (V, \rightarrow, \lambda, C, E)$$

where V is a finite set of operations, $\rightarrow$ is a directed acyclic graph of direct causal dependence, λ is a labeling that commits each $v \in V$ to its operation: model or code identity, parameters, types, and input/output port roles. C is the set of constraints the realization must satisfy (resource bounds, precision requirements, sandboxing policy). E is the set of declared effects (state transformations, I/O channels, observable outputs). The causal order is $\prec := \rightarrow^+$. There is deliberately no clock.

The labeling λ answers the reviewers’ “lottery” objection: the bare poset is shared by addition, concatenation, and a lottery, but the labeled structure is not. Edge roles matter too: “left operand” and “right operand” are part of λ , so the $A \leftrightarrow B$ automorphism that worried the reviewers cannot silently swap inputs.

I separate direct dependence $\rightarrow$ from its transitive closure $\prec$ because v0.1 conflated them. “Depends upon” naturally names the direct relation, but Postulate 2 needs transitivity, and the causal parents $\text{Pa}(v)$ used in Section 9 must be the direct parents, which are unrecoverable from the transitive relation without transitive reduction. The DAG is primitive; the order is derived.

5 The Crumblin is not a timeline

The Crumb Doctrine describes the Crumblin as an append-only lineage. Under the atemporal postulate, “append-only” must be restated, because chronological order is no longer the thing being preserved:

“Append-only” becomes causal immutability, not chronological order.

Definition 2 (Crumblin state). *The Crumblin state is $\mathcal{C} = (V, \rightarrow)$ where V is the set of admitted crumbs and receipts and $\rightarrow$ is the direct evidential-dependence DAG. The causal order is $\prec := \rightarrow^+$.*

Definition 3 (Structurally admissible extension). *An admissible extension of $\mathcal{C} = (V, \rightarrow)$ by a new receipt $r \notin V$ is*

$$\mathcal{C}' = (V \cup \{r\}, \rightarrow')$$

where, writing $\text{Dep}(r) \subseteq V$ for the declared dependencies of r ,

$$\rightarrow' = \rightarrow \cup \{(d, r) : d \in \text{Dep}(r)\}, \quad \text{and} \quad \prec' := \rightarrow'^+.$$

Two things in this definition repair v0.1. First, $\prec'$ is defined as a transitive closure over a graph that remains acyclic — the old graph is acyclic and the fresh vertex r is a sink — so $\prec'$ is a strict partial order; v0.1 merely required the restriction $\prec'|_V = \prec$ and left transitivity unproven. Second, all newly added edges terminate at the fresh vertex r , and no edge originates at r : r is maximal with respect to V , meaning $\neg(r \prec' v)$ for every $v \in V$. A new receipt can never become a cause of an already-sealed crumb.

The v0.1 definition admitted the following counterexample, found independently by two reviewers and verified programmatically. Start with $V = \{a, b\}$, $a \prec b$. Add r with $\text{Dep}(r) = \{a\}$, and define $a \prec' r$, $r \prec' b$, $a \prec' b$. All three of v0.1's conditions held: the dependencies were in V , each dependency preceded r , and the restriction to V was unchanged. Yet $\text{Past}(b)$ grew from $\{a\}$ to $\{a, r\}$: the new object was inserted into the causal past of a sealed object, violating Postulate 3. Under the repaired definition this is impossible, because $r \prec' b$ cannot be formed: no edge of $\rightarrow'$ leaves r .

Lemma 1 (Causal-past immutability). *For every admissible extension $\mathcal{C}'$ of $\mathcal{C}$ and every $v \in V$,*

$$\text{Past}_{\mathcal{C}'}(v) = \text{Past}_{\mathcal{C}}(v).$$

Proof sketch. Any path in $\rightarrow'$ ending at $v \in V$ cannot pass through r , since no edge leaves r . Hence every such path lies entirely in $\rightarrow$, and the pasts coincide. $\square$

Structural admissibility is not verified admission, and I do not conflate them. The rule above accepts any fresh r with $\text{Dep}(r) \subseteq V$, including a receipt whose claims are false: a purported receipt asserting that `return 0` satisfies “result = 1” passes structurally with empty dependencies. Admission additionally requires an independent judgment

$$\text{Check}(p, \varphi, \rho, \text{policy}) = \text{accept},$$

relating the program p , the claimed proposition φ , the inputs ρ , and the verifier policy. That acceptance implies the claimed semantic properties — including the noninterference of Postulate 4 over the specified observables — is an explicit architectural assumption and open obligation of AEGIS, not a theorem of this paper. The observables covered are computed values, declared effects, and output traces; timing is excluded; termination behavior must be declared. Finite testing cannot establish the universally quantified noninterference property for unrestricted programs, and I do not claim it does.

The append-only rule is therefore:

The causal past of an admitted crumb is immutable.

Two receipts A and B created independently satisfy neither $A \prec B$ nor $B \prec A$. The database may physically write A at 12:01 and B at 12:02; another replica may receive B before A . Neither ordering means anything to Omega. Later C may depend on both. Then A and B belong to C 's causal past but remain unordered relative to one another.

This is the “sequential growth” step of causal-set dynamics (Rideout and Sorkin): each new element is added maximal with respect to the existing structure. I cite it because the causal-set literature is the closest mathematical relative of this construction, now as a precise technical borrowing rather than the loose analogy of v0.1.

Retrospective causal inference. A strong objection, raised in review: science routinely revises causal understanding after the fact, discovering that A caused B though both were recorded as concurrent. Does immutability forbid this? It forbids rewriting the old graph, and it should. The revision is itself a new crumb: a receipt asserting the newly discovered relation, citing both A and B as evidence, admitted maximally like any other. A paper does not rewrite the lab notebook; it cites it. The formalism therefore distinguishes the record of what was admitted when from later claims about what those admissions mean, and both are preserved.

I keep two relations distinct here, because v1.0 blurred them: $A <_{\text{domain}} B$, an asserted fact about the subject matter, versus $A \prec_{\text{ledger}} B$, a recorded evidential-dependence edge. A new receipt can assert the first; it cannot retroactively create the second. Asserted domain propositions live in the claim language φ of the admission judgment, not in the ledger order. No change to the extension lemma is needed, because the lemma was never about domain facts.

Replicas. I acknowledge what the formalism relocates rather than eliminates: each replica’s admission sequence is a local total order, a linear extension of the causal order. A global “Crumblin state” across replicas needs consistent cuts in the sense of Chandy and Lamport, and the merge of replica states is future work, not claimed here.

6 Schedule invariance, with hypotheses

Theorem 1 (Schedule invariance). *Let $(V, \rightarrow)$ be a finite DAG with causal order $\prec := \rightarrow^+$, labels λ , and a fixed external-input valuation ρ . Suppose:*

1. **Determinism.** *Each event v computes a deterministic, terminating transition on execution configurations from its declared causal inputs: its direct predecessors’ outputs, its labeled parameters $\lambda(v)$, and the restriction of ρ to its declared external inputs — and nothing else.*
2. **Causal completeness.** *All randomness, I/O, clock reads, mutable state, and environment interactions are explicit declared inputs, i.e. members of $\text{Pa}(v)$ for the events they influence, or absent entirely.*
3. **Diamond condition.** *For causally incomparable enabled events a, b with semantic state transformations T_a, T_b on a configuration space containing all semantic observations, both execution orders are defined and $T_a \circ T_b = T_b \circ T_a$.*
4. **Result map.** *A specified map O from completed configurations to semantic results; two computations are “the same” when their O -images are equal.*

Then every dependency-respecting serial execution of the graph yields the same semantic result O .

Proof sketch. Any two linear extensions of a finite partial order can be transformed into one another by a sequence of swaps of adjacent incomparable elements. By hypothesis 3, each such swap leaves the composed transformation unchanged, and by hypotheses 1, 2, and 4 no event’s contribution depends on anything but its declared causal inputs. $\square$

Three things about this statement are new in v1.1, and each closes a real gap. First, the configuration space contains *all* semantic observations, including declared output traces: if incomparable A and B emit fixed messages **a** and **b** to a declared channel, their store-only transformations are both

the identity and commute, yet the observable traces $\mathbf{ab}$ and $\mathbf{ba}$ differ. Commutation must hold on the full observable state, not on a store-only projection, or the semantics must explicitly identify traces modulo independence. Second, the theorem is over dependency-respecting *serial* executions: a topological ordering is a serial ordering of atomic events, not an arbitrary parallel execution, so non-atomic read-modify-write implementations that lose updates are outside the theorem’s scope, not counterexamples to it. Third, semantic equality here is exact. A tolerance predicate $|x - y| \leq \varepsilon$ is not transitive and is not preserved by downstream branching, so it cannot ground the swap argument; floating-point acceptance tolerances belong to receipt acceptance, not to semantic equality. Approximate correctness with compositional error bounds is future work.

The theorem does not by itself cover physical executions. That transfer is a separate, explicit architectural obligation:

Realization. Every admitted physical execution must refine a dependency-respecting serial execution of the abstract transitions, preserving the observations in the configuration space. This refinement is an obligation of FORGE and the verifier, not a theorem of this paper. Errors, divergence, and resource exhaustion are inadmissible realizations unless explicitly modeled as semantic results. A failed run is evidence about the implementation or its enforcement, not a counterexample to the theorem, until the theorem’s premises and the refinement have been established for that run. I do not claim decidability merely because the hypotheses are written down.

The hypotheses are load-bearing, and v0.1’s omission of them was the second major hole. A concrete failure without hypothesis 2: give A and B an undeclared shared counter, each reading and incrementing it. The two topological schedules of $A \rightarrow C \leftarrow B \rightarrow D$ then produce $(A, B) = (0, 1)$ versus $(1, 0)$: same DAG, different result. Postulate 4 exists precisely to declare that counter inadmissible; causal completeness is therefore an essential hypothesis of the theorem, not an implementation detail. A concrete failure without hypothesis 1: GPU floating-point reductions executed in “overlapping waves” are not bitwise reproducible across schedules, so bitwise schedule invariance cannot be claimed for floating-point arithmetic without a determinism discipline stronger than the postulate provides.

The compiler theorem of v0.1 is retained in its careful bidirectional form, with the realization obligation made explicit:

same semantic causal graph $\not\Rightarrow$ same physical execution

every dependency-respecting serial execution, under the hypotheses above $\Rightarrow$ same Omega semantic result,

admitted physical executions must refine such a serial execution.

Work and span. Two quantities belong naturally to Omega rather than to FORGE. For a program with unit-cost nodes, the *work* W is the total number of operations and the *span* (causal depth) S is the length of the longest causal chain. For $A, B \rightarrow C \rightarrow D$: $W = 4$, $S = 3$. Neither depends on wall-clock time or on which valid schedule was chosen, yet they say something real about realizability: one processor needs at least four unit operations, and even unbounded parallelism cannot complete the computation in fewer than three causal stages. Omega owns W and S ; FORGE owns seconds, joules, utilization, and scheduling efficiency. This is the rigorous interface between atemporal semantics and temporal realization.

7 The compiler, in both directions

Lowering takes Omega meaning to a clocked physical realization. Lifting, or verification, takes a physical execution trace back up to Omega evidence plus a causal graph. The compiler may produce any physical schedule consistent with $\rightarrow$, and the theorem above says when those schedules are semantically identical.

The semantic/realization boundary must be a defined projection, not a gesture, and its preregistration must not be confused with its fidelity. I define it as follows. Let R be a physical execution record: event-instance identifiers, observed input/output bindings, committed effects, provenance and validation evidence, plus device identifiers, timestamps, durations, energy measurements, memory addresses, and scheduling decisions. Let π be the realization projection fixed before the experiment: $\pi(R)$ retains the causal graph, the labels, and the computed values, discarding exactly the physical coordinates (when, where, how long, how much energy). Preregistration prevents post-hoc changes to the projection; it does not establish fidelity. A fixed projection can be arbitrarily unfaithful — one that always returns the reference graph would let an implementation that hardcodes $C = 14$ pass. Fidelity requires separate justification, as explicit soundness obligations: $\pi(R).values = observedValues(R)$, and every projected dependency edge must be justified by the execution evidence (observed bindings, not the specification text). The recovered graph is claimed as an *actual* data-dependence graph evidenced by the record; compiler transformations such as constant folding belong to the declared lowering, not to silent projection, and speculative execution ahead of inputs is inadmissible unless the lowering declares it.

Semantic identity is then defined canonically, over event occurrences rather than operation types. Two occurrences of the same labeled operation are distinct vertices: with x_1, x_2 labeled X and y_1, y_2 labeled Y , the wirings $x_1 \rightarrow y_1, x_2 \rightarrow y_2$ and $x_1 \rightarrow y_1, x_1 \rightarrow y_2$ are non-isomorphic graphs that a label-multiset serialization cannot distinguish. Canonicalization is therefore over the complete occurrence graph: the lexicographically least encoding over all vertex numberings, each edge encoded by numbered endpoints and port roles, λ included in full. An implementation may use any canonical-labeling algorithm computing the same canonical form. Occurrence identifiers, where used, must be immutable and schedule-independent; equal-content occurrences, especially effectful ones, are never silently deduplicated. Concurrent A, B serialize identically regardless of whether a schedule ran A first or B first, because the serialization never consults a schedule.

Identity is typed, because v1.0’s single SEMANTIC_ID was not a well-typed function: its declared domain was $S = (V, \rightarrow, \lambda, C, E)$ but its preimage omitted C, E and added “claimed values,” which are not components of S . I define instead

$$\begin{aligned} \text{PROGRAM_ID}(S) &= H(\text{canon}(V, \rightarrow, \lambda, C_{\text{sem}}, E)), \\ \text{RESULT_ID}(S, \rho, y) &= H(\text{PROGRAM_ID}(S), \rho, y), \\ \text{CLAIM_ID}(S, \rho, y, \varphi) &= H(\text{RESULT_ID}(S, \rho, y), \varphi), \end{aligned}$$

where C_{sem} are the constraints that belong to meaning (precision requirements, designated observable outputs) as opposed to realization contracts (device placement, time budgets), and the split is stated per program rather than assumed. Program identity, execution-result identity, and claim identity are three different functions; hash equality is evidence of preimage equality under a collision-resistance assumption, not identity by definition.

Labels obey a typed semantic-label schema containing no references to physical receipt identity: semantic certificates are separated from timestamped audit envelopes, so two runs differing only in

when verification occurred do not differ semantically. I distinguish physical execution coordinates, excluded from semantic identity, from timestamp-valued *program data*, which is allowed as an explicit semantic input: a program computing $y = [t > t_0]$ takes the clock reading t as declared data, and schedule invariance holds conditional on the same external-input valuation ρ . A declared source whose value changes with scheduling is not made schedule-invariant merely by being declared.

8 The experiment, strengthened

The v0.1 experiment had two weaknesses the reviewers identified: equal semantic IDs across schedules are near-tautological if the ID is a canonical hash of the graph (it tests the implementation’s canonicalization, not the postulate), and the design lacked negative controls. I strengthen it.

Take one Omega causal graph: $A \rightarrow C, B \rightarrow C, C \rightarrow D$, with deterministic labeled operations, for example $A = 3 + 1, B = 2(5), C = A + B, D = C^2$, expected $A = 4, B = 10, C = 14, D = 196$.

Positive arm. Execute every topological ordering for small DAGs (here exactly two: A, B, C, D and B, A, C, D); for larger DAGs, execute randomized orderings and device placements, including GPU execution with arbitrary delays and reversed physical completion order of independent operations. Define the semantic output independently of the implementation under test: an oracle computes the expected values from the labeled graph by a separate code path. Vary the source inputs across runs (A, B drawn fresh each run) so that an implementation hardcoding $C = 14$ cannot pass. Require: (i) every schedule’s computed values equal the oracle’s; (ii) the label-preserving equality of causal graphs holds across schedules, where equality means identical occurrence structure with identical content-addressed vertex labels and identical edge roles (the $A \leftrightarrow B$ automorphism must not pass: feeding A ’s output into B ’s slot is a different computation and must fail); (iii) the physical receipts report their measurements accurately. I do not require receipts to differ across schedules: identical recorded durations or placements are honest outcomes of measurement resolution, and demanding predetermined differences would reject correct executions.

Negative controls. Run variants, each classified by the obligation it violates: a hidden clock read and undeclared mutable shared state (causal completeness); unseeded randomness (determinism); noncommuting concurrent declared effects, e.g. two events emitting different fixed messages to the same write-only channel (diamond condition — note this is a distinct failure from hidden inputs, and must be rejected as noncommutation, not mislabeled causal incompleteness); and a run with correct values but falsified input provenance or port bindings (projection soundness). The verifier must reject each for its specific stated reason rather than normalize the differences away. A schedule-invariance claim that passes only because the verifier silently discards discrepancies is worthless; the negative controls exist to catch exactly that.

A divergent result is a counterexample to the abstract claim only after the theorem’s premises and the realization refinement have been established for that run; otherwise it is evidence of an implementation or enforcement failure (a device kernel bug, an incorrect lowering, a missing environmental input). Passing runs are bounded conformance evidence for the tested graphs and schedules, not universal proof of the architectural claim. If the positive arm fails under established premises, the postulate fails in its strongest form, and I will record that.

9 The Turing without time, carefully

The v0.1 section on description length needs two corrections.

First, the causal factorization. The reachability order $\prec$ does not determine direct parents: transitive reduction recovers the cover relation, not the original edges, so $a \rightarrow b \rightarrow c$ and $a \rightarrow b \rightarrow c$ plus $a \rightarrow c$ share a reachability order but differ probabilistically (a noisy copy of A factorizes over the second, not the first). I therefore name the evidence-model DAG separately as $H = (U, F)$, with $\text{Pa}_H(v) = \{u : (u, v) \in F\}$, and condition on it explicitly. If the graph H is fixed and conditioned upon, the correct form is

$$L(x_U \mid H, M, G) = - \sum_{v \in U} \log_2 P_M(x_v \mid x_{\text{Pa}_H(v)}, H, G),$$

which additionally requires the Markov assumption $P_M(x_U \mid H, G) = \prod_v P_M(x_v \mid x_{\text{Pa}_H(v)}, H, G)$: the model must actually factorize over the graph, which does not follow from the graph's existence. If instead the graph structure is itself evidence, its model probability must be encoded:

$$P_M(D \mid G) = P_M(H \mid G) \prod_v P_M(x_v \mid x_{\text{Pa}_H(v)}, H, G).$$

The v0.1 formula silently discarded the graph's own description length; for a graph of probability 0.1 with node conditionals product 0.25, that is 3.3219 bits of structural evidence dropped. I take the first option: the evidence graph is conditioned upon, and the notation says so. Conditioning on H means the score measures predictive compression *given* that structure; it does not reward discovering or compressing H itself, and I state that limitation rather than presenting it as a contradiction.

Second, a terminological correction the reviewers forced: “causally independent” is used in this paper in two senses. In Postulate 2 it means poset incomparability (neither $a \prec b$ nor $b \prec a$). In the probabilistic section it would naturally mean statistical or interventional independence. These are different relations, and the provenance DAG (which receipts cite which) is not in general the data-generating causal DAG: imposing ledger structure as conditional-independence assumptions is false modeling. I keep the two graphs distinct and say which one I mean wherever both appear.

Third, the motivation. For a fixed model p_M , the chain rule joint $-\sum \log p(x_t \mid x_{<t})$ equals $-\log p(x_1..x_n)$ under any ordering; the joint is order-free. Temporal order does epistemic work only for prequential, adaptive predictors, where it keeps the code honest against peeking. I therefore state which case the Turing is: where the protocol evaluates a fixed fitted model, the timeless form loses nothing; where the protocol is prequential, the ordering is part of the anti-cheating apparatus and must be preserved as a declared causal constraint, not silently removed. The v0.1 text did not distinguish these, and the distinction matters.

With those corrections, the timeless Turing is

$$L(D \mid M, G, H) = -\log_2 P_M(D \mid G, H), \quad T(M; B, D, G, P) = [L(B \mid G) + L(D \mid B, G, H)] - [L(M \mid G) + L(D \mid M, G, H)]$$

with the evidence-model DAG H conditioned upon as above (absorbed into the shared background G wherever the protocol fixes it once for all candidates). The Turing requires evidence, not time. That claim survives the corrections unchanged.

10 Complexity: work and depth, stated honestly

The v0.1 text said that removing sequential steps entirely would “require new mathematics.” The reviewers corrected me: it does not. The weak reading, no wall-clock seconds, needs no change at all, because “polynomial time” in complexity theory already means a bound on computation steps, and I now state it as polynomial work $W(n) \leq n^k$. Three choices are independent and I keep them separate: (1) abstract versus wall-clock cost; (2) work/size versus depth; (3) uniform versus nonuniform computation. Removing a clock does not imply nonuniformity — uniform circuit families are atemporal DAG descriptions too — and $P/poly$ changes the uniformity and advice assumptions rather than removing causal stages, so I introduce it only where advice or nonuniform adversaries are an independently intended model. The current Claim A proof uses uniform polynomial-time adversaries, and the atemporal reading does not change that.

The Discovery Gap’s work bound is two-dimensional: work $W(n)$ and depth $S(n)$, the span of the longest causal chain. For unit-cost finite DAGs $S \leq W$ always, so polynomial work already bounds depth polynomially; the separate depth bound distinguishes levels of parallelizability, such as polylogarithmic versus polynomial depth. The postulate’s own framework provides the vocabulary to say so, and v1.0’s phrase “polynomial work with exponential depth” was impossible under the paper’s own definitions — I correct it openly rather than silently.

11 What changes in the existing papers, as versioned revisions

I keep the seven existing AIEN papers intact; what follows are stated revisions for future versions, not silent edits.

EXP-002. The diffusion worlds are Brownian motion with drift, $dX_t = \mu dt + \sqrt{2D} dW_t$ (v0.1 mislabeled this mean-reverting; mean reversion is Ornstein–Uhlenbeck). I propose replacing EXP-002 with relational and causal law discovery: inferring a hidden causal network from observations and interventions without an external clock. I withdraw the v0.1 claim that this is a “cleaner” experiment: observational data generally identify a DAG only up to Markov equivalence under standard assumptions, and temporal order is a key identifying assumption. Interventions can refine or eliminate that ambiguity — a perfect intervention on X distinguishes $X \rightarrow Y$ from $Y \rightarrow X$ where observations cannot — so identifiability depends on the intervention targets and the causal model’s assumptions, which the experiment specification must state. It is a different experiment that tests what AIEN claims to be, not an easier one.

T/s. Demoted to a physical realization metric belonging to FORGE and hardware. T , T per observation, T per causal event, and T per causal depth live inside Omega.

Stale receipts. Redefined as invalidation by changed environment, superseded dependency, or changed verification policy, not mere elapsed seconds. I note the mechanism question the reviewers raised: detecting “assumptions no longer match” requires observing the mismatch, which is itself future engineering, not claimed here.

The Turing definition. Revised as in Section 9: conditioned graph, Markov assumption explicit, prequential versus fixed-model distinguished.

12 Prior art and the narrowed novelty claim

The v0.1 novelty claim was argued against the wrong literature, and its physics invocations were unanimously judged a category error by the reviewers. I correct both.

On the physics: the causal-set program recovers Lorentzian geometry from locally finite orders plus counting, not computational dependency; Page–Wootters recovers a time parameter by conditioning, not a partial order, and faces known difficulties. I demote these to motivating analogies. The postulate depends only on order theory and concurrency semantics.

On the literature: partial orders as computational semantics are the core of concurrency theory, and I now argue against it directly. Kahn process networks (1974) with the determinacy theorem; Winskel event structures; Mazurkiewicz traces; Petri nets and occurrence nets, which pair causality with a conflict relation the current Omega lacks and will need for alternative or superseded receipts; the actor model; dataflow (Dennis, Arvind, Lucid); deterministic parallelism (LVars); vector clocks (Fidge, Mattern), which are the right comparison for Section 5’s construction since Lamport clocks extend happened-before to a total order rather than defining dependence; CRDTs and causal consistency; content-addressed builds (Nix, Bazel, Unison), whose methodology is the closest existing relative of the Section 8 experiment; W3C PROV for provenance vocabulary; Chandy–Lamport snapshots for cross-replica state; hybrid logical clocks and leases as the standard answer to staleness.

Given that landscape, I state the contribution as a proposed architectural integration, not an established novelty: making causal order the semantics and epistemology of a machine intelligence architecture, enforced by the verification and receipt machinery rather than used as a coordination mechanism between processes. I separate two things the v1.0 text blurred: implementation conformance, which the experiment in Section 8 can test, and architectural novelty, which requires comparison with prior systems and which no experiment of this form can establish. A hermetic deterministic build system could pass the same positive arm; the table below states what the architecture additionally claims to supply.

Prior system	What it already provides	What AIEN additionally claims
Nix / Bazel (hermetic builds)	Dependency graphs, reproducibility, environment isolation, content identity	Provenance connected to belief admission: verified receipts as the gate for what the machine may subsequently assert or do
Unison	Content-addressed program identity	Content-addressed <i>claims and evidence</i> , not just code
Proof-carrying code	Machine-checkable evidence attached to executables	Evidence attached to the evolving world-model, with revision as new crumbs rather than recompilation
LVars / deterministic parallelism	Schedule-independent observations via monotonicity	Schedule-independence as the enforced epistemology of an agent that plans and acts, not just computes
Kahn networks	Deterministic process networks over partial orders	The determinacy theorem’s obligations (deterministic, complete, commuting) made into per-receipt admission checks

Each row’s third column is proposed, not demonstrated. If the literature already contains any of these positions in full, I will cite it and narrow further. That is an architectural claim, and it stands or falls on whether the experiment in Section 8 passes and whether the comparison survives scrutiny.

I also correct v0.1’s Lamport characterization: Lamport clocks produce a total order consistent with causality; happened-before includes program order; and Lamport 1978 explicitly took the idea from relativity’s causal cones, so the physics analogy was in Lamport, not beyond him.

13 Objections

“This is just Kahn networks / Petri nets / dataflow.” The mathematics of partial-order semantics is indeed established, and Sections 5 through 8 now argue against that literature explicitly. What I claim as new is the architectural position: causal order as the enforced epistemology of the machine, with the receipt and verification machinery as its enforcement arm, in service of machine intelligence rather than process coordination. If the experiment passes, the claim has content; if the literature already contains this exact position, I will cite it and narrow further.

“Something still has to run first.” Agreed, and the postulate agrees. Implementation has sequence; meaning does not contain it. The admission sequence of a replica is a local total order, a linear extension of the causal order; the formalism relocates sequence rather than eliminating it, and Section 5 says so.

“The photon argument was wrong, so the whole thing collapses.” The photon framing was wrong and is dropped, along with the muddled Lorentz correction. The postulate depends on order theory and concurrency semantics, which stand independently.

“Schedule invariance is false; I have a counterexample.” Then the counterexample either

violates one of Theorem 1’s hypotheses or the realization refinement, in which case it is a negative control the verifier must reject for its specific violated obligation — or it satisfies all of them and still diverges, in which case the postulate fails and I will record that. A divergent run is a counterexample to the abstract claim only after the premises and the refinement have been established for that run; otherwise it is evidence about the implementation. The hypotheses are now stated precisely so that this objection is decidable.

“Science revises causal understanding; immutability forbids it.” Immutability forbids rewriting the record, not revising understanding. The revision is a new crumb asserting $A <_{\text{domain}} B$, citing the old admissions as evidence; it does not and cannot create $A <_{\text{ledger}} B$ retroactively. The notebook is never rewritten; the new paper cites it.

“Causal-cone admissibility is tautological.” As a slogan about declared dependencies, yes. As the noninterference property of Postulate 4, enforced by types and sandboxing, it is a checkable engineering requirement: vary everything outside the declared causal past and observe no change in behavior.

“This contradicts the seven papers.” It does not touch them. The seven papers stand at their versions and evidentiary statuses. This postulate is an eighth paper, and Section 11 lists its implied revisions as versioned future work, not edits.

“An untested postulate is just philosophy.” Fair, and Section 8 now specifies the experiment with an independent oracle, label-preserving pass criteria, and negative controls, instead of the near-tautological design of v0.1. The pass criteria are written before anyone runs it.

14 Status

Version 1.1. Proposed, not peer reviewed. The formalism of Sections 4 through 6 is now closed: the extension lemma is proved, the schedule-invariance theorem is stated with its hypotheses, and semantic identity has a canonical definition. The experiment is specified but not run. Five independent reviews (four outside models, one programmatic check relayed by the author) were folded into this version; their converged must-fix findings are all addressed, and the points of reviewer disagreement are recorded in the paper’s development notes rather than hidden. The seven AIEN papers are unchanged.

The line carried forward from v0.1 stands:

The Crumblin is not a timeline. It is a causal record.